

Lectures on Machine Learning

Jeremy Teitelbaum

2026-01-01

Table of contents

Preface	1
1 Linear Regression	3
1.1 Introduction	3
1.2 Least Squares (via Calculus)	4
1.2.1 Exercises	5
1.3 Least Squares (via Geometry)	5
1.3.1 Exercises	7
1.4 The Multivariate Case (Calculus)	7
1.5 The Multivariate Case (Geometry)	10
1.5.1 Orthogonal Projection	11
1.6 Centered coordinates	12
1.7 Caveats about Linear Regression	13
1.7.1 Basic considerations	13
1.7.2 Simpson's Effect	13
1.7.3 Exercises	14
2 Principal Component Analysis	15
2.1 Introduction	15
2.2 Variance and Covariance	15
2.2.1 Variance	15
2.2.2 Covariance	16
2.2.3 Correlation	17
2.2.4 The covariance matrix	18
2.2.5 Visualizing the covariance matrix	19
2.2.6 Linear Combinations of Features (Scores)	19
2.2.7 Mean and variance of scores	20
2.2.8 Geometry of Scores	21
2.3 Principal Components	24
2.3.1 Change of variance with direction	24
2.3.2 Directions of extremal variance	25
2.3.3 Critical values of the variance	27
2.3.4 Subspaces of extremal variance	29
2.3.5 Definition of Principal Components	32
2.4 Dimensionality Reduction via Principal Components	33
2.4.1 Loadings	35
2.4.2 The singular value decomposition	37
2.5 Eigenvalues and Eigenvectors of Real Symmetric Matrices (The Spectral Theorem)	39
2.5.1 Gram-Schmidt	39

2.5.2	The spectral theorem	40
3	Probability and Bayes Theorem	43
3.1	Introduction	43
3.2	Probability Basics	43
3.2.1	Discrete probability examples	44
3.2.2	Continuous probability examples	45
3.3	Conditional Probability and Bayes Theorem	45
3.3.1	Bayes Theorem	47
3.3.2	An example	47
3.4	Independence	50
3.4.1	Examples	50
3.5	Random Variables, Mean, and Variance	52
3.5.1	Independence and Random Variables	54
3.5.2	Expectation, Mean and Variance	54
3.6	Models and Likelihood	62
3.6.1	Maximum Likelihood (Discrete Case)	64
3.6.2	Maximum Likelihood (Continuous Case)	64
3.6.3	Linear Regression and likelihood	66
4	Bayesian Inference	67
4.1	Bayesian Perspective on Data	67
4.1.1	Bayesian experiments with the normal distribution	70
4.1.2	Bayesian coin flipping	72
4.1.3	Bayesian Regression (or Ridge Regression)	76
5	The Naive Bayes classification method	79
5.1	Introduction	79
5.2	An example dataset	79
5.3	Bernoulli tests	80
5.4	Feature vectors	82
5.5	Likelihood	83
5.6	The Bag of Words	84
5.7	Other applications	86
6	Gradient Descent	87
6.1	Introduction	87
6.2	The Key Idea	87
6.3	The Algorithm	88
6.3.1	Gradient Descent Algorithm	89
6.4	Linear Regression via Gradient Descent	92
6.4.1	Gradient Descent Algorithm for Linear Regression	92
6.5	Stochastic Gradient Descent	93
6.5.1	Stochastic Gradient Descent for Linear Regression	93

7	Logistic Regression	95
7.1	Likelihood and Logistic Regression	96
7.1.1	Another point of view on logistic regression	98
7.1.2	Logistic regression with multiple features	99
7.2	Finding the maximum likelihood solution by gradient descent	99
7.3	Gradient Descent and Logistic Regression	100
7.3.1	Gradient Descent on our synthetic data	101
7.3.2	Gradient Descent and Logistic Regression on “real” data	101
7.4	Logistic Regression and Classification	103
7.4.1	Weights as filters	105
7.5	Multiclass Logistic Regression	106
7.5.1	Multiclass logistic regression - the likelihood	108
7.5.2	Multiclass logistic regression - the gradient.	109
7.5.3	Multiclass Gradient Descent	111
7.6	Stochastic Gradient Descent for Logistic Regression	111
8	Support Vector Machines	113
8.1	Introduction	113
8.2	A simple example	113
8.3	The general case	115
8.3.1	Hyperplanes	115
8.3.2	Linear separability and Margins	117
8.4	Convexity, Convex Hulls, and Margins	120
8.5	Finding the Optimal Margin Classifier	126
8.5.1	Relaxing the constraints	128
8.5.2	Sequential Minimal Optimization	129
8.6	Inseparable Sets	131
8.6.1	Best Separating Hyperplanes	132
8.6.2	Nonlinear kernels	132
8.7	Exercises	133
9	Neural Networks	135
9.1	Introduction	135
9.2	Basics of Neural Networks	135
9.3	Graphical Representation of Neural Networks	136
9.4	Loss functions and training	141
9.5	Backpropagation	142
9.5.1	Backpropagation: first step	142
9.5.2	Backpropagation - inductive steps	143
9.5.3	Backpropagation - putting it together	146
9.5.4	Training	147
9.6	Convolution	147
9.6.1	Discrete Convolution for Image Data	155
9.6.2	Pooling	156
9.7	Convolutional Neural Networks	156
9.7.1	Convolution Layers	156

Table of contents

9.7.2	Pooling Layers	157
9.7.3	A neural net classifier	157
9.7.4	Training the network	159
9.7.5	Counting the weights	159
References		161

Preface

These notes are being developed for the UConn Math Department's undergraduate course on the Mathematics of Machine Learning, Math 3180.

$$\int_0^\infty f(x)dx$$

This is a (very) *rough draft*.

You can view this project on GitHub. Please let the author know if you have questions or find mistakes by opening an issue there.

1 Linear Regression

1.1 Introduction

Suppose studying two quantities x and y that we suspect are related – at least approximately – by a linear equation $y = ax + b$. Sometimes this linear relationship is predicted by theoretical considerations, and sometimes it is just an empirical hypothesis.

For example, if we are trying to determine the velocity of an object travelling towards us at constant speed, and we measure the distances $d_1, d_2, \dots, d_n$ between us and the object at a series of times $t_1, t_2, \dots, t_n$, then since “distance equals rate times time” we have a theoretical foundation for the assumption that $d = rt + b$ for some constants r and b . On the other hand, because of unavoidable experimental errors, we can’t expect that this relationship will hold exactly for the observed data; instead, we likely get a graph like that shown in Figure 1.1. We’ve drawn a line on the plot that seems to capture the true slope (and hence velocity) of the object.

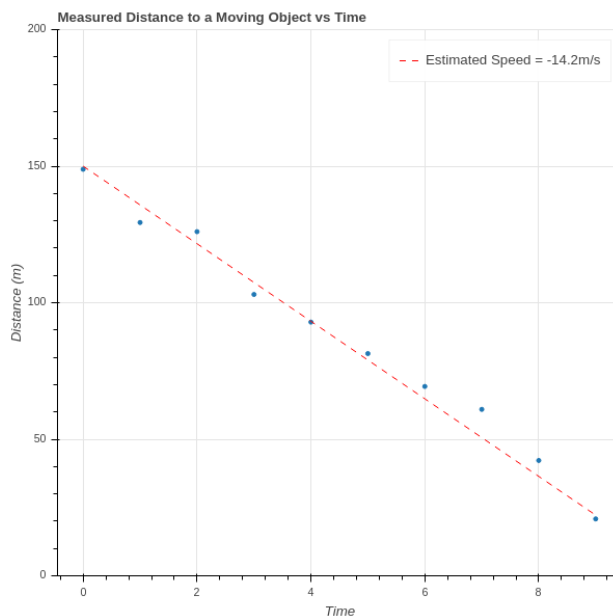


Figure 1.1: Physics Experiment

On the other hand, we might look at a graph such as Figure 1.2, which plots the gas mileage of various car models against their engine size (displacement), and observe a general trend in which bigger engines get lower mileage. In this situation we could ask for the best line of the form $y = mx + b$ that captures this relationship and then draw empirical conclusions about this data without necessarily having an underlying theory.

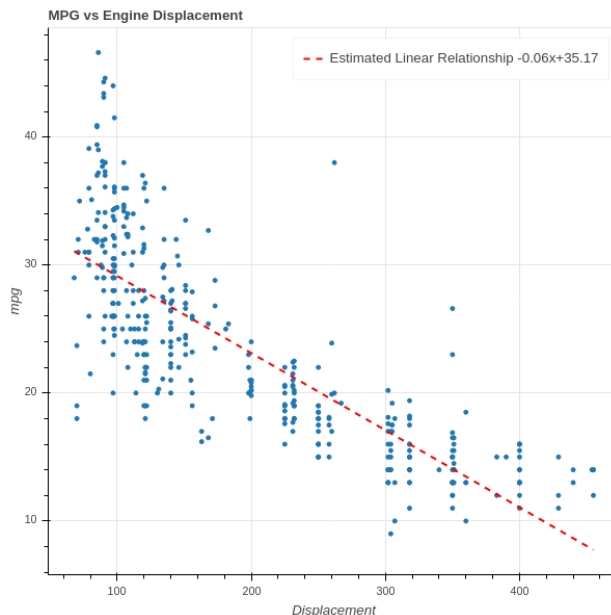


Figure 1.2: MPG vs Displacement ([1])

1.2 Least Squares (via Calculus)

In either of the two cases above, the question we face is to determine the line $y = mx + b$ that “best fits” the data $\{(x_i, Y_i)\}_{i=1}^N$. The classic approach is to determine the equation of a line $y = mx + b$ that minimizes the “mean squared error”:

$$MSE(m, b) = \frac{1}{N} \sum_{i=1}^N (Y_i - mx_i - b)^2$$

It’s worth emphasizing that the MSE is a function of two variables – the slope m and the intercept b – and that the data points $\{(x_i, Y_i)\}$ are constants for these purposes. Furthermore, it’s a quadratic function in those two variables. Since our goal is to find m and b that minimize the MSE , we have a Calculus problem that we can solve by taking partial derivatives and setting them to zero.

To simplify the notation, let’s abbreviate MSE by E .

$$\begin{aligned} \frac{\partial E}{\partial m} &= -\frac{1}{N} \sum_{i=1}^N 2x_i(Y_i - mx_i - b) \\ \frac{\partial E}{\partial b} &= -\frac{1}{N} \sum_{i=1}^N 2(Y_i - mx_i - b) \end{aligned}$$

We set these two partial derivatives to zero, so we can drop the -2 and regroup the sums to obtain two equations in two unknowns (we keep the $\frac{1}{N}$ because it is illuminating in the final result):

$$\begin{aligned}
\frac{1}{N} \left(\sum_{i=1}^N x_i^2 \right) m + \frac{1}{N} \left(\sum_{i=1}^N x_i \right) b &= \frac{1}{N} \sum_{i=1}^N x_i Y_i \\
\frac{1}{N} \left(\sum_{i=1}^N x_i \right) m + b &= \frac{1}{N} \sum_{i=1}^N Y_i
\end{aligned} \tag{1.1}$$

In these equations, notice that $\frac{1}{N} \sum_{i=1}^N x_i$ is the average (or mean) value of the x_i . Let's call this $\bar{x}$. Similarly, $\frac{1}{N} \sum_{i=1}^N Y_i$ is the mean of the Y_i , and we'll call it $\bar{Y}$. If we further simplify the notation and write S_{xx} for $\frac{1}{N} \sum_{i=1}^N x_i^2$ and S_{xy} for $\frac{1}{N} \sum_{i=1}^N x_i Y_i$ then we can write down a solution to this system using Cramer's rule:

$$\begin{aligned}
m &= \frac{S_{xy} - \bar{x}\bar{Y}}{S_{xx} - \bar{x}^2} \\
b &= \frac{S_{xx}\bar{Y} - S_{xy}\bar{x}}{S_{xx} - \bar{x}^2}
\end{aligned} \tag{1.2}$$

where we must have $S_{xx} - \bar{x}^2 \neq 0$.

1.2.1 Exercises

1. Verify that Equation 1.2 is in fact the solution to the system in Equation 1.1 .
2. Suppose that $S_{xx} - \bar{x}^2 = 0$. What does that mean about the x_i ? Does it make sense that the problem of finding the “line of best fit” fails in this case?

1.3 Least Squares (via Geometry)

In our discussion above, we thought about our data as consisting of N pairs (x_i, Y_i) corresponding to N points in the xy -plane $\mathbf{R}^2$. Now let's turn that picture “on its side”, and instead think of our data as consisting of *two* points in $\mathbf{R}^N$:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \text{ and } Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

Let's also introduce one other vector

$$E = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

1 Linear Regression

First, let's assume that E and X are linearly independent. If not, then X is a constant vector (why?), which we already know is a problem from Section 1.2, Exercise 2. Therefore E and X span a plane in $\mathbf{R}^N$.

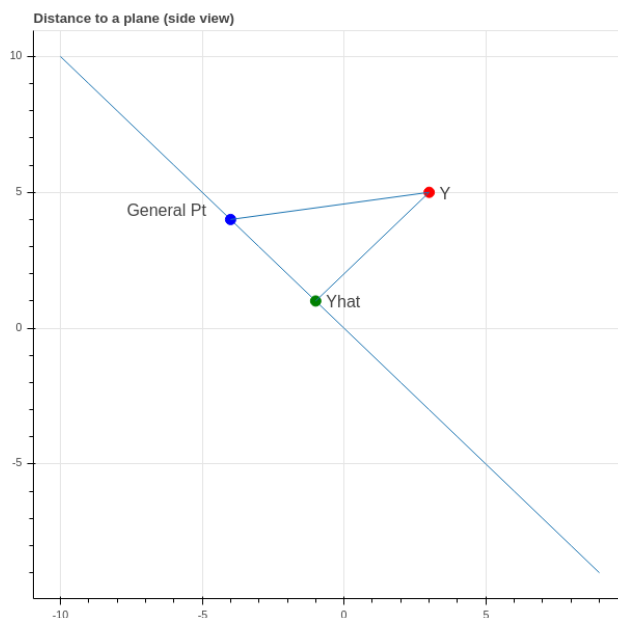


Figure 1.3: Distance to A Plane

Now if our data points (x_i, Y_i) all *did* lie on a line $y = mx + b$, then the three vectors X , Y , and E would be linearly dependent:

$$Y = mX + bE.$$

Since our data is only approximately linear, that's not the case. So instead we look for an approximate solution. One way to phrase that is to ask:

What is the point $\hat{Y}$ in the plane H spanned by X and E in $\mathbf{R}^N$ which is closest to Y ?

If we knew this point $\hat{Y}$, then since it lies in H we would have $\hat{Y} = mX + bE$ and the coefficients m and b would be a candidate for defining a line of best fit $y = mx + b$. Finding the point in a plane closest to another point in $\mathbf{R}^N$ is a geometry problem that we can solve.

Proposition: The point $\hat{Y}$ in the plane spanned by X and E is the point such that the vector $Y - \hat{Y}$ is perpendicular to H .

Proof: See Figure 1.3 for an illustration – perhaps you are already convinced by this, but let's be careful. We seek $\hat{Y} = mX + bE$ such that

$$D = \|Y - \hat{Y}\|^2 = \|Y - mX - bE\|^2$$

is minimal. Using some vector calculus, we have

$$\frac{\partial D}{\partial m} = \frac{\partial}{\partial m} (Y - mX - bE) \cdot (Y - mX - bE) = -2(Y - mX - bE) \cdot X$$

and

$$\frac{\partial D}{\partial b} = \frac{\partial}{\partial b}(Y - mX - bE) \cdot (Y - mX - bE) = -2(Y - mX - bE) \cdot E.$$

So both derivatives are zero exactly when $Y - \hat{Y} = (Y - mX - bE)$ is orthogonal to both X and E , and therefore every vector in H .

We also obtain equations for m and b just as in our first look at this problem.

$$\begin{aligned} m(X \cdot E) + b(E \cdot E) &= (Y \cdot E) \\ m(X \cdot X) + b(E \cdot X) &= (Y \cdot X) \end{aligned} \tag{1.3}$$

We leave it as an exercise below to check that these are the same equations that we obtained in Equation 1.2.

1.3.1 Exercises

1. Verify that Equation 1.2 and Equation 1.3 are equivalent.

1.4 The Multivariate Case (Calculus)

Having worked through the problem of finding a “line of best fit” from two points of view, let’s look at a more general problem. We looked above at a scatterplot showing the relationship between gas mileage and engine size (displacement). There are other factors that might contribute to gas mileage that we want to consider as well – for example:

- a car that is heavy compared to its engine size may get worse mileage
- a sports car with a drive train that gives fast acceleration as compared to a car with a transmission designed for long trips may have different mileage for the same engine size.

Suppose we wish to use engine displacement, vehicle weight, and acceleration all together to predict mileage. Instead of looking at points (x_i, Y_i) where x_i is the displacement of the i^{th} car model and we try to predict a value y from a corresponding x as $y = mx + b$ – let’s look at a situation in which our measured value y depends on multiple variables – say displacement d , weight w , and acceleration a with $k = 3$ – and we are trying to find the best linear equation

$$y = M_1d + M_2w + M_3a + b \tag{1.4}$$

But to handle this situation more generally we need to adopt a convention that will allow us to use indexed variables instead of d , w , and a . We will use the *tidy* data convention.

Tidy Data: A dataset is tidy if it consists of values x_{ij} for $i = 1, \dots, N$ and $j = 1, \dots, k$ so that:

- the row index corresponds to a *sample* – a set of measurements from a single event or item;

1 Linear Regression

- the column index corresponds to a *feature* – a particular property measured for all of the events or items.

In our case,

- the *samples* are the different types of car models,
- the *features* are the properties of those car models.

For us, N is the number of different types of cars, and k is the number of properties we are considering. Since we are looking at displacement, weight, and acceleration, we have $k = 3$.

So the “independent variables” for a set of data that consists of N samples, and k measurements for each sample, can be represented by a $N \times k$ matrix

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1k} \\ x_{21} & x_{22} & \cdots & x_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{Nk} \end{pmatrix}$$

and the measured dependent variables Y are a column vector

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}.$$

If $M_1, \dots, M_k$ are “slopes” associated with these properties in Equation 1.4, and b is the “intercept”, then the predicted value $\hat{Y}$ is given by a matrix equation

$$\hat{Y} = X \begin{bmatrix} M_1 \\ M_2 \\ \cdots \\ M_k \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ \cdots \\ 1 \end{bmatrix} b$$

and our goal is to choose these parameters M_i and b to make the mean squared error:

$$MSE(M_1, \dots, M_k, b) = \|Y - \hat{Y}\|^2 = \sum_{i=1}^N (Y_i - \sum_{j=1}^k x_{ij} M_j - b)^2.$$

Here we are summing over the N different car models, and for each model taking the squared difference between the true mileage Y_i and the “predicted” mileage $\sum_{j=1}^k x_{ij} M_j + b$. We wish to minimize this MSE.

Let’s make one more simplification. The intercept variable b is annoying because it requires separate treatment from the M_i . But we can use a trick to eliminate the need for special treatment. Let’s add a new feature to our data matrix (a new column) that has the constant value 1.

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1k} & 1 \\ x_{21} & x_{22} & \cdots & x_{2k} & 1 \\ \vdots & \vdots & \ddots & \vdots & 1 \\ x_{N1} & x_{N2} & \cdots & x_{Nk} & 1 \end{pmatrix}$$

Now our data matrix X is $N \times (k+1)$ and we can put our “intercept” $b = M_{k+1}$ into our vector of “slopes” $M_1, \dots, M_k, M_{k+1}$:

$$\hat{Y} = X \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_k \\ M_{k+1} \end{bmatrix}$$

and our MSE becomes

$$MSE(M) = \|Y - XM\|^2$$

where

$$M = \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_k \\ M_{k+1} \end{bmatrix}.$$

Remark: Later on (see {Section 1.6}) we will see that if we “center” our features about their mean, by subtracting the average value of each column of X from that column; and we also subtract the average value of Y from the entries of Y , then the b that emerges from the least squares fit is zero. As a result, instead of adding a column of 1’s, you can change coordinates to center each feature about its mean, and keep your X matrix $N \times k$.

The Calculus approach to minimizing the MSE is to take its partial derivatives with respect to the M_i and set them to zero. Let’s first work out the derivatives in a nice form for later.

Proposition: The gradient of $MSE(M) = E$ is given by

$$\nabla E = \begin{bmatrix} \frac{\partial}{\partial M_1} E \\ \frac{\partial}{\partial M_2} E \\ \vdots \\ \frac{\partial}{\partial M_{k+1}} E \end{bmatrix} = -2X^\top Y + 2X^\top XM \quad (1.5)$$

where $X^\top$ is the transpose of X .

Proof:

1 Linear Regression

Since

$$E = \sum_{j=1}^N (Y_j - \sum_{s=1}^{k+1} X_{js} M_s)^2$$

we can use the chain rule to compute:

$$\frac{\partial}{\partial M_t} E = -2 \sum_{j=1}^N X_{jt} (Y_j - \sum_{s=1}^{k+1} X_{js} M_s) \quad (1.6)$$

If we look closely at this expression, we can see that it is the dot product of the t^{th} column of the $N \times (k+1)$ matrix X with the N -dimensional column vector $(Y - XM)$. Given the structure of the gradient as a column vector, this means

$$\nabla E = -2X^\top(Y - XM) = -2X^\top Y + 2X^\top XM$$

as claimed.

Proposition: Assume that $D = X^\top X$ is invertible (notice that it is a $(k+1) \times (k+1)$ square matrix so this makes sense). The solution M to the multivariate least squares problem is

$$M = D^{-1} X^\top Y \quad (1.7)$$

and the “predicted value” $\hat{Y}$ for Y is

$$\hat{Y} = XD^{-1} X^\top Y. \quad (1.8)$$

1.5 The Multivariate Case (Geometry)

Let’s look more closely at the equation obtained by setting the gradient of the error, Equation 1.5, to zero. Remember that M is the unknown vector in this equation, everything else is known:

$$X^\top Y = X^\top XM$$

Here is how to think about this:

1. As M varies, the $N \times 1$ matrix XM varies over the space spanned by the columns of the matrix X . So as M varies, XM is a general element of the subspace H of $\mathbf{R}^N$ spanned by the $k+1$ columns of X .
2. The product $X^\top XM$ is a $(k+1) \times 1$ matrix. Each entry is the dot product of the general element of H with one of the $k+1$ basis vectors of H .
3. The product $X^\top Y$ is a $(k+1) \times 1$ matrix whose entries are the dot product of the basis vectors of H with Y .

Therefore, this equation asks for us to find M so that the vector XM in H has the same dot products with the basis vectors of H as Y does. The condition

$$X^\top(Y - XM) = 0$$

says that $Y - XM$ is orthogonal to H . This argument establishes the following proposition.

Proposition: Just as in the simple one-dimensional case, the predicted value $\hat{Y}$ of the least squares problem is the point in H closest to Y – or in other words the point $\hat{Y}$ in H such that $Y - \hat{Y}$ is perpendicular to H .

1.5.1 Orthogonal Projection

Recall that we introduced the notation $D = X^\top X$, and let's assume, for now, that D is an invertible matrix. We have the formula (see Equation 1.8):

$$\hat{Y} = XD^{-1}X^\top Y.$$

Proposition: The matrix $P = XD^{-1}X^\top$ is an $N \times N$ matrix called the orthogonal projection operator onto the subspace H spanned by the columns of X . It has the following properties:

- PY belongs to the subspace H for any $Y \in \mathbf{R}^N$.
- $(Y - PY)$ is orthogonal to H .
- $PP = P$.

Proof: First of all, $PY = XD^{-1}X^\top Y$ so PY is a linear combination of the columns of X and is therefore an element of H . Next, we can compute the dot product of PY with each basis vector of H by computing

$$X^\top PY = X^\top XD^{-1}X^\top Y = X^\top Y$$

since $X^\top X = D$. This equation means that $X^\top(Y - PY) = 0$ which tells us that $Y - PY$ has dot product zero with each basis vector of H . Finally,

$$PP = XD^{-1}X^\top XD^{-1}X^\top = XD^{-1}X^\top = P.$$

It should be clear from the above discussion that the matrix $D = X^\top X$ plays an important role in the study of this problem. In particular it must be invertible or our analysis above breaks down. In the next section we will look more closely at this matrix and what information it encodes about our data.

1.6 Centered coordinates

In our discussion above, we introduced an artificial column of ones into our data matrix. This accounts for the intercept (or “bias term”) b in the linear equation

$$y = M_1x_1 + \cdots + M_kx_k + b.$$

An alternative to this approach is to make a simple change of coordinates in our data so that the fitted bias becomes zero. Then we can ignore it and just work with k features and weights.

Remember the basic equation

$$X^\top(Y - XM) = 0$$

that determines the solution to the least squares problem. Looking at the last row of this equation, and using that the last column of X is the N -dimensional column vector E consisting entirely of ones, we see that at the solution point,

$$E \cdot (Y - XM) = 0.$$

But $E \cdot Y = \sum_{i=1}^N Y_i$ is the sum of the target values. Furthermore, since XM is the vector $\hat{Y}$ of predicted values, $E \cdot XM$ is the sum of the predicted values. Dividing by N , we see that, at the least squares solution point, the mean of Y equals the mean of $\hat{Y}$.

Averaging the component equations for $\hat{Y}_i$ gives

$$\hat{Y}_i = x_{i1}M_1 + x_{i2}M_2 + \cdots + x_{ik}M_k + b$$

and dividing by N tells us that

$$\overline{\hat{Y}} = \bar{x}_1M_1 + \bar{x}_2M_2 + \cdots + \bar{x}_kM_k + b$$

where $\bar{x}_i$ is the mean of the i^{th} feature. As a result, we see that the bias b is

$$b = \bar{Y} - \sum_{i=1}^k \bar{x}_i M_i$$

We can exploit this by a “trick”. Before we start our analysis, we can change coordinates for both the features and the target so that the mean of the target data, and the mean of each feature, is zero. This amounts to a “shift” in the measurement scale of each feature, so it’s essentially harmless.

If we begin with this shift, then we know *a priori* that the bias is zero, and so there’s no need to include it in our calculations and there’s no need to introduce a column of ones into our data matrix.

Everything else we did remains the same, but we have an $N \times k$ matrix to work with instead of $N \times (k + 1)$.

1.7 Caveats about Linear Regression

1.7.1 Basic considerations

Reflecting on our long discussion up to this point, we should take note of some of the potential pitfalls that lurk in the use of linear regression.

1. When we apply linear regression, we are explicitly assuming that the variable Y is associated to X via linear equations. This is a big assumption!
2. When we use multilinear regression, we are assuming that changes in the different features have independent effects on the target variable Y . In other words, suppose that $Y = ax_1 + bx_2$. Then an increase of x_1 by 1 increases Y by a , and an increase of x_2 by 1 increases Y by b . These effects are independent of one another and combine to yield an increase of $a + b$.
3. We showed in our discussion above that the linear regression problem has a solution when the matrix $D = X^\top X$ is invertible, and this happens when the columns of X are linearly independent. When working with real data, which is messy, we could have a situation in which the features we are studying are, in fact, dependent – but because of measurement error, the samples that we collected need not be. As a result, the matrix D will be “close” to being non-invertible, although formally still invertible. In this case, computing D^{-1} leads to numerical instability and the solution we obtain is very unreliable.

1.7.2 Simpson’s Effect

Simpson’s effect is a famous phenomenon that illustrates that linear regression can be very misleading in some circumstances. It is often a product of “pooling” results from multiple experiments. Suppose, for example, that we are studying the relationship between a certain measure of blood chemistry and an individual’s weight gain or loss on a particular diet. We do our experiments in three labs, the blue, green, and red labs. Each lab obtains similar results – higher levels of the blood marker correspond to greater weight gain, with a regression line of slope around 1. However, because of differences in the population that each lab is studying, some populations are more susceptible to weight gain and so the red lab sees a mean increase of almost 9 lbs while the blue lab sees a weight gain of only 3 lbs on average.

The three groups of scientists pool their results to get a larger sample size and do a new regression. Surprise! Now the regression line has slope -1.6 and increasing amounts of the marker seem to lead to *less* weight gain!

This is called Simpson’s effect, or Simpson’s paradox, and it shows that unknown factors (confounding factors) may cause linear regression to yield misleading results. This is particularly true when data from experiments conducted under different conditions are combined; in this case, the differences in experimental settings, called *batch effects*, can throw off the analysis very dramatically. See Figure 1.4.

1 Linear Regression

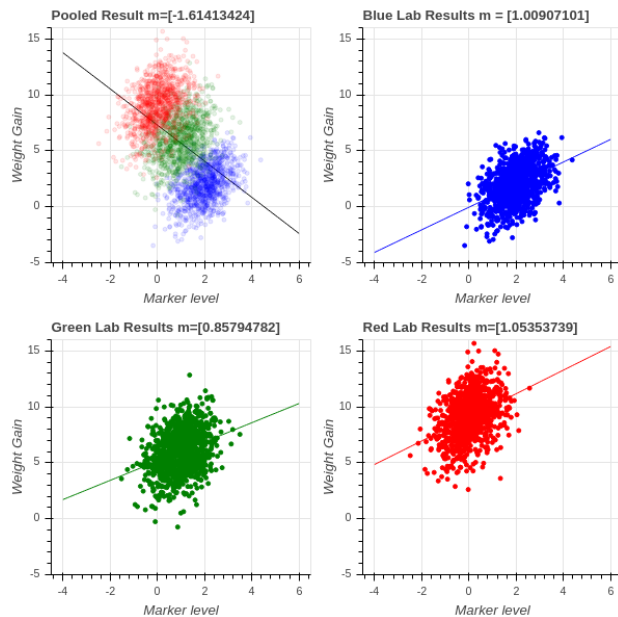


Figure 1.4: Simpson's Effect

1.7.3 Exercises

1. When proving that D is invertible if and only if the columns of X are linearly independent, we argued that if $X^\top X m = 0$ for a nonzero vector m , then Xm is orthogonal to the span of the columns of X , and is also an element of that span, and therefore it is zero. Provide the details: show that if H is a subspace of $\mathbf{R}^N$, and x is a vector in H such that $x \cdot h = 0$ for all $h \in H$, then $x = 0$.

2 Principal Component Analysis

2.1 Introduction

Suppose that, as usual, we begin with a collection of measurements of different features for a group of samples. Some of these measurements will tell us quite a bit about the difference among our samples, while others may contain relatively little information. For example, if we are analyzing the effect of a certain weight loss regimen on a group of people, the age and weight of the subjects may have a great deal of influence on how successful the regimen is, while their blood pressure might not. One way to help identify which features are more significant is to ask whether or not the feature varies a lot among the different samples. If nearly all the measurements of a feature are the same, it can't have much power in distinguishing the samples, while if the measurements vary a great deal then that feature has a chance to contain useful information.

In this section we will discuss a way to measure the variability of measurements and then introduce principal component analysis (PCA). PCA is a method for finding which linear combinations of measurements have the greatest variability and therefore might contain the most information. It also allows us to identify combinations of measurements that don't vary much at all. Combining this information, we can sometimes replace our original system of features with a smaller set that still captures most of the interesting information in our data, and thereby find hidden characteristics of the data and simplify our analysis a great deal.

2.2 Variance and Covariance

2.2.1 Variance

Suppose that we have a collection of measurements $(x_1, \dots, x_N)$ of a particular feature X . For example, x_i might be the initial weight of the i th participant in our weight loss study. The mean of the values $(x_1, \dots, x_N)$ is

$$\mu_X = \frac{1}{N} \sum_{i=1}^N x_i.$$

The simplest measure of the variability of the data is called its *variance*.

Definition: The (sample) variance of the data $x_1, \dots, x_N$ is

$$\sigma_X^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_X)^2 = \frac{1}{N} \left(\sum_{i=1}^N x_i^2 \right) - \mu_X^2 \quad (2.1)$$

The square root of the variance is called the *standard deviation*.

As we see from the formula, the variance is a measure of how ‘spread out’ the data is from the mean.

Recall that in our discussion of linear regression we thought of our set of measurements $x_1, \dots, x_N$ as a vector – it’s one of the columns of our data matrix. From that point of view, the variance has a geometric interpretation – it is $\frac{1}{N}$ times the square of the distance from the point $X = (x_1, \dots, x_N)$ to the point $\mu_X(1, 1, \dots, 1) = \mu_X E$:

$$\sigma_X^2 = \frac{1}{N} (X - \mu_X E) \cdot (X - \mu_X E) = \frac{1}{N} \|X - \mu_X E\|^2. \quad (2.2)$$

2.2.2 Covariance

The variance measures the dispersion of measures of a single feature. Often, we have measurements of multiple features and we might want to know something about how two features are related. The *covariance* is a measure of whether two features tend to be related, in the sense that when one increases, the other one increases; or when one increases, the other one decreases.

Definition: Given measurements $(x_1, \dots, x_N)$ and $(y_1, \dots, y_N)$ of two features X and Y , the covariance of X and Y is

$$\sigma_{XY} = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_X)(y_i - \mu_Y) \quad (2.3)$$

There is a nice geometric interpretation of this, as well, in terms of the dot product. If $X = (x_1, \dots, x_N)$ and $Y = (y_1, \dots, y_N)$ then

$$\sigma_{XY} = \frac{1}{N} ((X - \mu_X E) \cdot (Y - \mu_Y E)).$$

From this point of view, we can see that σ_{XY} is positive if the $X - \mu_X E$ and $Y - \mu_Y E$ vectors “point roughly in the same direction” and its negative if they “point roughly in the opposite direction.”

2.2.3 Correlation

One problem with interpreting the variance and covariance is that we don't have a scale – for example, if σ_{XY} is large and positive, then we'd like to say that X and Y are closely related, but it could be just that the entries of $X - \mu_X E$ and $Y - \mu_Y E$ are large. Here, though, we can really take advantage of the geometric interpretation. Recall that the dot product of two vectors satisfies the formula

$$a \cdot b = \|a\| \|b\| \cos(\theta)$$

where θ is the angle between a and b . So

$$\cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}.$$

Let's apply this to the variance and covariance, by noticing that

$$\frac{(X - \mu_X E) \cdot (Y - \mu_Y E)}{\|(X - \mu_X E)\| \|(Y - \mu_Y E)\|} = \frac{\sigma_{XY}}{\sigma_{XX} \sigma_{YY}}$$

so the quantity

$$r_{XY} = \frac{\sigma_{XY}}{\sigma_X \sigma_Y} \tag{2.4}$$

measures the cosine of the angle between the vectors $X - \mu_X E$ and $Y - \mu_Y E$.

Definition: The quantity r_{XY} defined in Equation 2.4 is called the (sample) *correlation coefficient* between X and Y . We have $0 \leq |r_{XY}| \leq 1$ with $r_{XY} = \pm 1$ if and only if the two vectors $X - \mu_X E$ and $Y - \mu_Y E$ are collinear in $\mathbf{R}^N$.

*Figure 2.1 illustrates data with different values of the correlation coefficient.

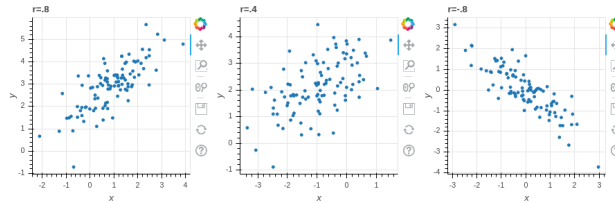


Figure 2.1: Correlation

2.2.4 The covariance matrix

In a typical situation we have many features for each of our (many) samples, that we organize into a data matrix X . To recall, each column of X corresponds to a feature that we measure, and each row corresponds to a sample. For example, each row of our matrix might correspond to a person enrolled in a study, and the columns correspond to height (cm), weight (kg), systolic blood pressure, and age (in years):

Table 2.1: A sample data matrix X

sample	Ht	Wgt	Bp	Age
A	180	75	110	35
B	193	80	130	40
...	...	...	...	...
U	150	92	105	55

If we have multiple features, as in this example, we might be interested in the variance of each feature and all of their mutual covariances. This “package” of information can be obtained “all at once” by taking advantage of some matrix algebra.

Definition: Let X be a $N \times k$ data matrix, where the k columns of X correspond to different features and the N rows to different samples. Let X_0 be the centered version of this data matrix, obtained by subtracting the mean μ_i of column i from all the entries x_{si} in that column. Then the $k \times k$ symmetric matrix

$$D_0 = \frac{1}{N} X_0^\top X_0$$

is called the (sample) covariance matrix for the data.

Proposition: The diagonal entries d_{ii} of D_0 are the variances of the columns of X :

$$d_{ii} = \sigma_i^2 = \frac{1}{N} \sum_{s=1}^N (x_{si} - \mu_i)^2$$

and the off-diagonal entries $d_{ij} = d_{ji}$ are the covariances of the i^{th} and j^{th} columns of X :

$$d_{ij} = \sigma_{ij} = \frac{1}{N} \sum_{s=1}^N (x_{si} - \mu_i)(x_{sj} - \mu_j)$$

The sum of the diagonal entries, the trace of D_0 is the **total** variance of the data.

Proof: This follows from the definitions, but it’s worth checking the details, which we leave as an exercise.

2.2.5 Visualizing the covariance matrix

If the number of features in the data is not too large, a density matrix plot provides a tool for visualizing the covariance matrix of the data. A density matrix plot is an $k \times k$ grid of plots (where k is the number of features). The entry with (i, j) coordinates in the grid is a scatter plot of the i^{th} feature against the j^{th} one if $i \neq j$, and is a histogram of the i^{th} variable if $i = j$.

*Figure 2.2 is an example of a density matrix plot for a dataset with 50 samples and 2 features. This data has been centered, so it can be represented in a 50×2 data matrix X_0 . The upper left and lower right graphs are scatter plots of the two columns, while the lower left and upper right are the histograms of the columns.

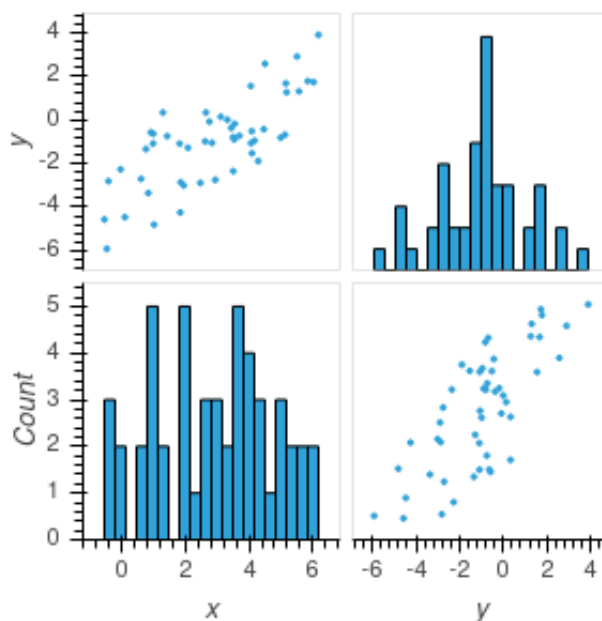


Figure 2.2: Density Matrix Plot

2.2.6 Linear Combinations of Features (Scores)

Sometimes useful information about our data can be revealed if we combine different measurements together to obtain a “hybrid” measure that captures something interesting. For example, in the Auto MPG dataset that we studied in the section on Linear Regression, we looked at the influence of both vehicle weight w and engine displacement e on gas mileage; perhaps there is some value in considering a hybrid “score” defined as

$$S = aw + be$$

for some constants a and b – maybe by choosing a good combination we could find a better predictor of gas mileage than using one or the other of the features individually.

As another example, suppose we are interested in the impact of the nutritional content of food on weight gain in a study. We know that both calorie content and the level dietary fiber contribute to

2 Principal Component Analysis

the weight gain of participants eating this particular food; maybe there is some kind of combined “calorie/fiber” score we could introduce that captures the impact of that food better.

Finally, when we assign grades in a course, we typically compute a weighted combination of the scores of each student on a series of assignments. Such a combination is another example of a “score” and may help explain the origin of the term.

Definition: Let X_0 be a (centered) $N \times k$ data matrix giving information about k features for each of N samples. A linear synthetic feature, or a linear score, is a linear combination of the k features. The linear score is defined by constants $a_1, \dots, a_k$ so that If $y_1, \dots, y_k$ are the values of the features for a particular sample, then the linear score for that sample is

$$S = a_1 y_1 + a_2 y_2 + \dots + a_k y_k$$

Lemma: The values of the linear score for each of the N samples can be calculated as

$$\begin{bmatrix} S_1 \\ \vdots \\ S_N \end{bmatrix} = X_0 \begin{bmatrix} a_1 \\ \vdots \\ a_k \end{bmatrix}. \quad (2.5)$$

Proof: Multiplying a matrix by a column vector computes a linear combination of the columns – that’s what this lemma says. Exercise 3 asks you to write out the indices and make sure you believe this.

2.2.7 Mean and variance of scores

When we combine features to make a hybrid score, we assume that the features were centered to begin with, so that each features has mean zero. As a result, the mean of the hybrid features is again zero.

Lemma: A linear combination of features with mean zero again has mean zero.

Proof: Let S_i be the score for the i^{th} sample, so

$$S_i = \sum_{j=1}^k x_{ij} a_j.$$

where X_0 has entries x_{ij} . Then the mean value of the score is

$$\mu_S = \frac{1}{N} \sum_{i=1}^N S_i = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k x_{ij} a_j.$$

Reversing the order of the sum yields

$$\mu_S = \frac{1}{N} \sum_{j=1}^k \sum_{i=1}^N x_{ij} a_j = \sum_{j=1}^k a_j \frac{1}{N} \left(\sum_{i=1}^N x_{ij} \right) = \sum_{j=1}^k a_j \mu_j = 0$$

where $\mu_j = 0$ is the mean of the j^{th} feature (column) of X_0 .

The variance is more interesting, and gives us an opportunity to put the covariance matrix to work. Remember from Equation 2.2 that, since a score S has mean zero, its variance is $\sigma_S^2 = \frac{1}{N} S \cdot S$ – where here the score S is represented by the column vector with entries $S_1, \dots, S_k$ as in Equation 2.5.

Lemma: The variance of the score S with weights $a_1, \dots, a_k$ is

$$\sigma_S^2 = a^\top D_0 a = \begin{bmatrix} a_1 & \cdots & a_k \end{bmatrix} D_0 \begin{bmatrix} a_1 \\ \vdots \\ a_k \end{bmatrix} \quad (2.6)$$

More generally, if S_1 and S_2 are scores with weights $a_1, \dots, a_k$ and $b_1, \dots, b_k$ respectively, then the covariance $\sigma_{S_1 S_2}$ is

$$\sigma_{S_1 S_2} = a^\top D_0 b.$$

Proof: From Equation 2.2 and Equation 2.5 we know that

$$\sigma_S^2 = \frac{1}{N} S \cdot S$$

and

$$S = X_0 a.$$

Since $\frac{1}{N} S \cdot S = \frac{1}{N} S^\top S$, this gives us

$$\frac{1}{N} \sigma_S^2 = \frac{1}{N} (X_0 a)^\top (X_0 a) = \frac{1}{N} a^\top X_0^\top X_0 a = a^\top D_0 a$$

as claimed.

For the covariance, use a similar argument with Equation 2.3 and Equation 2.5. writing $\sigma_{S_1 S_2} = \frac{1}{N} S_1 \cdot S_2$ and the fact that S_1 and S_2 can be written as $X_0 a$ and $X_0 b$.

The point of this lemma is that the covariance matrix contains not just the variances and covariances of the original features, but also enough information to construct the variances and covariances for *any linear combination of features*.

In the next section we will see how to exploit this idea to reveal hidden structure in our data.

2.2.8 Geometry of Scores

Let's return to the dataset that we looked at in Section 2.2.5. We simplify the density matrix plot in Figure 2.3, which shows one of the scatter plots and the two histograms.

The scatter plot shows that the data points are arranged in a more or less elliptical cloud oriented at an angle to the xy -axes which represent the two given features. The two individual histograms show the distribution of the two features – each has mean zero, with the x -features distributed between -2 and 2 and the y feature between -4 and 4 . Looking just at the two features individually, meaning only at the two histograms, we can't see the overall elliptical structure.

2 Principal Component Analysis

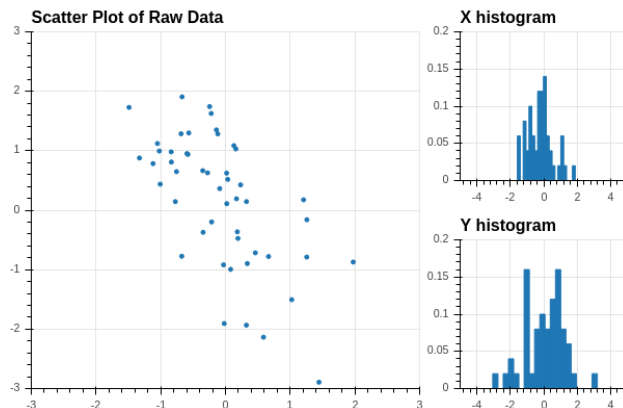


Figure 2.3: Simulated Data with Two Features

How can we get a better grip on our data in this situation? We can try to find a “direction” in our data that better illuminates the variation of the data. For example, suppose that we pick a unit vector at the origin pointing in a particular direction in our data. See Figure 2.4.

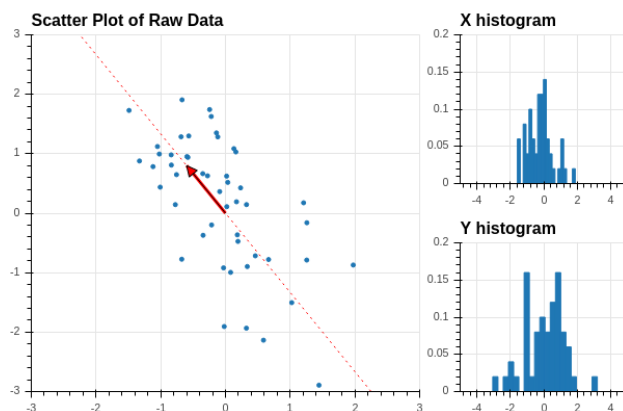


Figure 2.4: A direction in the data

Now we can orthogonally project the datapoints onto the line defined by this vector, as shown in Figure 2.5.

Recall that if the unit vector is defined by coordinates $u = [u_1, u_2]$, then the orthogonal projection of the point x with coordinates (x_1, x_2) is $(x \cdot u)u$. Now

$$x \cdot u = u_1 x_1 + u_2 x_2$$

so the coordinates of the points along the line defined by u are the values of the score Z defined by $u = [u_1, u_2]$. Using our work in the previous section, we see that we can find all of these coordinates by matrix multiplication:

$$Z = X_0 u$$

where X_0 is our data matrix. Now let’s add a histogram of the values of Z to our picture:

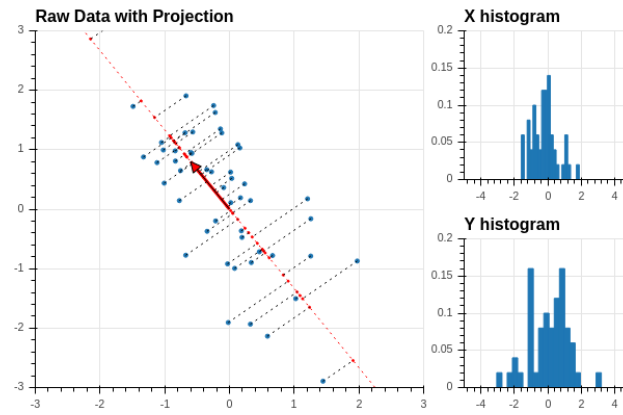


Figure 2.5: Projecting the datapoints

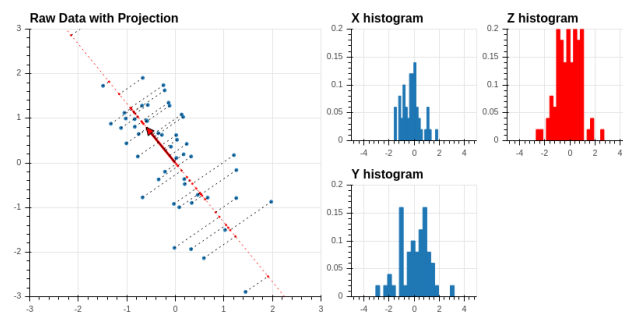


Figure 2.6: Distribution of Z

2 Principal Component Analysis

This histogram shows the distribution of the values of Z along the tilted line defined by the unit vector u .

Finally, using our work on the covariance matrix, we see that the variance of Z is given by

$$\sigma_Z^2 = \frac{1}{50} u^\top X_0^\top X_0 u = u^\top D_0 u$$

where D_0 is the covariance matrix of the data X_0 .

Lemma: Let X_0 be a $N \times k$ centered data matrix, and let $D_0 = \frac{1}{N} X_0^\top X_0$ be the associated covariance matrix. Let u be a unit vector in “feature space” $\mathbf{R}^k$. Then the score $S = X_0 u$ can be interpreted as the coordinates of the points of X_0 projected onto the line generated by u . The variance of this score is

$$\sigma_S^2 = u^\top D_0 u = \sum_{i=1}^N s_i^2$$

where $s_i = X_0[i, :] u$ is the dot product of the i^{th} row $X_0[i, :]$ with u . It measures the variability in the data “in the direction of the unit vector u ”.

2.3 Principal Components

2.3.1 Change of variance with direction

As we’ve seen in the previous section, if we choose a unit vector u in the feature space and find the projection $X_0 u$ of our data onto the line through u , we get a “score” that we can use to measure the variance of the data in the direction of u . What happens as we vary u ?

To study this question, let’s continue with our simulated data from the previous section, and introduce a unit vector

$$u(\theta) = [\cos(\theta) \quad \sin(\theta)] .$$

This is in fact a unit vector, since $\sin^2(\theta) + \cos^2(\theta) = 1$, and it is oriented at an angle θ from the x -axis.

The variance of the data in the direction of $u(\theta)$ is given by

$$\sigma_\theta^2 = u(\theta)^\top D_0 u(\theta).$$

A plot of this function for the data we have been considering is in Figure 2.7. As you can see, the variance goes through two full periods with the angle, and it reaches a maximum and minimum value at intervals of $\pi/2$ – so the two angles where the variance are maximum and minimum are orthogonal to one another.

The two directions where the variance is maximum and minimum are drawn on the original data scatter plot in Figure 2.8 .

Let’s try to understand why this is happening.

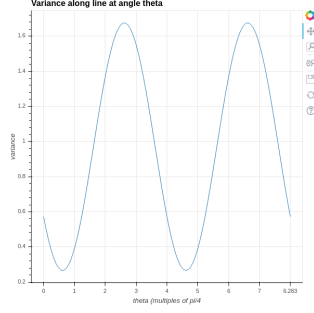


Figure 2.7: Change of variance with angle theta

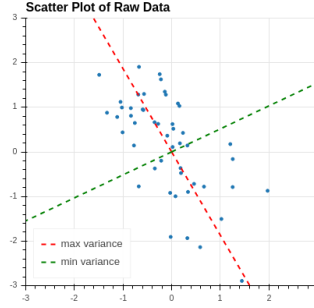


Figure 2.8: Data with principal directions

2.3.2 Directions of extremal variance

Given our centered, $N \times i$ data matrix X_0 , with its associated covariance matrix $D_0 = \frac{1}{N}X_0^\top X_0$, we would like to find unit vectors u in $\mathbf{R}^k$ so that

$$\sigma_u^2 = u^\top D_0 u$$

reaches its maximum and its minimum. Here σ_u^2 is the variance of the “linear score” $X_0 u$ and it represents how dispersed the data is in the “ u direction” in $\mathbf{R}^k$.

In this problem, remember that the coordinates of $u = (u_1, \dots, u_k)$ are the variables and the symmetric matrix D_0 is given. As usual, we to find the maximum and minimum values of σ_u^2 , we should look at the partial derivatives of σ_u^2 with respect to the variables u_i and set them to zero. Here, however, there is a catch – we want to restrict u to being a unit vector, with $u \cdot u = \sum u_i^2 = 1$.

So this is a *constrained optimization problem*:

- Find extreme values of the function

$$\sigma_u^2 = u^\top D_0 u$$

- Subject to the constraint $\|u\|^2 = u \cdot u = 1$ (or $u \cdot u - 1 = 0$)

We will use the technique of *Lagrange Multipliers* to solve such a problem.

To apply this method, we introduce the function

$$S(u, \lambda) = u^\top D_0 u - \lambda(u \cdot u - 1) \quad (2.7)$$

Then we compute the gradient

$$\nabla S = \begin{bmatrix} \frac{\partial S}{\partial u_1} \\ \vdots \\ \frac{\partial S}{\partial u_k} \\ \frac{\partial S}{\partial \lambda} \end{bmatrix} \quad (2.8)$$

and solve the system of equations $\nabla S = 0$. Here we have written the gradient as a column vector for reasons that will become clearer shortly.

Computing all of these partial derivatives looks messy, but actually if we take advantage of matrix algebra it's not too bad. The following two lemmas explain how to do this.

Lemma: Let M be a $N \times k$ matrix with constant coefficients and let u be a $k \times 1$ column vector whose entries are $u_1, \dots, u_k$. The function $F(u) = Mu$ is a linear map from $\mathbf{R}^k \rightarrow \mathbf{R}^N$. Its (total) derivative is a linear map between the same vector spaces, and satisfies

$$D(F)(v) = Mv$$

for any $k \times 1$ vector v . If u is a $1 \times N$ matrix, and $G(u) = uM$, then

$$D(G)(v) = vM$$

for any $1 \times N$ vector v . (This is the matrix version of the derivative rule that $\frac{d}{dx}(ax) = a$ for a constant a .)

Proof: Since $F : \mathbf{R}^k \rightarrow \mathbf{R}^N$, we can write out F in more traditional function notation as

$$F(u) = (F_1(u_1, \dots, u_k), \dots, F_N(u_1, \dots, u_k))$$

where

$$F_i(u_1, \dots, u_k) = \sum_{j=1}^k m_{ij} u_j.$$

Thus $\frac{\partial F_i}{\partial u_j} = m_{ij}$. The total derivative $D(F)$ is the linear map with matrix

$$D(F)_{ij} = \frac{\partial F_i}{\partial u_j} = m_{ij}$$

and so $D(F) = M$.

The other result is proved the same way.

Lemma: Let D be a symmetric $k \times k$ matrix with constant entries and let u be an $k \times 1$ column vector of variables $u_1, \dots, u_k$. Let $F : \mathbf{R}^k \rightarrow \mathbf{R}$ be the function $F(u) = u^\top Du$. Then the gradient $\nabla_u F$ is a vector field – that is, a vector-valued function of u , and is given by the formula

$$\nabla_u F = 2Du$$

Proof: Let d_{ij} be the i, j entry of D . We can write out the function F to obtain

$$F(u_1, \dots, u_k) = \sum_{i=1}^k \sum_{j=1}^k u_i d_{ij} u_j.$$

Now $\frac{\partial F}{\partial u_i}$ is going to pick out only terms where u_i appears, yielding:

$$\frac{\partial F}{\partial u_i} = \sum_{j=1}^k d_{ij} u_j + \sum_{j=1}^k u_j d_{ji}$$

Here the first sum catches all of the terms where the first “u” is u_i ; and the second sum catches all the terms where the second “u” is u_i . The diagonal terms $u_i^2 d_{ii}$ contribute once to each sum, which is consistent with the rule that the derivative of $u_i^2 d_{ii} = 2u_i d_{ii}$. To finish the proof, notice that

$$\sum_{j=1}^k u_j d_{ji} = \sum_{j=1}^k d_{ij} u_j$$

since D is symmetric, so in fact the two terms are the same. Thus

$$\frac{\partial}{\partial u_i} F = 2 \sum_{j=1}^k d_{ij} u_j$$

But the right hand side of this equation is twice the i^{th} entry of Du , so putting the results together we get

$$\nabla_u F = \begin{bmatrix} \frac{\partial F}{\partial u_1} \\ \vdots \\ \frac{\partial F}{\partial u_k} \end{bmatrix} = 2Du.$$

The following theorem puts all of this work together to reduce our questions about how variance changes with direction.

2.3.3 Critical values of the variance

Theorem: The critical values of the variance σ_u^2 , as u varies over unit vectors in $\mathbf{R}^N$, are the eigenvalues $\lambda_1, \dots, \lambda_k$ of the covariance matrix D , and if e_i is a unit eigenvector corresponding to λ_i , then $\sigma_{e_i}^2 = \lambda_i$.

Proof: Recall that we introduced the Lagrange function $S(u, \lambda)$, whose critical points give us the solutions to our constrained optimization problem. As we said in Equation 2.7:

$$S(u, \lambda) = u^\top D_0 u - \lambda(u \cdot u - 1) = u^\top D_0 u - \lambda(u \cdot u) + \lambda$$

Now apply our Matrix calculus lemmas. First, let’s treat λ as a constant and focus on the u variables. We can write $u \cdot u = u^\top I_N u$ where I_N is the identity matrix to compute:

$$\nabla_u S = 2D_0 u - 2\lambda u$$

2 Principal Component Analysis

For λ we have

$$\frac{\partial}{\partial \lambda} S = -u \cdot u + 1.$$

The critical points occur when

$$\nabla_u S = 2(D_0 - \lambda)u = 0$$

and

$$\frac{\partial}{\partial \lambda} S = 1 - u \cdot u = 0$$

The first equation says that λ must be an eigenvalue, and u an eigenvector:

$$D_0 u = \lambda u$$

while the second says u must be a unit vector $u \cdot u = \|u\|^2 = 1$. The second part of the result follows from the fact that if e_i is a unit eigenvector with eigenvalue λ_i then

$$\sigma_{e_i}^2 = e_i^\top D_0 e_i = \lambda_i \|e_i\|^2 = \lambda_i.$$

To really make this result pay off, we need to recall some key facts about the eigenvalues and eigenvectors of symmetric matrices. Because these facts are so central to this result, and to other applications throughout machine learning and mathematics generally, we provide proofs in Section 2.5.

Table 2.2: Properties of Eigenvalues of Real Symmetric Matrices

Summary

1. All of the eigenvalues $\lambda_1, \dots, \lambda_l$ of D are real. If $u^\top D u \geq 0$ for all $u \in \mathbf{R}^k$, then all eigenvalues λ_i are non-negative. In the latter case we say that D is *positive semi-definite*.
2. If v is an eigenvector for D with eigenvalue λ , and w is an eigenvector with a different eigenvalue λ' , then v and w are orthogonal: $v \cdot w = 0$.
3. There is an orthonormal basis $u_1, \dots, u_k$ of $\mathbf{R}^k$ made up of eigenvectors of D corresponding to the eigenvalues λ_i .
4. Let Λ be the diagonal matrix with entries $\lambda_1, \dots, \lambda_N$ and let P be the matrix whose columns are made up of the vectors u_i . Then $D = P\Lambda P^\top$.

If we combine our theorem on the critical values with the spectral theorem we get a complete picture. Let D_0 be the covariance matrix of our data. Since

$$\sigma_u^2 = u^\top D_0 u \geq 0 \text{ (it's a sum of squares)}$$

we know that the eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_k \geq 0$ are all nonnegative. Choose a corresponding sequence $u_1, \dots, u_k$ of orthogonal eigenvectors where all $\|u_i\|^2 = 1$. Since the u_i form a basis of $\mathbf{R}^N$, any score is a linear combination of the u_i :

$$S = \sum_{i=1}^k a_i u_i.$$

Since $u_i^\top D_0 u_j = \lambda_j u_i^\top u_j = 0$ unless $i = j$, in which case it is λ_i , we can compute

$$\sigma_S^2 = \sum_{i=1}^k \lambda_i a_i^2,$$

and $\|S\|^2 = \sum_{i=1}^k a_i^2$ since the u_i are an orthonormal set. So in these coordinates, our optimization problem is:

- maximize $\sum \lambda_i a_i^2$
- subject to the constraint $\sum a_i^2 = 1$.

We don't need any fancy math to see that the maximum happens when $a_1 = 1$ and the other $a_j = 0$, and in that case, the maximum is λ_1 . (If λ_1 occurs more than once, there may be a whole subspace of directions where the variance is maximal). Similarly, the minimum value is λ_k and occurs when $a_k = 1$ and the others are zero.

2.3.4 Subspaces of extremal variance

We can generalize the idea of the variance of our data in a particular direction to a higher dimensional version of *total variance* in a subspace. Suppose that E is a subspace of $\mathbf{R}^k$ and U is a

2 Principal Component Analysis

matrix whose columns span E – the columns of U are the weights of a family of scores that span E . The values of these scores are XU and the covariance matrix of this projected data is

$$\frac{1}{N}U^\top X^\top XU = U^\top D_0 U.$$

Finally, the *total variance* σ_E^2 of the data projected into E is the sum of the diagonal entries of the matrix

$$\sigma_E^2 = \text{trace}(U^\top D_0 U)$$

Just as the variance in a given direction u depends on the scaling of u , the variance in a subspace depends on the scaling of the columns of U . To normalize this scaling, we assume that the columns of U are an orthonormal basis of the subspace E .

Now we can generalize the question asked in Section 2.3.2 by seeking, not just a vector u pointing in the direction of the extremal variance, but instead the *subspace* U_s of dimension s with the property that the total variance of the projection of the data into U_s is maximal compared to its projection into other subspaces of that dimension. This is called a *subspace of extremal variance*.

To make this concrete, suppose we consider a subspace E of $\mathbf{R}^k$ of dimension t with basis $w_1, \dots, w_t$. Complete this to a basis $w_1, \dots, w_t, w_{t+1}, \dots, w_k$ of $\mathbf{R}^k$ and then apply the Gram Schmidt Process (see Section 2.5.1) to find an orthonormal basis $w'_1, \dots, w'_s, w'_{s+1}, \dots, w'_k$ where the $w'_1, \dots, w'_t$ are an orthonormal basis for E . Let W be the $k \times t$ matrix whose columns are the w'_i for $i = 1, \dots, t$. The rows of the matrix $X_0 W$ given the coordinates of the projection of each sample into the subspace E expressed in terms of the scores corresponding to these vectors w'_i . The total variance of these projections is

$$\sigma_E^2 = \sum_{i=1}^t \|X_0 w'_i\|^2 = \sum_{i=1}^t (w'_i)^\top X_0^\top X_0 w'_i = \sum_{i=1}^t (w'_i)^\top D_0 w'_i$$

If we want to maximize this, we have the constrained optimization problem of finding $w'_1, \dots, w'_t$ so that

- $\sum_{i=1}^t (w'_i)^\top D_0 w'_i$ is maximal
- subject to the constraint that each w'_i has $\|w'_i\|^2 = 1$,
- and that the w'_i are orthogonal, meaning $w'_i \cdot w'_j = 0$ for $i \neq j$,
- and that the w'_i are linearly independent.

Then the span E of these w'_i is subspace of extremal variance.

Theorem: A t -dimensional subspace E is a subspace of extremal variance if and only if it is spanned by t orthonormal eigenvectors of the matrix D_0 corresponding to the t largest eigenvalues for D_0 .

Proof: We can approach this problem using Lagrange multipliers and matrix calculus if we are careful. Our unknown is $k \times t$ matrix W whose columns are the t (unknown) vectors w'_i . The objective function that we are seeking to maximize is

$$F = \text{trace}(W^\top D_0 W) = \sum_{i=1}^t (w'_i)^\top D_0 w_i.$$

The constraints are the requirements that $\|w'_i\|^2 = 1$ and $w'_i \cdot w'_j = 0$ if $i \neq j$. If we introduction a matrix of lagrange multipliers $\Lambda = (\lambda_{ij})$, where λ_{ij} is the multiplier that goes with the the first of these constraints when $i = j$, and the second when $i \neq j$, we can express our Lagrange function as:

$$S(W, \Lambda) = \text{trace}(W^\top D_0 W) - (W^\top W - I)\Lambda$$

where I is the $t \times t$ identity matrix.

Taking the derivatives with respect to the entries of W and of Λ yields the following two equations:

$$\begin{aligned} D_0 W &= W \Lambda \\ W^\top W &= I \end{aligned}$$

The first of these equations says that the space E spanned by the columns of W is *invariant* under D_0 , while the second says that the columns of W form an orthonormal basis.

Let's assume for the moment that we have a matrix W that satisfies these conditions. Then it must be the case that Λ is a symmetric, real valued $t \times t$ matrix, since

$$W^\top D_0 W = W^\top W \Lambda = \Lambda.$$

and the matrix on the left is symmetric.

By the properties of real symmetric matrices (the spectral theorem), there are orthonormal vectors $q_1, \dots, q_t$ that are eigenvectors of Λ with corresponding eigenvalues τ_i . If we let Q be the matrix whose columns are the vectors q_i and let T be the diagonal $t \times t$ matrix whose entries are the τ_i , we have

$$\Lambda Q = QT.$$

If we go back to our original equations, we see that if W exists such that $DW = W\Lambda$, then there is a matrix Q with orthonormal columns and a diagonal matrix T such that

$$D_0 W Q = W \Lambda Q = W Q T.$$

In other words, WQ is a matrix whose columns are eigenvectors of D_0 with eigenvalues τ_i for $i = 1, \dots, t$.

Thus we see how to construct an invariant subspace E and a solution matrix W . Such an E is spanned by t orthonormal eigenvectors q_i with eigenvalues τ_i of D_0 ; and W is the matrix whose columns are the q_i . Further, in that case, the total variance associated to E is the sum of the eigenvalues τ_i ; to make this as large as possible, we should choose our eigenvectors to correspond to t of the largest eigenvalues of D_0 . This concludes the proof.

2.3.5 Definition of Principal Components

Definition: The orthonormal unit eigenvectors u_i for D_0 are the *principal directions* or *principal components* for the data X_0 .

Theorem: The maximum variance occurs in the principal direction(s) associated to the largest eigenvalue, and the minimum variance in the principal direction(s) associated with the smallest one. The covariance between scores in principal directions associated with different eigenvalues is zero.

At this point, the picture in Figure 2.8 makes sense – the red and green dashed lines are the principal directions, they are orthogonal to one another, and the point in the directions where the data is most (and least) “spread out.”

Proof: The statement about the largest and smallest eigenvalues is proved at the very end of the last section. The covariance of two scores corresponding to different eigenvectors u_i and u_j is

$$u_i^\top D_0 u_j = \lambda_j (u_i \cdot u_j) = 0$$

since the u_i and u_j are orthogonal.

Sometimes the results above are presented in a slightly different form, and may be referred to, in part, as Rayleigh’s theorem.

Corollary: (Rayleigh’s Theorem) Let D be a real symmetric matrix and let

$$H(v) = \max_{v \neq 0} \frac{v^\top D v}{v^\top v}.$$

Then $H(v)$ is the largest eigenvalue of D . (Similarly, if we replace max by min, then the minimum is the least eigenvalue).

Proof: The maximum of the function $H(v)$ is the solution to the same optimization problem that we considered above.

Exercises.

1. Prove that the two expressions for σ_X^2 given in Equation 2.1 are the same.
2. Prove that the covariance matrix is as described in the proposition in Section 2.2.4.
3. Let X_0 be a $k \times N$ matrix with entries x_{ij} for $1 \leq i \leq k$ and $1 \leq j \leq N$. If a linear score is defined by the constants $a_1, \dots, a_N$, check that equation Equation 2.5 holds as claimed.
4. Why is it important to use a unit vector when computing the variance of X_0 in the direction of u ? Suppose $v = \lambda u$ where u is a unit vector and $\lambda > 0$ is a constant. Let S' be the score $X_0 v$. How is the variance of S' related to that of $S = X_0 u$?

2.4 Dimensionality Reduction via Principal Components

The principal components associated with a dataset separate out directions in the feature space in which the data is most (or least) variable. One of the main applications of this information is to enable us to take data with a great many features – a set of points in a high dimensional space – and, by focusing our attention on the scores corresponding to the principal directions, capture most of the information in the data in a much lower dimensional setting.

To illustrate how this is done, let X be a $N \times k$ data matrix, let X_0 be its centered version, and let $D_0 = \frac{1}{N} X_0^\top X_0$ be the associated covariance matrix.

Apply the spectral theorem (proved in Section 2.5) to the covariance matrix to obtain eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_k \geq 0$ and associated eigenvectors $u_1, \dots, u_k$. The scores $S_i = X_0 u_i$ give the values of the data in the principal directions. The variance of S_i is λ_i .

Now choose a number $t < k$ and consider the vectors $S_1, \dots, S_t$. The j^{th} entry in S_i is the value of the score S_i for the j^{th} data point. Because $S_1, \dots, S_t$ capture the most significant variability in the original data, we can learn a lot about our data by considering just these t features of the data, instead of needing all N .

To illustrate, let's look at an example. We begin with a synthetic dataset X_0 which has 200 samples and 15 features. The data (some of it) for some of the samples is shown in Table 2.3.

Table 2.3: Simulated Data for PCA Analysis

	f-0	f-1	f-2	f-3	f-4	...	f-10	f-11	f-12	f-13	f-14
s-0	1.18	-0.41	2.02	0.44	2.24	...	0.32	0.95	0.88	1.10	0.89
s-1	0.74	0.58	1.54	0.23	2.05	...	0.99	1.14	1.56	0.99	0.59
...	...	...	...	...	...	...	...	...	...	...	...
s-198	1.04	2.02	1.44	0.40	1.33	...	0.62	0.62	0.54	1.96	0.04
s-199	0.92	2.09	1.58	1.19	1.17	...	0.42	0.85	0.83	2.22	0.90

The full dataset is a 200×15 matrix; it has 3000 numbers in it and we're not really equipped to make sense of it. We could try some graphing – for example, Figure 2.9 shows a scatter plot of two of the features plotted against each other.

Unfortunately there's not much to see in Figure 2.9 – just a blob – because the individual features of the data don't tell us much in isolation, whatever structure there is in this data arises out of the relationship between different features.

In Figure 2.10 we show a “density grid” plot of the data. The graph in position i, j shows a scatter plot of the i^{th} and j^{th} columns of the data, except in the diagonal positions, where in position i, i we plot a histogram of column i . There's not much structure visible; it is a lot of blobs.

So let's apply the theory of principal components. We use a software package to compute the eigenvalues and eigenvectors of the matrix D_0 . The 15 eigenvalues $\lambda_1 \geq \dots \geq \lambda_{15}$ are plotted, in descending order, in Figure 2.11 .

2 Principal Component Analysis

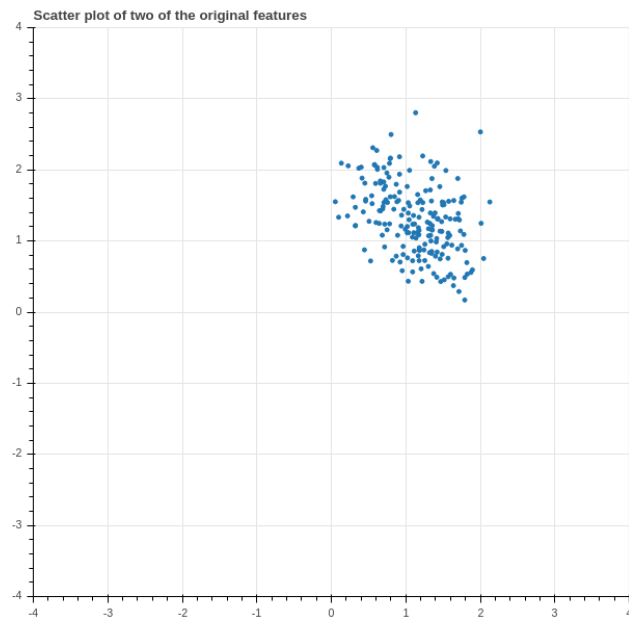


Figure 2.9: Scatter Plot of Two Features

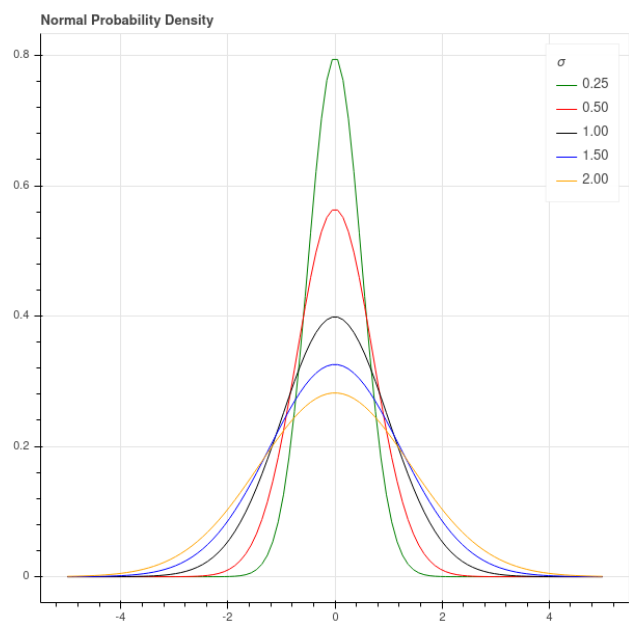


Figure 2.10: Density Grid Plot of All Features

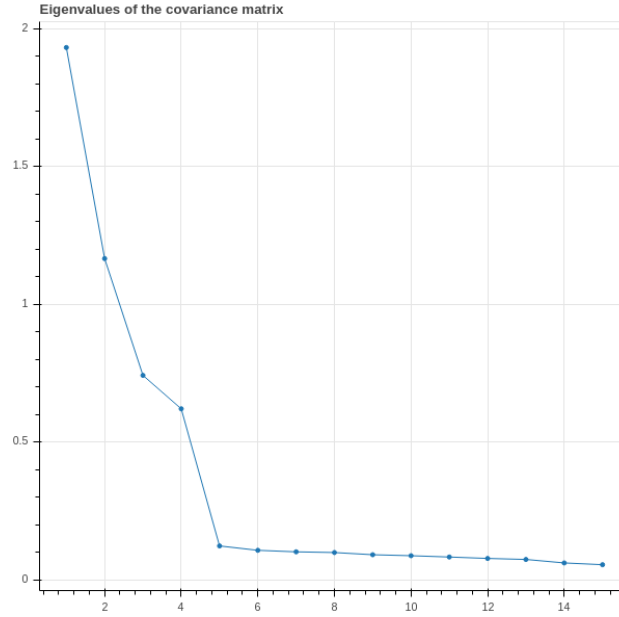


Figure 2.11: Eigenvalues of the Covariance Matrix

This plot shows that the first 4 eigenvalues are relatively large, while the remaining 11 are smaller and not much different from each other. We interpret this as saying that *most of the variation in the data is accounted for by the first four principal components*. We can even make this quantitative. The *total variance* of the data is the sum of the eigenvalues of the covariance matrix – the trace of D_0 – and in this example that sum is around 5. The sum of the first 4 eigenvalues is about 4, so the first four eigenvalues account for about $4/5$ of the total variance, or about 80% of the variation of the data.

Now let's focus in on the two largest eigenvalues λ_1 and λ_2 and their corresponding eigenvectors u_1 and u_2 . The 200×1 column vectors $S_1 = X_0 u_1$ and $S_2 = X_0 u_2$ are the values of the scores associated with these two eigenvectors. So for each data point (each row of X_0) we have two values (the corresponding entries of S_1 and S_2 .) In Figure 2.12 we show a scatter plot of these scores.

Notice that suddenly some structure emerges in our data! We can see that the 200 points are separated into five clusters, distinguished by the values of their scores! This ability to find hidden structure in complicated data, is one of the most important applications of principal components.

If we were dealing with real data, we would now want to investigate the different groups of points to see if we can understand what characteristics the principal components have identified.

2.4.1 Loadings

There's one last piece of the PCA puzzle that we are going to investigate. In Figure 2.12, we plotted our data points in the coordinates given by the first two principal components. In geometric terms, we took the cloud of 200 points in $\mathbf{R}^{15}$ given by the rows of X_0 and projected those points into the

2 Principal Component Analysis

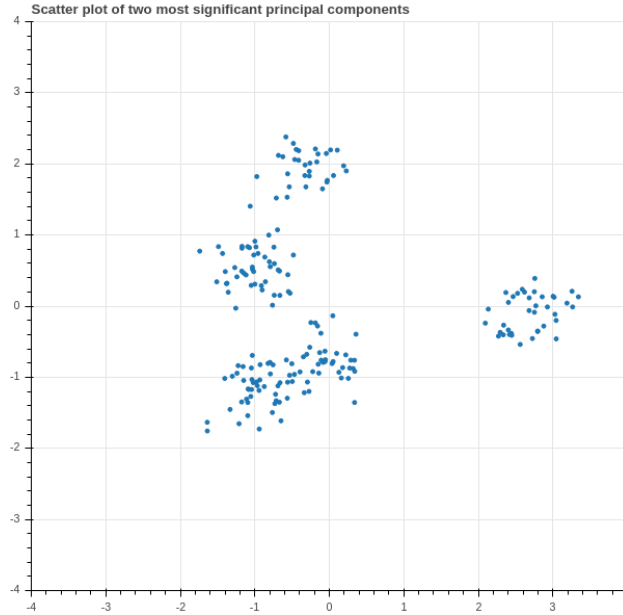


Figure 2.12: Scatter Plot of Scores in the First Two Principal Directions

two dimensional plane spanned by the eigenvectors u_1 and u_2 , and then plotted the distribution of the points in that plane.

More generally, suppose we take our dataset X_0 and consider the first t principal components corresponding to the eigenvectors $u_1, \dots, u_t$. The projection of the data into the space spanned by these eigenvectors is represented by the $S = k \times t$ matrix X_0U where U is the $k \times t$ matrix whose columns are the eigenvectors u_i . Each row of S gives the values of the score arising from u_i in the i^{th} column for $i = 1, \dots, t$.

The remaining question that we wish to consider is: how can we see some evidence of the original features in subspace? We can answer this by imagining that we had an artificial sample x that has a measurement of 1 for the i^{th} feature and a measurement of zero for all the other features. The corresponding point is represented by a $1 \times k$ row vector with a 1 in position i . The projection of this synthetic sample into the span of the first t principal components is the $1 \times t$ vector xU . Notice, however, that xU is just the i^{th} row of the matrix U . This vector in the space spanned by the u_i is called the “loading” of the i^{th} feature in the principal components.

This is illustrated in Figure 2.13, which shows a line along the direction of the loading corresponding to the each feature added to the scatter plot of the data in the plane spanned by the first two principal components. One observation one can make is that some of the features are more “left to right”, like features 7 and 8, while others are more “top to bottom”, like 6. So points that lie on the left side of the plot have smaller values of features 7 and 8, while those at the top of the plot have larger values of feature 6.

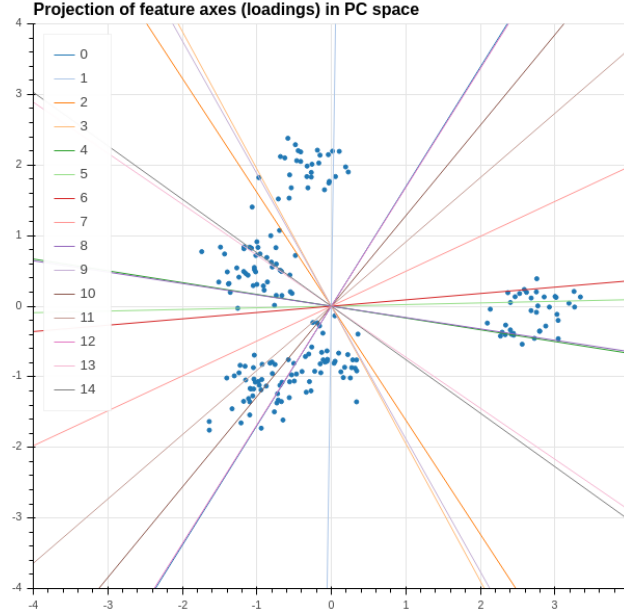


Figure 2.13: Loadings in the Principal Component Plane

2.4.2 The singular value decomposition

The singular value decomposition is a slightly more detailed way of looking at principal components. Let Λ be the diagonal matrix of eigenvalues of D_0 and let P be the $k \times k$ orthogonal matrix whose columns are the principal components. Then we have

$$D_0 = \frac{1}{N} X_0^\top X_0 = P \Lambda P^\top.$$

Consider the $N \times k$ matrix

$$X_0 P = A.$$

As we saw in the previous section, the columns of A give the projection of the data into the k principal directions. Then

$$A^\top A = P^\top X_0^\top X_0 P = N \Lambda.$$

In other words, the columns of A are orthogonal and the diagonal entries of $A^\top A$ are N times the variance of the data in the various principal directions.

Now we are going to tinker with the matrix A in order to make an $N \times N$ orthogonal matrix. The first modification we make is to normalize the columns of A so that they have length 1. We do this by setting

$$A_1 = A(N\Lambda)^{-1/2}.$$

Then $A_1^\top A_1$ is the identity, so the columns of A_1 are orthonormal. Here we are assuming that the eigenvalues of D_0 are nonzero – this isn't strictly necessary, and we could work around this, but for

2 Principal Component Analysis

simplicity we will assume it's true. It amounts to the assumption that the independent variables are not linearly related, as we've seen before.

The second modification is to extend A_1 to an $N \times N$ matrix. The k columns of A_1 span only a k -dimensional subspace of the N -dimensional space where the feature vectors lie.

Complete the subspace by finding an orthogonal complement to it – that is, find $N - k$ mutually orthogonal unit vectors all orthogonal to the column space of A_1 . By adding these vectors to A as columns, create an extended $N \times N$ matrix $\tilde{A}_1$ which is orthogonal.

Notice that $\tilde{A}_1^\top A$ is an $N \times k$ matrix whose upper $k \times k$ block is $(N\Lambda)^{1/2}$ and whose final $N - k$ rows are all zero. We call this matrix $\tilde{\Lambda}$.

To maintain consistency with the traditional formulation, we let $U = \tilde{A}_1^\top$ and then we have the following proposition.

Proposition: We have a factorization

$$X_0 = U\tilde{\Lambda}P^\top \quad (2.9)$$

where U and P are orthogonal matrices of size $N \times N$ and $k \times k$ respectively, and $\tilde{\Lambda}$ is an $N \times k$ diagonal matrix. This is called the “singular value decomposition” of X_0 , and the entries of $\tilde{\Lambda}$ are called the singular values. If we let $u_1, \dots, u_k$ be the first k rows of U , then the k column vectors $u_i^\top$ are an orthonormal basis for the feature space spanned by the columns of X_0 , and they point in the “principal directions” for the data matrix X_0 .

In this section we take a slight detour and apply what we've learned about the covariance matrix, principal components, and the singular value decomposition to the original problem of linear regression that we studied in Chapter 1.

In this setting, in addition to our centered data matrix X_0 , we have a vector Y of target values and we find the “best” approximation

$$\hat{Y} = X_0 M$$

using the least squares method. As we showed in Chapter 1, the optimum M is found as

$$M = (X_0^\top X_0)^{-1} X_0^\top Y = N D_0^{-1} X_0^\top Y$$

and the predicted values $\hat{Y}$ are

$$\hat{Y} = N X_0 D_0^{-1} X_0^\top Y.$$

Geometrically, we understood this process as defining $\hat{Y}$ to be the orthogonal projection of Y into the subspace spanned by the columns of X_0 .

Let's use the decomposition (see Equation 2.9) $X_0 = U\tilde{\Lambda}P^\top$ in this formula. First, notice that

$$X_0^\top X_0 = P\tilde{\Lambda}^\top U^\top U\tilde{\Lambda}P^\top = P\tilde{\Lambda}^\top \tilde{\Lambda}P^\top.$$

The middle term $\tilde{\Lambda}^\top \tilde{\Lambda}$ is the $k \times k$ matrix Λ whose diagonal entries are $N\lambda_i$ where λ_i are the eigenvalues of the covariance matrix D_0 . Assuming these are all nonzero (which is tantamount to the assumption that the covariance matrix is invertible), we obtain

$$\hat{Y} = N U \tilde{\Lambda} P^\top P \Lambda^{-1} P^\top P \tilde{\Lambda}^\top U^\top Y.$$

2.5 Eigenvalues and Eigenvectors of Real Symmetric Matrices (The Spectral Theorem)

There is a lot of cancellation here, and in the end what's left is

$$\hat{Y} = UEU^\top Y$$

where E is an $N \times N$ matrix whose upper $k \times k$ block is the identity and whose remaining entries are zero. Rearranging a bit more we have

$$U^\top \hat{Y} = EU^\top Y.$$

To unpack this equation, let $u_1, \dots, u_N$ be the rows of the matrix U . Since U is an orthogonal matrix, the column vectors $u_i^\top$ are an orthonormal basis for the N dimensional space where the columns of X_0 lie. We can write the target vector Y

$$Y = \sum_{j=1}^N (u_j \cdot Y) u_j^\top.$$

Then the projection $\hat{Y}$ of Y into the subspace spanned by the data is obtained by dropping the last $N - k$ terms in the sum:

$$\hat{Y} = \sum_{j=1}^k (u_j \cdot Y) u_j^\top$$

2.5 Eigenvalues and Eigenvectors of Real Symmetric Matrices (The Spectral Theorem)

Now that we've shown how to apply the theory of eigenvalues and eigenvectors of symmetric matrices to extract principal directions from data, and to use those principal directions to find structure, we will give a proof of the properties that we summarized in Table 2.2.

A key tool in the proof is the Gram-Schmidt orthogonalization process.

2.5.1 Gram-Schmidt

Proposition (Gram-Schmidt Process): Let $w_1, \dots, w_k$ be a collection of linearly independent vectors in $\mathbf{R}^N$ and let W be the span of the w_i . Let $u_1 = w_1$ and let

$$u_i = w_i - \sum_{j=1}^{i-1} \frac{w_i \cdot u_j}{u_j \cdot u_j} u_j$$

for $i = 2, \dots, k$. Then

- The vectors u_i are orthogonal: $u_i \cdot u_j = 0$ unless $i = j$.
- The vectors u_i span W .
- Each u_i is orthogonal to the all of $w_1, \dots, w_{i-1}$.
- The vectors $u'_i = u_i / \|u_i\|$ are orthonormal.

Proof: This is an inductive exercise, and we leave it to you to work out the details.

2.5.2 The spectral theorem

Theorem: Let D be a real symmetric $N \times N$ matrix. Then:

1. All of the N eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N$ are real. If $u^\top Du \geq 0$ for all $u \in \mathbf{R}^N$, then all eigenvalues $\lambda_i \geq 0$.
2. The matrix D is diagonalizable – that is, it has N linearly independent eigenvectors.
3. If v and w are eigenvectors corresponding to eigenvalues λ and λ' , with $\lambda \neq \lambda'$, then v and w are orthogonal: $v \cdot w = 0$.
4. There is an orthonormal basis $u_1, \dots, u_N$ of $\mathbf{R}^N$ made up of eigenvectors for the eigenvalues λ_i .
5. Let Λ be the diagonal matrix with entries $\lambda_1, \dots, \lambda_N$ and let P be the matrix whose columns are made up of the eigenvectors u_i . Then $D = P\Lambda P^\top$.

Proof: First of all, we use the fact that any matrix has at least one eigenvector with associated eigenvalue. This is a theorem from linear algebra that relies on the fundamental theorem of algebra. With this result available, we start by proving part 1. Suppose that λ is an eigenvalue of D . Let u be a corresponding nonzero eigenvector. Then $Du = \lambda u$ and $D\bar{u} = \bar{\lambda}\bar{u}$, where $\bar{u}$ is the vector whose entries are the conjugates of the entries of u (and $\bar{\lambda} = \lambda$ since D is real). Now we have

$$\bar{u}^\top Du = \lambda \bar{u} \cdot u = \lambda \|u\|^2$$

and

$$u^\top D\bar{u} = \bar{\lambda} u \cdot \bar{u} = \bar{\lambda} \|u\|^2.$$

But the left hand side of both of these equations are the same (take the transpose and use the symmetry of D) so we must have $\lambda \|u\|^2 = \bar{\lambda} \|u\|^2$ so $\lambda = \bar{\lambda}$, meaning λ is real.

If we have the additional property that $u^\top Du \geq 0$ for all u , then in particular $u_i^\top Du_i = \lambda \|u\|^2 \geq 0$, and since $\|u\|^2 > 0$ we must have $\lambda \geq 0$.

Property 2 is in some ways the most critical fact. We know from the general theory of the characteristic polynomial, and the fundamental theorem of algebra, that D has N complex eigenvalues, although some may be repeated. However, it may not be the case that D has N linearly independent eigenvectors – it may not be *diagonalizable*. So we will establish that any *symmetric* matrix over the real numbers is diagonalizable.

A one-by-one matrix is automatically symmetric and diagonalizable. In the N -dimensional case, we know, at least, that D has at least one eigenvector, and real one at that by part 1, and this gives us a place to begin an inductive argument.

Let $v_N \neq 0$ be an eigenvector with eigenvalue λ and normalized so that $\|v_N\|^2 = 1$, and extend this to a basis $v_1, \dots, v_N$ of $\mathbf{R}^N$. Apply the Gram-Schmidt process to construct an orthonormal basis of $\mathbf{R}^N$ $u_1, \dots, u_N$ so that $u_N = v_N$.

Any vector $v \in \mathbf{R}^N$ is a linear combination

$$v = \sum_{i=1}^N a_i u_i$$

2.5 Eigenvalues and Eigenvectors of Real Symmetric Matrices (The Spectral Theorem)

and, since the u_i are orthonormal, the coefficients can be calculated as $a_i = (u_i \cdot v)$.

Using this, we can find the matrix D' of the linear map defined by our original matrix D in this new basis. By definition, if d'_{ij} are the entries of D' , then

$$Du_i = \sum_{j=1}^N d'_{ij} u_j$$

and so

$$d'_{ij} = u_j \cdot Du_i = u_j^\top Du_i.$$

Since D is symmetric, $u_j^\top Du_i = u_i^\top Du_j$ and so $d'_{ij} = d'_{ji}$. In other words, the matrix D' is still symmetric. Furthermore,

$$d'_{Ni} = u_i \cdot Du_N = u_i \cdot \lambda u_N = \lambda(u_i \cdot u_N)$$

since $u_N = v_N$. Since the u_i are an orthonormal basis, we see that $d'_{iN} = 0$ unless $i = N$, and $d'_{NN} = \lambda$.

In other words, the matrix D' has a block form:

$$D' = \begin{pmatrix} * & * & \cdots & * & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ * & * & \cdots & * & 0 \\ 0 & 0 & \cdots & 0 & \lambda \end{pmatrix}$$

and the block denoted by $*$'s is symmetric. If we call that block D_* , the inductive hypothesis tells us that the symmetric matrix D_* is diagonalizable, so it has a basis of eigenvectors $u'_1, \dots, u'_{N-1}$ with eigenvalues $\lambda_1, \dots, \lambda_{N-1}$; this gives us a basis for the subspace of $\mathbf{R}^N$ spanned by $u_1, \dots, u_{N-1}$ which, together with u_N gives us a basis of $\mathbf{R}^N$ consisting of eigenvectors of D .

This finishes the proof of Property 2.

For property 3, compute

$$v^\top Dw = \lambda'(v \cdot w) = w^\top Dv = \lambda(w \cdot v).$$

Since $\lambda \neq \lambda'$, we must have $v \cdot w = 0$.

For property 4, if the eigenvalues are all distinct, this is a consequence of property 2 – you have N eigenvectors, scaled to length 1, for different eigenvalues, and by 2 they are orthogonal. So the only complication is the case where some eigenvalues are repeated. If λ occurs r times, then you have r linearly independent vectors $u_1, \dots, u_r$ that span the λ eigenspace. The Gram-Schmidt process allows you to construct an orthonormal set that spans this eigenspace, and while this orthonormal set isn't unique, any one of them will do.

2 Principal Component Analysis

For property 5, let e_i be the column vector that is zero except for a 1 in position i . The product $e_j^\top D e_i = d_{ij}$. Let's write e_i and e_j in terms of the orthonormal basis $u_1, \dots, u_N$:

$$e_i = \sum_{k=1}^N (e_i \cdot u_k) u_k \text{ and } e_j = \sum_{k=1}^N (e_j \cdot u_k) u_k.$$

Using this expansion, we compute $e_j^\top D e_i$ in a more complicated way:

$$e_j^\top D e_i = \sum_{r=1}^N \sum_{s=1}^N (e_j \cdot u_r) (e_i \cdot u_s) (u_r^\top D u_s).$$

But $u_r^\top D u_s = \lambda_s (u_r \cdot u_s) = 0$ unless $r = s$, in which case it equals λ_r , so

$$e_j^\top D e_i = \sum_{r=1}^N \lambda_r (e_j \cdot u_r) (e_i \cdot u_r).$$

On the other hand,

$$P^\top e_i = \begin{bmatrix} (e_i \cdot u_1) \\ (e_i \cdot u_2) \\ \vdots \\ (e_i \cdot u_N) \end{bmatrix}$$

and

$$\Lambda P^\top e_i = \begin{bmatrix} \lambda_1 (e_i \cdot u_1) \\ \lambda_2 (e_i \cdot u_2) \\ \vdots \\ \lambda_N (e_i \cdot u_N) \end{bmatrix}$$

Therefore the i, j entry of $P \Lambda P^\top$ is

$$(e_j^\top P) \Lambda (P^\top e_i) = \sum_{r=1}^N \lambda_r (e_i \cdot u_r) (e_j \cdot u_r) = d_{ij}$$

so the two matrices D and $P \Lambda P^\top$ are in fact equal.

Exercises:

1. Prove the rest of the first lemma in Section 2.4.2.
2. Prove the Gram-Schmidt Process has the claimed properties in Section 2.5.1.

3 Probability and Bayes Theorem

3.1 Introduction

Probability theory is one of the three central mathematical tools in machine learning, along with multivariable calculus and linear algebra. Tools from probability allow us to manage the uncertainty inherent in data collected from real-world experiments, and to measure the reliability of predictions that we make from that data. In these notes, we will review some of the basic terminology of probability and introduce Bayesian inference as a technique in machine learning problems.

This is a superficial introduction to ideas from probability. For a thorough treatment, see this open-source introduction to probability. For a more applied emphasis, I recommend the excellent online course Probabilistic Systems Analysis and Applied Probability and its associated textbook [2].

3.2 Probability Basics

The theory of probability begins with a set X of possible events or outcomes, together with a “probability” function P on certain subsets of X that measures “how likely” that combination of events is to occur.

The set X can be discrete or continuous. For example, when flipping a coin, our set of possible events would be the discrete set $\{H, T\}$ corresponding to the outcomes of flipping heads or tails. When measuring the temperature using a thermometer, our set of possible outcomes might be the set of real numbers, or perhaps an interval in $\mathbb{R}$. The thermometer’s measurement is random because it is affected by, say, electronic noise, and so its reading is the true temperature perturbed by a random amount.

The values of P are between 0, meaning that the event *will not* happen, and 1, meaning that it is certain to occur. As part of our set up, we assume that the probability of *some* event from X occurring is 1, so that $P(X) = 1$; and the chance of “nothing” happening is zero, so $P(\emptyset) = 0$. And if $U \subset X$ is some collection, then $P(U)$ is the chance of an event from U occurring.

The last ingredient of this picture of probability is additivity. Namely, we assume that if U and V are subsets of X that are disjoint, then

$$P(U \cup V) = P(U) + P(V).$$

Even more generally, we assume that this holds for countably infinite collections of disjoint subsets $U_1, U_2, \dots$, where

$$P(U_1 \cup U_2 \cup \dots) = \sum_{i=1}^{\infty} P(U_i)$$

Definition: The combination of a set X of possible outcomes and a probability function P on subsets of X that satisfies $P(X) = 1$, $0 \leq P(U) \leq 1$ for all U , and is additive on countable disjoint collections of subsets of X is called a (naive) probability space. X is called the *sample space* and the subsets of X are called *events*.

Warning: The reason for the term “naive” in the above definition is that, if X is an uncountable set such as the real numbers $\mathbb{R}$, then the conditions in the definition are self-contradictory. This is a deep and rather surprising fact. To make a sensible definition of a probability space, one has to restrict the domain of the probability function P to certain subsets of X . These ideas form the basis of the mathematical subject known as measure theory. In these notes we will work with explicit probability functions and simple subsets such as intervals that avoid these technicalities.

3.2.1 Discrete probability examples

The simplest probability space arises in the analysis of coin flipping. As mentioned earlier, the set X contains two elements $\{H, T\}$. The probability function P is determined by its value $P(\{H\}) = p$, where $0 \leq p \leq 1$, which is the chance of the coin yielding a “head”. Since $P(X) = 1$, we have $P(\{T\}) = 1 - p$.

Other examples of discrete probability spaces arise from dice-rolling and playing cards. For example, suppose we roll two six-sided dice. There are 36 equally likely outcomes from this experiment. If instead we consider the sum of the two values on the dice, our outcomes range from 2 to 12 and the probabilities of these outcomes are given by

sum(s)	probability
2, 12	1/36
3, 11	1/18
4, 10	1/12
5, 9	1/9
6, 8	5/36
7	1/6

A traditional deck of 52 playing cards contains 4 aces. Assuming that the chance of drawing any card is the same (and is therefore equal to $1/52$), the probability of drawing an ace is $4/52 = 1/13$ since

$$P(\{A_{\clubsuit}, A_{\spadesuit}, A_{\heartsuit}, A_{\diamondsuit}\}) = 4P(\{A_{\clubsuit}\}) = 4/52 = 1/13$$

3.2.2 Continuous probability examples

When the set X is continuous, such as in the temperature measurement, we measure $P(U)$, where $U \subset X$, by giving a “probability density function” $f : X \rightarrow [0, \infty)$ and declaring that

$$P(U) = \int_U f(x)dx.$$

Notice that our function $f(x)$ has to satisfy the condition

$$P(X) = \int_X f(x)dx = 1.$$

For example, in our temperature measurement example, suppose the “true” outside temperature is t_0 , and our thermometer gives a reading t . Then a good model for the random error is to assume that the error $x = t - t_0$ is governed by the density function

$$f_\sigma(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-x^2/2\sigma^2}$$

where σ is a parameter. In a continuous situation such as this one, the probability of any single point in X is zero since

$$P(\{t\}) = \int_t^t f_\sigma(x)dx = 0$$

Still, the shape of the density function does tell you where the values are concentrated – values where the density function is larger are more likely than those where it is smaller.

With this density function, and $x = t - t_0$, the error in our measurement is given by

$$P(|t - t_0| < \delta) = \int_{-\delta}^{\delta} \frac{1}{\sigma\sqrt{2\pi}}e^{-x^2/2\sigma^2}dx \quad (3.1)$$

The parameter σ (called the *standard deviation*) controls how tightly the thermometer’s measurement is clustered around the true value t_0 ; when σ is large, the measurements are scattered widely, when it is small, they are clustered tightly. See Figure 3.1.

3.3 Conditional Probability and Bayes Theorem

The theory of conditional probability gives a way to study how partial information about an event informs us about the event as a whole. For example, suppose you draw a card at random from a deck. As we’ve seen earlier, the chance that card is an ace is $1/13$. Now suppose that you somehow learn that the card is definitely not a jack, queen, or king. Since there are 12 cards in the deck that are jacks, kings, or queens, the card you’ve drawn is one of the remaining 40 cards, which includes 4 aces. Thus the chance you are holding an ace is now $4/40 = 1/10$.

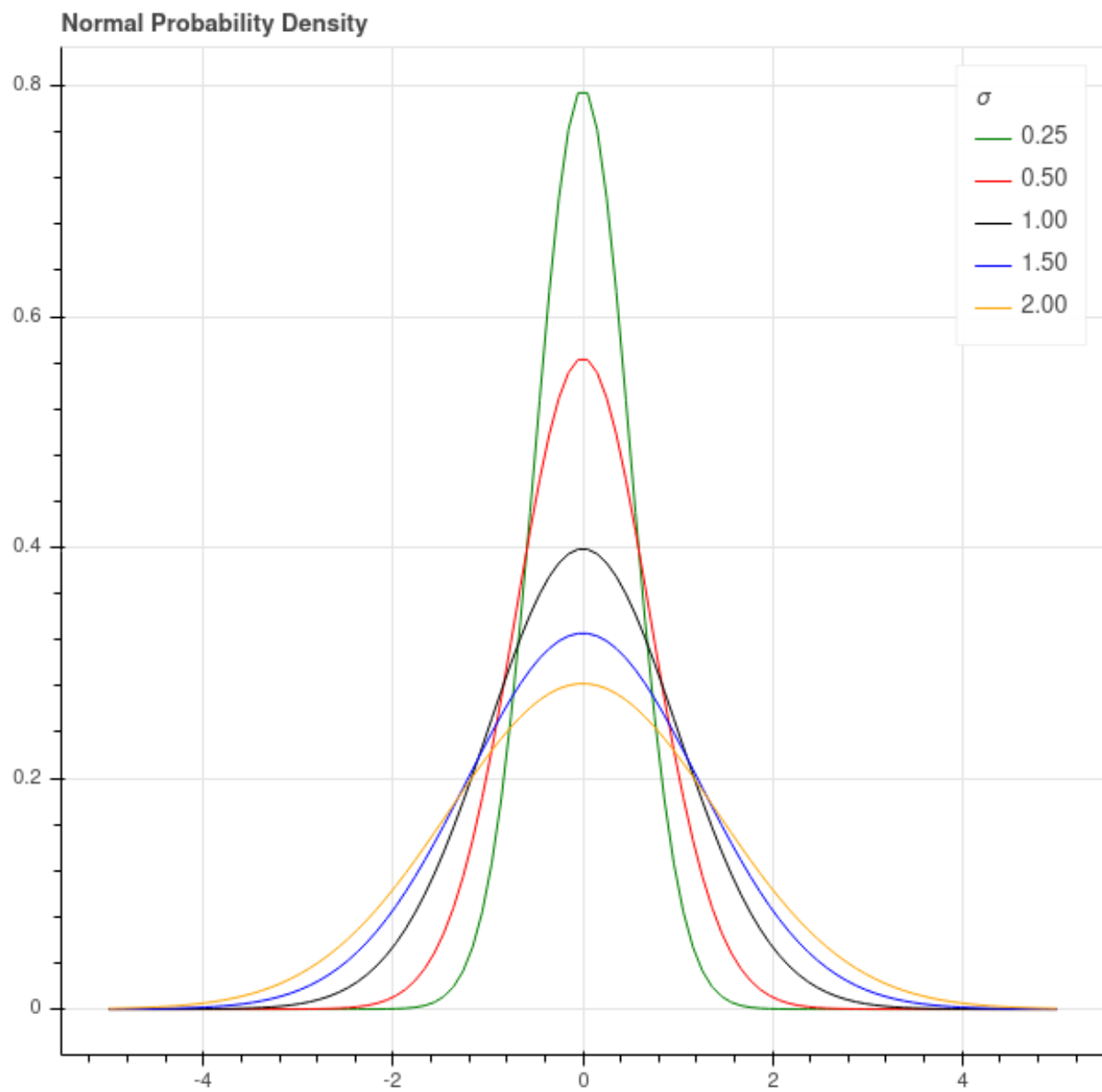


Figure 3.1: Normal Density

In terms of notation, if A is the event “my card is an ace” and B is the event “my card is not a jack, queen, or king” then we say that *the probability of A given B* is $1/10$. The notation for this is

$$P(A|B) = 1/10.$$

More generally, if A and B are events in a sample space X , and $P(B) > 0$, then

$$P(A|B) = \frac{P(A \cap B)}{P(B)},$$

so that $P(A|B)$ measures the chance that A occurs among those situations in which B occurs.

3.3.1 Bayes Theorem

Bayes theorem is a foundational result in probability.

Theorem: Bayes Theorem says

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

If we use the definition of conditional probability given above, this is straightforward:

$$\frac{P(B|A)P(A)}{P(B)} = \frac{P(A \cap B)}{P(B)} = P(A|B).$$

3.3.2 An example

To illustrate conditional probability, let’s consider what happens when we administer the PCR test for COVID-19 to an individual drawn from the population at large. There are two possible test results (positive and negative) and two possible true states (infected and not infected). Suppose I go to the doctor and get a COVID test which comes back positive. What is the probability that I actually have COVID?

Let’s let S and W stand for infected (sick) and not infected (well) respectively, and let $+/-$ stand for test results positive or negative respectively. Note that there are four possible outcomes of our experiment:

- test positive and infected ($S+$) – this is a *true positive*.
- test positive and not infected ($W+$) – this is a *false positive*.
- test negative and infected ($S-$) – this is a *false negative*.
- test negative and not infected ($W-$) – this is a *true negative*.

3 Probability and Bayes Theorem

This ICD10 report claimsthat the chance of a false positive – that is, the percentage of samples from well people that incorrectly yields a positive result – is about one-half of one percent, or 5 in 1000.

(Disclaimer: these numbers are unverified and are used for illustration purposes only.)

In other words,

$$P(+|W) = P(W+)/P(W) = 5/1000 = 1/200$$

On the other hand, the same source tells us that chance of a false negative is 1 in 4, so

$$P(-|S) = P(S-)/P(S) = .25.$$

Since every test is either positive or negative, $P(S-) + P(S+) = P(S)$ and so

$$P(+|S) = .75.$$

Suppose furthermore that the overall incidence of COVID-19 in the population is p . In other words, $P(S) = p$ so $P(W) = 1 - p$. Then

$$P(S+) = P(S)P(+|S) = .75p$$

and

$$P(W+) = P(W)P(+|W) = .005(1 - p).$$

Putting these together we get $P(+) = .005 + .745p$

We are interested in $P(S|+)$ – the chance that I’m sick, given that my test result was positive. By Bayes Theorem,

$$P(S|+) = \frac{P(+|S)P(S)}{P(+)} = .75p/ (.005 + .745p) = \frac{750p}{5 + 745p}.$$

As Figure 3.2 shows, if the population incidence is low then a positive test is far from conclusive. Indeed, if the overall incidence of COVID is one percent, then a positive test result only implies a 60 percent chance that I am in fact infected.

Just to fill out the picture, we have

$$P(-) = P(S-) + P(W-) = (P(S) - P(S+)) + (P(W) - P(W+))$$

which yields

$$P(-) = 1 - .005 + .005p - .75p = .995 - .745p.$$

Using Bayes Theorem, we obtain

$$P(S|-) = \frac{P(-|S)P(S)}{P(-)} = .25p/ (.995 - .745p) = \frac{250p}{995 - 745p}.$$

In this case, even though the false negative rate is pretty high (25 percent) overall, if the population incidence is one percent, then the probability that you’re sick given a negative result is only about .25 percent. So negative results are very likely correct.

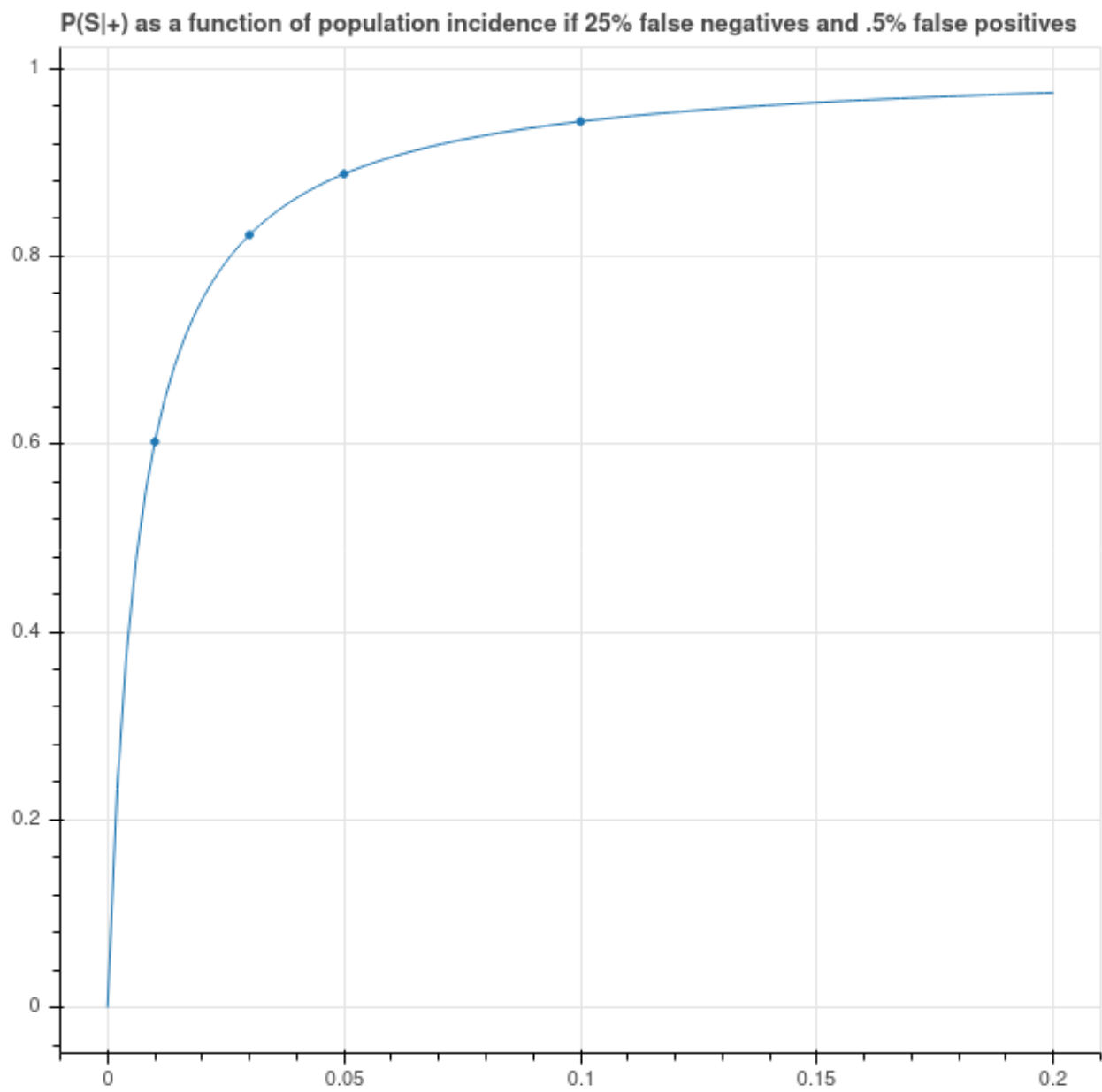


Figure 3.2: $P(S|+)$ vs $P(S)$

3.4 Independence

Independence is one of the fundamental concepts in probability theory. Conceptually, two events are independent if the occurrence of one has does not influence the likelihood of the occurrence of the other. For example, successive flips of a coin are independent events, since the result of the second flip doesn't have anything to do with the result of the first. On the other hand, whether or not it rains today and tomorrow are not independent events, since the weather tomorrow depends (in a complicated way) on the weather today.

We can formalize this idea of independence using the following definition.

Definition: Let X be a sample space and let A and B be two events. Then A and B are *independent* if $P(A \cap B) = P(A)P(B)$. Equivalently, A and B are independent if $P(A|B) = P(A)$ and $P(B|A) = P(B)$.

3.4.1 Examples

3.4.1.1 Coin Flipping

Suppose our coin has a probability of heads given by a real number p between 0 and 1, and we flip our coin N times. What is the chance of getting k heads, where $0 \leq k \leq N$? Any particular sequence of heads and tails containing k heads and $N - k$ tails has probability

$$P(\text{a particular sequence of } k \text{ heads among } N \text{ flips}) = p^k(1-p)^{N-k}.$$

In addition, there are $\binom{N}{k}$ sequences of heads and tails containing k heads. Thus the probability $P(k, N)$ of k heads among N flips is

$$P(k, N) = \binom{N}{k} p^k (1-p)^{N-k}. \quad (3.2)$$

Notice that the binomial theorem gives us $\sum_{k=0}^N P(k, N) = 1$ which is a reassuring check on our work.

The probability distribution on the set $X = \{0, 1, \dots, N\}$ given by $P(k, N)$ is called the *binomial distribution* with parameters N and p .

3.4.1.2 A simple 'mixture'

Now let's look at an example of events that are not independent. Suppose that we have two coins, with probabilities of heads p_1 and p_2 respectively; and assume these probabilities are different. We play the a game in which we first choose one of the two coins (with equal chance) and then flip it twice. Is the result of the second flip independent of the first? In other words, is $P(HH) = P(H)^2$?

This type of situation is called a 'mixture distribution' because the probability of a head is a "mixture" of the probability coming from the two different coins.

The chance that the first flip is a head is $(p_1 + p_2)/2$ because it's the chance of picking the first coin, and then getting a head, plus the chance of picking the second, and then getting a head. The chance of getting two heads in a row is $(p_1^2 + p_2^2)/2$ because it's the chance, having picked the first coin, of getting two heads, plus the chance, having picked the second, of getting two heads.

Since

$$\frac{p_1^2 + p_2^2}{2} \neq \left(\frac{p_1 + p_2}{2} \right)^2$$

we see these events are not independent.

In terms of conditional probabilities, the chance that the second flip is a head, given that the first flip is, is computed as:

$$P(HH|H) = \frac{p_1^2 + p_2^2}{p_1 + p_2}.$$

From the Cauchy-Schwartz inequality one can show that

$$\frac{p_1^2 + p_2^2}{p_1 + p_2} > \frac{p_1 + p_2}{2}.$$

Why should this be? Why should the chance of getting a head on the second flip go up given that the first flip was a head? One way to think of this is that the first coin flip contains a little bit of information about which coin we chose. If, for example $p_1 > p_2$, and our first flip is heads, then it's just a bit more likely that we chose the first coin. As a result, the chance of getting another head is just a bit more likely than if we didn't have that information. We can make this precise by considering the conditional probability $P(p = p_1|H)$ that we've chosen the first coin given that we flipped a head. From Bayes' theorem:

$$P(p = p_1|H) = \frac{P(H|p = p_1)P(p = p_1)}{P(H)} = \frac{p_1}{p_1 + p_2} = \frac{1}{1 + (p_2/p_1)} > \frac{1}{2}$$

since $(1 + (p_2/p_1)) < 2$.

Exercise: Push this argument a bit further. Let $p_1 = \max(p_1, p_2)$. Let P_N be the conditional probability of getting heads assuming that the first N flips were heads. Show that $P_N \rightarrow p_1$ as $N \rightarrow \infty$. All those heads piling up make it more and more likely that you're flipping the first coin and so the chance of getting heads approaches p_1 .

3.4.1.3 An example with a continuous distribution

Suppose that we return to our example of a thermometer which measures the ambient temperature with an error that is distributed according to the normal distribution, as in Equation 3.1. Suppose that we make 10 independent measurements $t_1, \dots, t_{10}$ of the true temperature t_0 . What can we say about the distribution of these measurements?

In this case, independence means that

$$P = P(|t_1 - t_0| < \delta, |t_2 - t_0| < \delta, \dots) = P(|t_1 - t_0| < \delta)P(|t_2 - t_0| < \delta) \cdots P(|t_{10} - t_0| < \delta)$$

and therefore

$$P = \left(\frac{1}{\sigma\sqrt{2\pi}} \right)^{10} \int_{-\delta}^{\delta} \cdots \int_{-\delta}^{\delta} e^{-(\sum_{i=1}^{10} x_i^2)/2\sigma^2} dx_1 \cdots dx_{10}$$

One way to look at this type of multivariate distribution is that the vector $\mathbf{e}$ of errors ($|t_1 - t_0|, \dots, |t_{10} - t_0|$) is distributed according to a *multivariate gaussian distribution*:

$$P(\mathbf{e} \in U) = \left(\frac{1}{\sigma\sqrt{2\pi}} \right)^{10} \int_U e^{-\|\mathbf{x}\|^2/2\sigma^2} d\mathbf{x} \quad (3.3)$$

where x is the vector of differences $t_i - t_0$ and U is a region in $\mathbf{R}^{10}$.

In particular, the magnitude $\|x\|$ of the error vector x is normally distributed.

3.5 Random Variables, Mean, and Variance

Typically, when we are studying a random process, we aren't necessarily accessing the underlying events, but rather we are making measurements that provide us with some information about the underlying events. For example, suppose our sample space X is the set of throws of a pair of dice, so X contains the 36 possible combinations that can arise from the throws. What we are actually interested is the sum of the values of the two dice – that's our "measurement" of this system. This rather vague notion of a measurement of a random system is captured by the very general idea of a *random variable*.

Definition: Let X be a sample space with probability function P . A *random variable* on X is a function $f : X \rightarrow \mathbb{R}$.

Given a random variable f , we can use the probability measure to decide how likely f is to take a particular value, or values in a particular set by the formula

$$P(f(x) \in U) = P(f^{-1}(U))$$

In the dice rolling example, the random variable S that assigns their sum to the pair of values obtained on two dice is a random variable. Those values lie between 2 and 12 and we have

$$P(S = k) = P(S^{-1}(\{k\})) = P(\{(x, y) : x + y = k\})$$

where (x, y) runs through $\{1, 2, \dots, 6\}^2$ representing the two values and $P((x, y)) = 1/36$ since all throws are equally likely.

Let's look at a few more examples, starting with what is probably the most fundamental of all.

Definition: Let X be a sample space with two elements, say H and T , and suppose that $P(H) = p$ for some $0 \leq p \leq 1$. Then the random variable that satisfies $f(H) = 1$ and $f(T) = 0$ is called a Bernoulli random variable with parameter p .

In other words, a Bernoulli random variable gives the value 1 when a coin flip is heads, and 0 for tails.

Now let's look at what we earlier called the binomial distribution.

Definition: Let X be a sample space consisting of strings of H and T of length N , with the probability of a *particular string* S with k heads and $N - k$ tails given by

$$P(S) = p^k(1 - p)^{N-k}$$

for some $0 \leq p \leq 1$. In other words, X is the sample space consisting of N independent flips of a coin with probability of heads given by p .

Let $f : X \rightarrow \mathbb{R}$ be the function which counts the number of H in the string. We can express f in terms of Bernoulli random variables; indeed,

$$f = X_1 + \dots + X_N$$

where each X_i is a Bernoulli random variable with parameter p .

Now

$$P(f = k) = \binom{N}{k} p^k (1 - p)^{N-k}$$

since $f^{-1}(\{k\})$ is the number of elements in the subset of strings of H and T of length N containing exactly k H 's. This is our old friend the binomial distribution. So *a binomial distribution is the distribution of the sum of N independent Bernoulli random variables.*

For an example with a continuous random variable, suppose our sample space is $\mathbf{R}^2$ and the probability density is the simple multivariate normal

$$P(\mathbf{x} \in U) = \left(\frac{1}{\sqrt{2\pi}} \right)^2 \int_U e^{-\|\mathbf{x}\|^2/2} d\mathbf{x}.$$

Let f be the random variable $f(\mathbf{x}) = \|\mathbf{x}\|$. The function f measures the Euclidean distance of a randomly drawn point from the origin. The set

$$U = f^{-1}([0, r]) \subseteq \mathbf{R}^2$$

is the circle of radius r in $\mathbf{R}^2$. The probability that a randomly drawn point lies in this circle is

$$P(f < r) = \left(\frac{1}{\sqrt{2\pi}} \right)^2 \int_U e^{-\|\mathbf{x}\|^2/2} d\mathbf{x}.$$

We can actually evaluate this integral in closed form by using polar coordinates. We obtain

$$P(f < r) = \left(\frac{1}{\sqrt{2\pi}} \right)^2 \int_{\theta=0}^{2\pi} \int_{\rho=0}^r e^{-\rho^2/2} \rho d\rho d\theta.$$

Since

$$\frac{d}{d\rho} e^{-\rho^2/2} = -\rho e^{-\rho^2/2}$$

3 Probability and Bayes Theorem

we have

$$\begin{aligned} P(f < r) &= -\frac{1}{2\pi} \theta e^{-\rho^2/2} \Big|_{\theta=0}^{2\pi} \Big|_{\rho=0}^r \\ &= 1 - e^{-r^2/2} \end{aligned}$$

The probability density associated with this random variable is the derivative of $1 - e^{-r^2/2}$

$$P(f \in [a, b]) = \int_{r=a}^b r e^{-r^2/2} dr$$

as you can see by the fundamental theorem of calculus. This density is drawn in Figure 3.3 where you can see that the points are clustered at a distance of 1 from the origin.

3.5.1 Independence and Random Variables

We can extend the notion of independence from events to random variables.

Definition: Let f and g be two random variables on a sample space X with probability P . Then f and g are independent if, for all intervals U and V in $\mathbb{R}$, the events $f^{-1}(U)$ and $g^{-1}(V)$ are independent.

For discrete probability distributions, this means that, for all $a, b \in \mathbb{R}$,

$$P(f = a \text{ and } g = b) = P(f = a)P(g = b).$$

For continuous probability distributions given by a density function $P(x)$, independence can be more complicated to figure out.

3.5.2 Expectation, Mean and Variance

The most fundamental tool in the study of random variables is the concept of “expectation”, which is a fancy version of average. The word “mean” is a synonym for expectation – the mean of a random variable is the same as its expectation or “expected value.”

Definition: Let X be a sample space with probability measure P . Let $f : X \rightarrow \mathbb{R}$ be a random variable. Then the *expectation* or *expected value* $E[f]$ of f is

$$E[f] = \int_X f(x) dP.$$

More specifically, if X is discrete, then

$$E[f] = \sum_{x \in X} f(x) P(x)$$

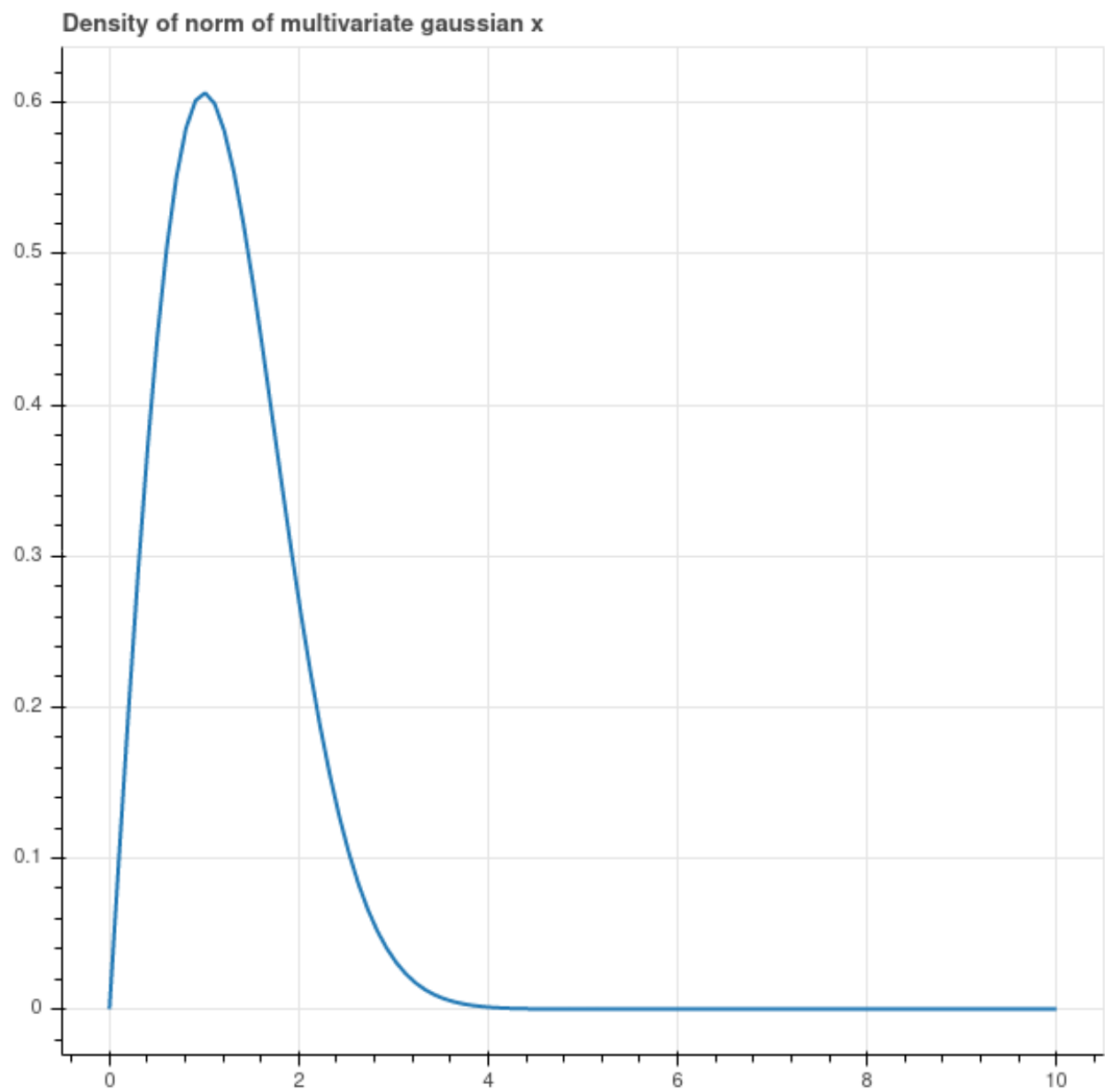


Figure 3.3: Density of the Norm

3 Probability and Bayes Theorem

while if X is continuous with probability density function $p(x)dx$ then

$$E[f] = \int_X f(x)p(x)dx.$$

If f is a Bernoulli random variable with parameter p , then

$$E[f] = 1 \cdot p + 0 \cdot (1 - p) = p$$

If f is a binomial random variable with parameters p and N , then

$$E[f] = \sum_{i=0}^N i \binom{N}{i} p^i (1-p)^{N-i}$$

One can evaluate this using some combinatorial tricks, but it's easier to apply this basic fact about expectations.

Proposition: Expectation is linear: $E[aX + bY] = aE[X] + bE[Y]$ for random variables X, Y and constants a and b .

The proof is an easy consequence of the expression of E as a sum (or integral).

Since a binomial random variable Z with parameters N and p is the sum of N Bernoulli random variables, its expectation is

$$E[X_1 + \dots + X_N] = Np.$$

A more sophisticated property of expectation is that it is multiplicative when the random variables are independent.

Proposition: Let f and g be two independent random variables. Then $E[fg] = E[f]E[g]$.

Proof: Let's suppose that the sample space X is discrete. By definition,

$$E[f] = \sum_{x \in X} f(x)P(x)$$

and we can rewrite this as

$$E[f] = \sum_{a \in \mathbf{R}} aP(\{x : f(x) = a\}).$$

Let $Z \subset \mathbf{R}$ be the range of f . Then

$$\begin{aligned} E[fg] &= \sum_{a \in Z} aP(\{x : fg(x) = a\}) \\ &= \sum_{a \in Z} \sum_{(u,v) \in \mathbf{Z}^2 \atop uv=a} aP(\{x : f(x) = u \text{ and } g(x) = v\}) \\ &= \sum_{a \in Z} \sum_{\mathbf{Z}^2 \atop uv=a} uvP(\{x : f(x) = u\})P(\{x : g(x) = v\}) \\ &= \sum_{u \in Z} uP(\{x : f(x) = u\}) \sum_{v \in Z} vP(\{x : f(x) = v\}) \\ &= E[f]E[g] \end{aligned}$$

3.5.2.1 Variance

The variance of a random variable is a measure of its dispersion around its mean.

Definition: Let f be a random variable. Then the variance is the expression

$$\sigma^2(f) = E[(f - E[f])^2] = E[f^2] - (E[f])^2$$

The square root of the variance is called the “standard deviation.”

The two formulae for the variance arise from the calculation

$$E[(f - E[f])^2] = E[(f^2 - 2fE[f] + E[f]^2)] = E[f^2] - 2E[f]^2 + E[f]^2 = E[f^2] - E[f]^2.$$

To compute the variance of the Bernoulli random variable f with parameter p , we first compute

$$E[f^2] = p(1)^2 + (1-p)0^2 = p.$$

Since $E[f] = p$, we have

$$\sigma^2(f) = p - p^2 = p(1 - p).$$

If f is the binomial random variable with parameters N and p , we can again use the fact that f is the sum of N Bernoulli random variables $X_1 + \dots + X_n$ and compute

$$\begin{aligned} E[(\sum_i X_i)^2] - E[\sum_i X_i]^2 &= E[\sum_i X_i^2 + \sum_{i,j} X_i X_j] - N^2 p^2 \\ &= Np + N(N-1)p^2 - N^2 p^2 \\ &= Np(1 - p) \end{aligned}$$

where we have used the fact that the square X^2 of a Bernoulli random variable is equal to X .

For a continuous example, suppose that we consider a sample space $\mathbb{R}$ with the normal probability density

$$P(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-x^2/2\sigma^2} dx.$$

The mean of the random variable x is

$$E[x] = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{\infty} x e^{-x^2/2\sigma^2} dx = 0$$

since the function being integrated is odd. The variance is

$$E[x^2] = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{\infty} x^2 e^{-x^2/2\sigma^2} dx.$$

The trick to evaluating this integral is to consider the derivative:

$$\frac{d}{d\sigma} \left[\frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-x^2/(2\sigma^2)} dx \right] = 0$$

where the result is zero since the quantity being differentiated is a constant (namely 1). Sorting through the resulting equation leads to the fact that

$$E[x^2] = \sigma^2$$

so that the σ^2 parameter in the normal distribution really *is* the variance of the associated random variable.

3.5.2.2 Covariance and the Multivariate Normal Distribution

Given two random variables f and g , we can measure how much they vary together by looking at their covariance, which is the expected value of the product of their deviations from their means:

$$\text{Cov}(f, g) = E[(f - E[f])(g - E[g])] = E[fg] - E[f]E[g].$$

If f and g are independent, then their covariance is zero.

In our discussion of linear regression and principal components, we made use of the “covariance matrix” D_0 which was computed from sample data as $X_0^T X_0 / N$ where X_0 was the mean-centered data matrix. The entries of this matrix were the covariances of samples from the various random variables corresponding to the columns of X .

The theoretical origin of the covariance matrix arises from the “multivariate normal distribution” which generalizes the one-dimensional normal distribution and the more general distribution of n independent normal random variables.

A general multivariate gaussian can be constructed from a covariance matrix D_0 , where D_0 is an $n \times n$, symmetric, positive definite matrix like we considered in the chapter on principal component analysis. In this case, the probability that an n -dimensional vector x lies in a set $U \subset \mathbf{R}^n$ is given by the integral

$$P(x \in U) = \frac{1}{(2\pi)^{n/2} \sqrt{\det(D_0)}} \int_U e^{-x^T D_0^{-1} x / 2} d\mathbf{x}.$$

If a random vector $x = (x_1, \dots, x_n)$ has this distribution, then the individual components x_i are normally distributed with variances given by the diagonal entries of D_0 , and covariances corresponding to the off-diagonal entries. In other words, D_0 is the covariance matrix of the components of x .

To see this, imagine that x has the given distribution. Using the spectral theorem, we can write $D_0 = V\Lambda V^T$ where V is an orthogonal matrix and Λ is a diagonal matrix with positive entries $\lambda_1, \dots, \lambda_n$.

If $y = V^T x$, which is a rotation of the random vector $x = Vy$, then the exponential in the probability density function becomes

$$x^T D_0^{-1} x = y^T V^T D_0^{-1} Vy = y^T V^T V \Lambda^{-1} V^T Vy = y^T \Lambda^{-1} y.$$

Since Λ is diagonal, the normalizing constant $\sqrt{\det(D_0)} = \sqrt{\prod_{i=1}^n \lambda_i}$ becomes the square root of the product of the variances λ_i , and the density function is visibly the product of n independent normal distributions with variances λ_i .

In other words, the random vector v , when rotated into the coordinates given by the eigenvectors of D_0 , becomes a vector y whose components are independent normal random variables with variances λ_i .

Working backwards, we can find the covariance matrix of the x_i . The matrix $n \times n$ matrix xx^T has entries $x_i x_j$, so the expected values of the entries are the variances/covariances of the x_i . But

$$E(xx^T) = E(Vyy^T V^T) = VE(yy^T)V^T = V\Lambda V^T = D_0.$$

In other words, the covariance matrix of the components of x is D_0 .

This density function as a “bump” concentrated near the origin in $\mathbf{R}^2$, and its level curves are a family of ellipses centered at the origin. See Figure 3.4 for a plot of this function with $\sigma = 1$.

In this situation we can look at the conditional probability of the first variable given the second, and see that the two variables are not independent. Indeed, if we fix x_2 , then the distribution of x_1 depends on our choice of x_2 . We could see this by a calculation, or we can just look at the graph: if $x_2 = 0$, then the most likely values of x_1 cluster near zero, while if $x_2 = 1$, then the most likely values of x_1 cluster somewhere above zero.

3.5.2.3 Application: a tiny bit of portfolio theory

We can see one application of the multivariate gaussian distribution in a very simple model from financial mathematics.

Suppose we look at a collection of n stocks. We make the very naive assumption that the performance of these stocks is given by (for each stock) an average price per share of μ_i with variance σ_i^2 for $i = 1, \dots, n$. Rule out longer term trends over time and assume that each stock fluctuates around its average price per share according to a normal distribution with mean μ_i and variance σ_i^2 .

We can think of the individual variances σ_i^2 as measuring the “risk” associated with each stock. A high variance means the return fluctuates a lot around the mean return μ_i .

It would be a mistake, however, to think that the performance of these stocks is independent of each other. For example, the performance of stocks of companies in the same industry, tend to be correlated, and all stocks are correlated with the underlying economy. So a reasonable model for the collective performance of these stocks is the return is given by a multivariate gaussian

Multivariate Gaussian

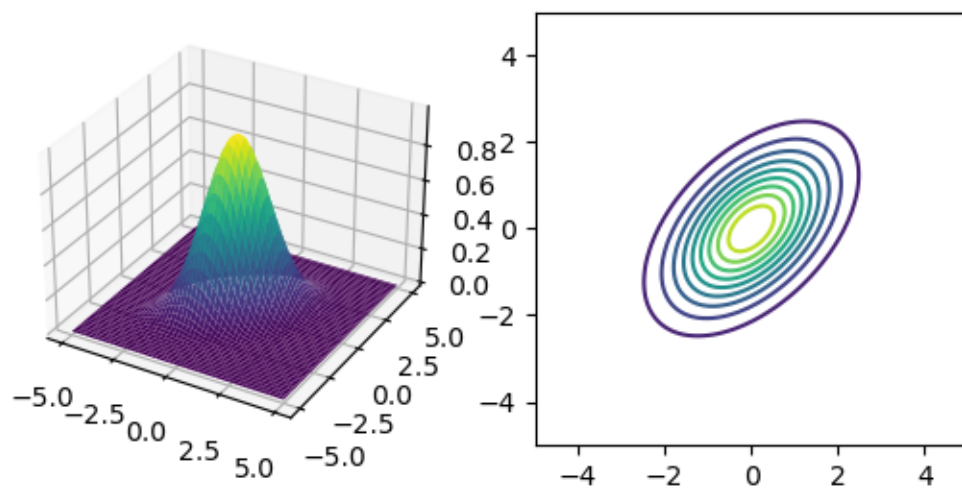


Figure 3.4: Multivariate Gaussian

distribution associated with an $n \times n$ covariance matrix. It's worth noting that, in finance, the covariance between the performance of two securities is typically called “beta”.

To make this concrete, suppose we are dealing with two securities and their covariance matrix is

$$D = \begin{pmatrix} 1.5 & .7 \\ .7 & 1.0 \end{pmatrix}$$

If we sample the returns of these stocks (centered at their means, so the distribution of returns is around the origin) we get a plot like this.



Figure 3.5: Correlated Stock Returns

Notice that the stocks perform well or poorly together reflecting the fact that their returns are correlated.

Now suppose we have 1000 dollars to invest in these stocks. We can buy p_i shares of each stock and we must have

$$\sum p_i \mu_i = 1000.$$

3 Probability and Bayes Theorem

The risk associated with this “portfolio” is captured by the variance of this linear combination of the normally distributed stock returns. That, in turn, is yet another example of the variance of a “score” and it is given by

$$\sigma^2 = [p_1 \quad \cdots \quad p_n] D_0 \begin{bmatrix} p_1 \\ \cdots \\ p_n \end{bmatrix}$$

which works out to the quadratic function

$$\sigma^2 = \sum_{i,j} p_i p_j \sigma_{ij}$$

where σ_{ij} is the covariance of stock i with stock j (and $\sigma_{ii} = \sigma_i^2$ is the variance of stock i .)

A reasonable goal for us would be to invest our money so as to obtain *the same return* with *the minimum risk*. In this setting, that means we want to choose p_i so that $\sum p_i \mu_i = 1000$ and σ^2 is minimized.

This is a constrained optimization problem that we can solve analytically by Lagrange multipliers. If we consider only two dimensions for simplicity, the objective function (the risk) is a quadratic:

$$\sigma^2(p_1, p_2) = \sigma_1^2 p_1^2 + 2\sigma_{12} p_1 p_2 + \sigma_2^2 p_2^2$$

while the constraint is a line $p_1 \mu_1 + p_2 \mu_2 = 1000$.

The Lagrange formulation is

$$L(p_1, p_2, \lambda) = \sigma_1^2 p_1^2 + 2\sigma_{12} p_1 p_2 + \sigma_2^2 p_2^2 - \lambda(p_1 \mu_1 + p_2 \mu_2 - 1000)$$

The geometry looks like this graph:

and the point of minimum risk occurs on the “risk ellipse” that is tangent to the “portfolio line.”

The most significant conclusion one obtains from this analysis is that, in general, *a diversified portfolio generates the same return at lower risk than a single security*. In our graph, the point of tangency occurs at a point where both p_1 and p_2 are nonzero, meaning we have bought a mix of both stocks.

3.6 Models and Likelihood

A *statistical model* is a mathematical model that accounts for data via a process that incorporates random behavior in a structured way. We have seen several examples of such models in our discussion so far. For example, the Bernoulli process that describes the outcome of a series of coin flips as independent choices of heads or tails with probability p is a simple statistical model; our more complicated mixture model in which we choose one of two coins at random and then flip that is a more complicated model.

Our description of the variation in temperature measurements as arising from perturbations from the true temperature by a normally distributed amount is another example of a statistical model, this one involving a continuous random variable.

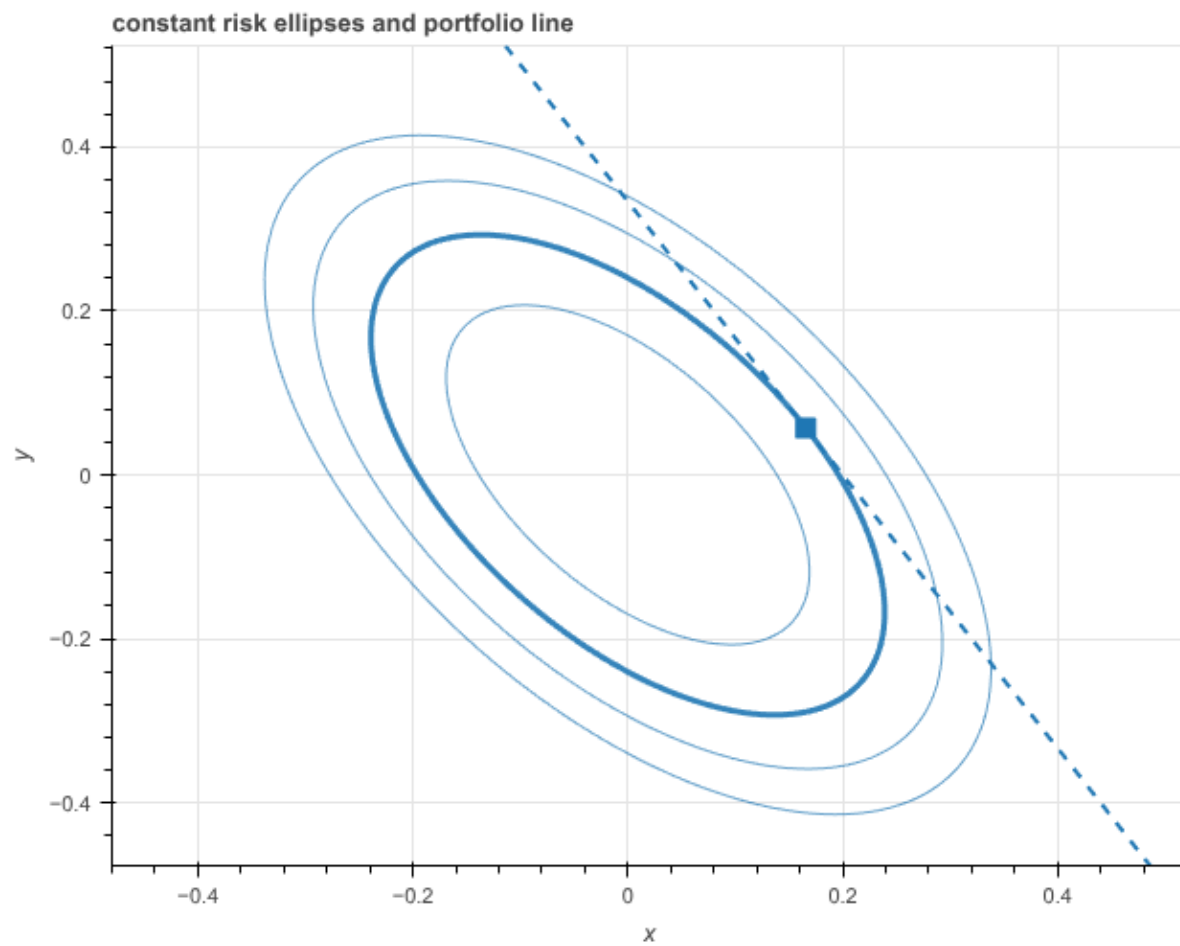


Figure 3.6: Constant Risk with Portfolio Line

When we apply a mathematical model to understand data, we often have a variety of parameters in the model that we must adjust to get the model to best “fit” the observed data. For example, suppose that we observe the vibrations of a block attached to a spring. We know that the motion is governed by a second order linear differential equation, but the dynamics depend on the mass of the block, the spring constant, and the damping coefficient. By measuring the dynamics of the block over time, we can try to work backwards to figure out these parameters, after which we will be able to predict the block’s motion into the future.

3.6.1 Maximum Likelihood (Discrete Case)

To see this process in a statistical setting, let’s return to the simple example of a coin flip. The only parameter in our model is the probability p of getting heads on a particular flip. Suppose that we flip the coin 100 times and get 55 heads and 45 tails. What can we say about p ?

We will approach this question via the “likelihood” function for our data. We ask: for a particular value of the parameter p , how likely is this outcome? From Equation 3.2 we have

$$P(55H, 45T) = \binom{100}{55} p^{55} (1-p)^{45}.$$

This function is plotted in Figure 3.7. As you can see from that plot, it is extremely unlikely that we would have gotten 55 heads if p was smaller than .4 or greater than .7, while the *most likely* value of p occurs at the maximum value of this function, and a little calculus tells us that this point is where $p = .55$. This *most likely* value of p is called the *maximum likelihood estimate* for the parameter p .

3.6.2 Maximum Likelihood (Continuous Case)

Now let’s look at our temperature measurements where the error is normally distributed with variance parameter σ^2 . As we have seen earlier, the probability density of errors $\mathbf{x} = (x_1, \dots, x_n)$ of n independent measurements is

$$P(\mathbf{x}) = \left(\frac{1}{\sigma\sqrt{2\pi}} \right)^n e^{-\|\mathbf{x}\|^2/(2\sigma^2)} d\mathbf{x}.$$

(see Equation 3.3). What should we use as the parameter σ ? We can ask which choice of σ makes our data *most likely*. To calculate this, we think of the probability of a function of σ and use Calculus to find the maximum. It’s easier to do this with the logarithm.

$$\log P(\mathbf{x}) = \frac{-\|\mathbf{x}\|^2}{2\sigma^2} - n \log \sigma + C$$

where C is a constant that we’ll ignore. Taking the derivative and setting it to zero, we obtain

$$-\|\mathbf{x}\|^2 \sigma^{-3} - n \sigma^{-1} = 0$$

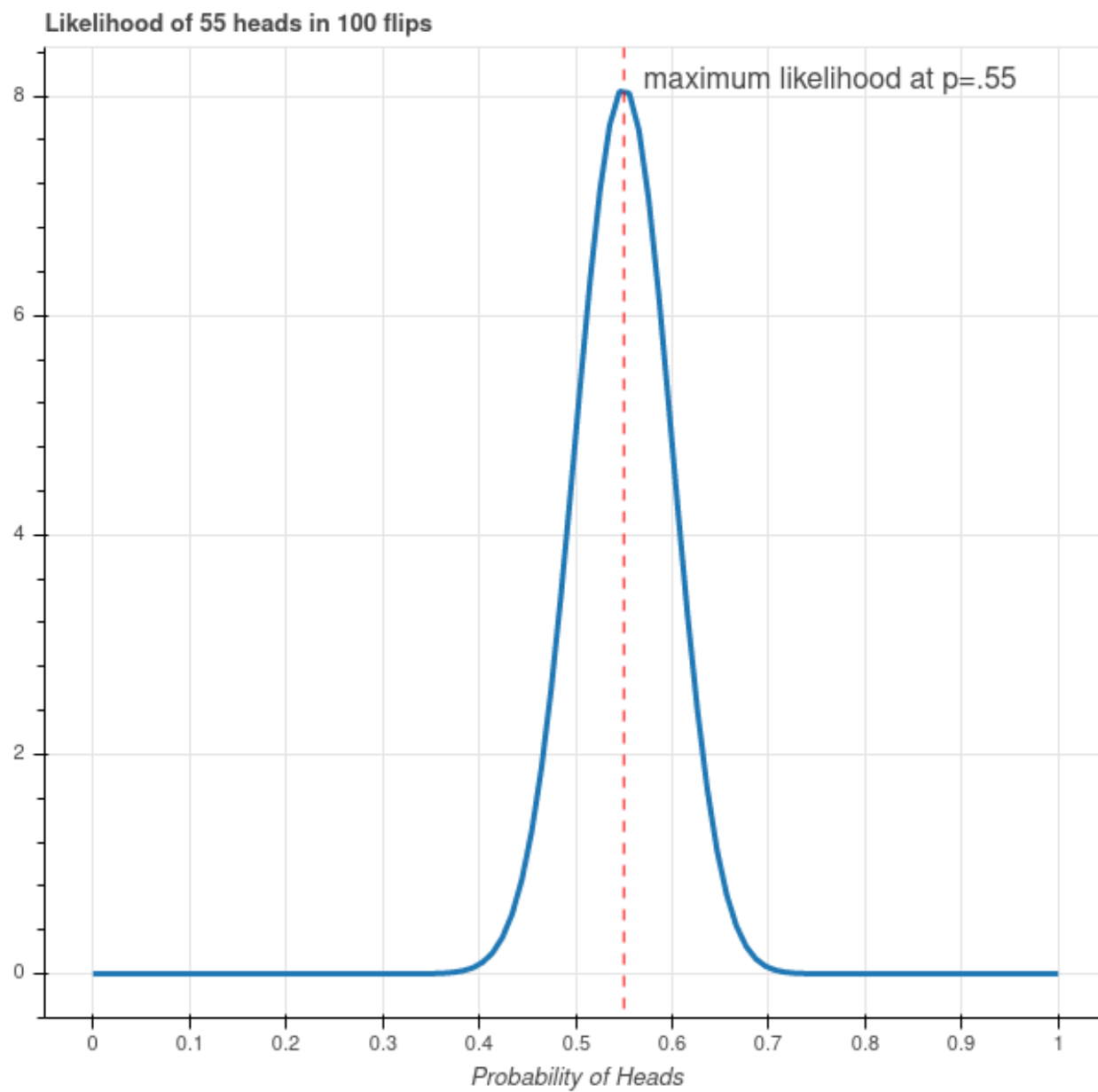


Figure 3.7: Likelihood Plot

which gives the formula

$$\sigma^2 = \frac{\|\mathbf{x}\|^2}{n}$$

This should look familiar! The maximum likelihood estimate of the variance is the *mean-squared-error*.

3.6.3 Linear Regression and likelihood

In our earlier lectures we discussed linear regression at length. Our introduction of ideas from probability give us new insight into this fundamental tool. Consider a statistical model in which certain measured values y depend linearly on x up to a normally distributed error:

$$y = mx + b + \epsilon$$

where ϵ is drawn from the normal distribution with variance σ^2 .

The classic regression setting has us measuring a collection of N points (x_i, y_i) and then asking for the “best” m , b , and σ^2 to explain these measurements. Using the likelihood perspective, each value $y_i - mx_i - b$ is an independent draw from the normal distribution with variance σ^2 , exactly like our temperature measurements in the one variable case.

The likelihood (density) of those draws is therefore

$$P = \left(\frac{1}{\sigma\sqrt{2\pi}} \right)^N e^{-\sum_i (y_i - mx_i - b)^2 / (2\sigma^2)}.$$

What is the maximum likelihood estimate of the parameters m , b , and σ^2 ?

To find this we look at the logarithm of P and take derivatives.

$$\log(P) = -N \log(\sigma) - \frac{1}{2\sigma^2} \sum_i (y_i - mx_i - b)^2.$$

As far as m and b are concerned, the minimum comes from the derivatives with respect to m and b of

$$\sum_i (y_i - mx_i - b)^2.$$

In other words, the maximum likelihood estimate m_* and b_* for m and b are *exactly the ordinary least squares estimates*.

As far as σ^2 is concerned, we find just as above that the maximum likelihood estimate σ_*^2 is the mean squared error

$$\sigma_*^2 = \frac{1}{N} \sum_i (y_i - m_* x_i - b_*)^2.$$

The multivariate case of regression proposes a model of the form

$$Y = X\beta + \epsilon$$

and a similar calculation again shows that the least squares estimates for β are the maximum likelihood values for this model.

4 Bayesian Inference

4.1 Bayesian Perspective on Data

We conclude our review of ideas from probability by examining the Bayesian perspective on data.

Suppose that we wish to conduct an experiment to determine the temperature outside our house. We begin our experiment with a statistical model that is supposed to explain the variability in the results. The model depends on some parameters that we wish to estimate. For example, the parameters of our experiment might be the ‘true’ temperature t_* and the variance σ^2 of the error.

From the Bayesian point of view, at the beginning of this experiment we have an initial sense of what the temperature is likely to be, expressed in the form of a probability distribution. This initial information is called the *prior* distribution.

For example, if we know that it’s December in Connecticut, our prior distribution might say that the temperature is more likely to be between 20 and 40 degrees Fahrenheit and is quite unlikely to be higher than 60 or lower than 0. So our prior distribution might look like Figure 4.1.

If we really have no opinion about the temperature other than it being between, say, -20 and 100 degrees, our prior distribution might be uniform over that range, as in Figure 4.2.

The choice of a prior will guide the interpretation of our experiments in ways that we will see shortly.

The next step in our experiment is the collection of data. Suppose we let $\mathbf{t} = (t_1, t_2, \dots, t_n)$ be a random variable representing n independent measurements of the temperature. We consider the likelihood of the parameters t_* and σ^2 and the possible measurements $\mathbf{t}$:

$$P(\mathbf{t} \mid t_*, \sigma^2) = \left(\frac{1}{\sigma\sqrt{2\pi}} \right)^n e^{-\|\mathbf{t} - t_*\mathbf{e}\|^2 / (2\sigma^2)}$$

where $\mathbf{e} = (1, 1, \dots, 1)$.

The conditional probability $P(t_*, \sigma^2 \mid \mathbf{t})$ is the distribution of the values of t_* and σ^2 *given* a value of the $\mathbf{t}$. This is what we hope to learn by our experiment – namely, if we make a particular measurement, what does it tell us about t_* and σ^2 ?

Now suppose that we actually make some measurements, and so we obtain a specific set of values $\mathbf{t}_0$ for $\mathbf{t}$.

By Bayes Theorem,

$$P(t_*, \sigma^2 \mid \mathbf{t} = \mathbf{t}_0) = \frac{P(\mathbf{t} = \mathbf{t}_0 \mid t_*, \sigma^2) P(t_*, \sigma^2)}{P(\mathbf{t} = \mathbf{t}_0)}$$

We interpret this as follows:

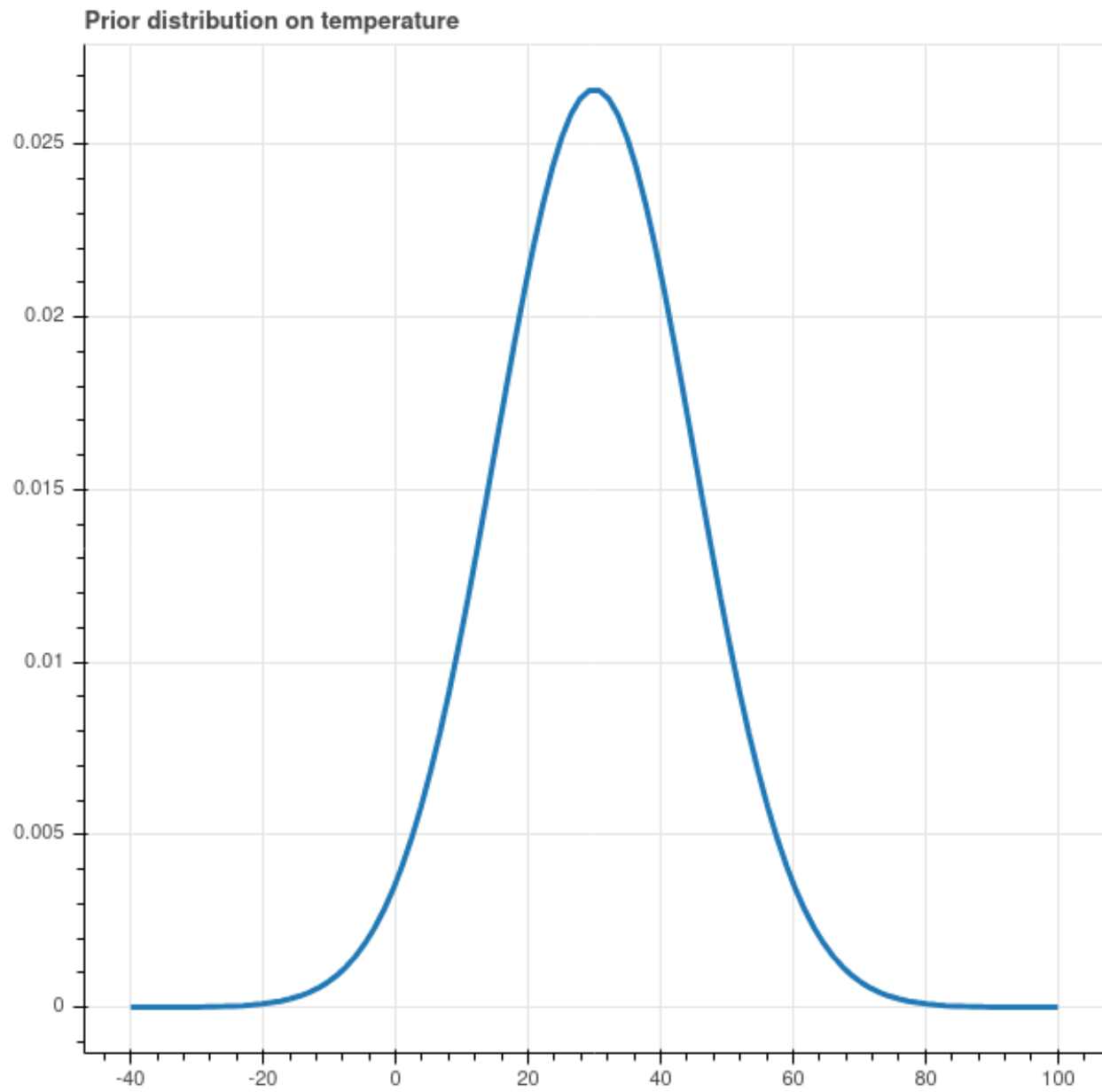


Figure 4.1: Prior Distribution on Temperature

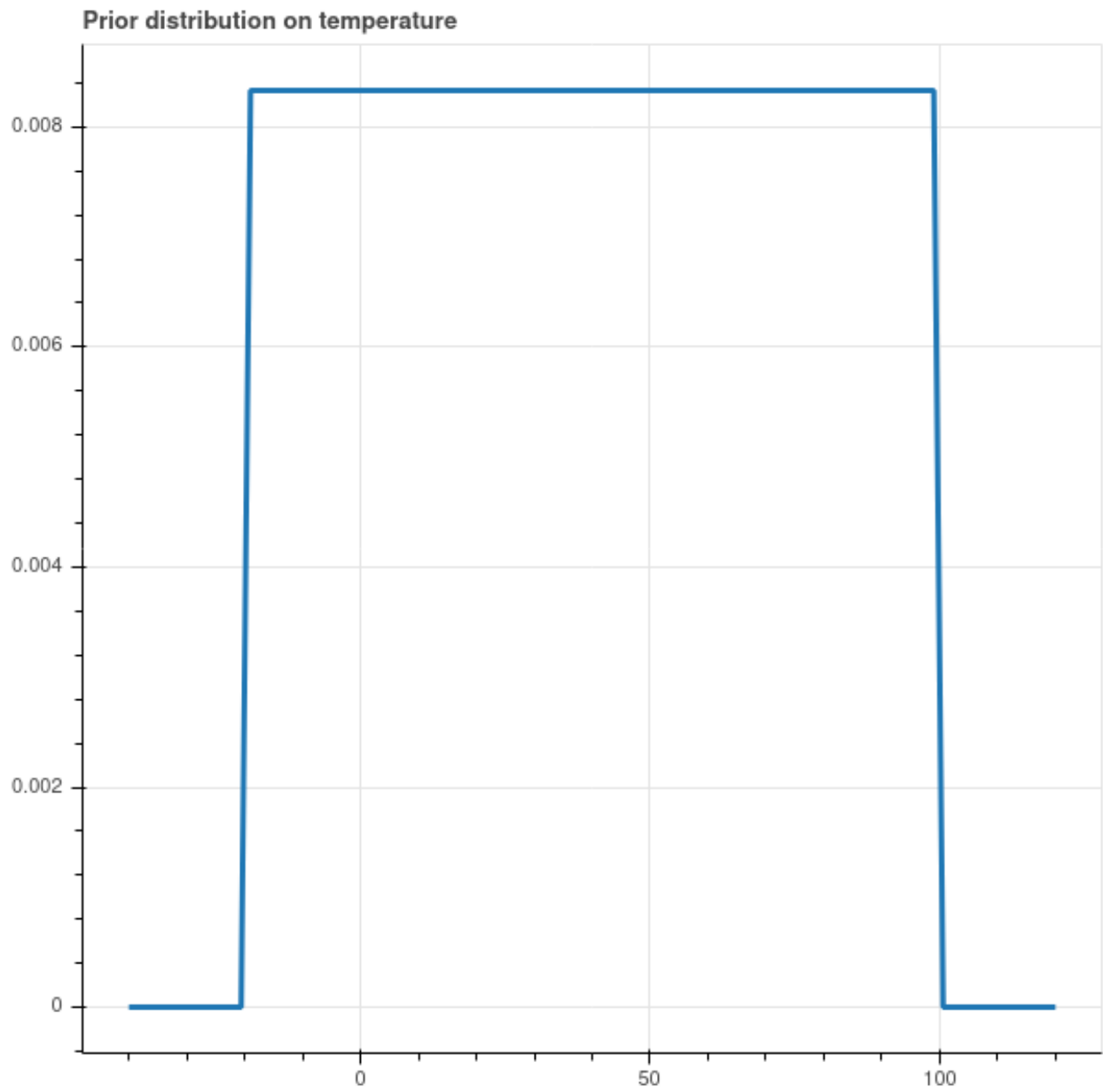


Figure 4.2: Uniform Prior

- the left hand side $P(t_*, \sigma^2 | \mathbf{t} = \mathbf{t}_0)$ is called the *posterior distribution* and is the distribution of t_* and σ^2 obtained by *updating our prior knowledge with the results of our experiment*.
- The probability $P(\mathbf{t} = \mathbf{t}_0 | t_*, \sigma^2)$ is the probability of obtaining the measurements we found for a particular value of the parameters t_* and σ^2 .
- The probability $P(t_*, \sigma^2)$ is the *prior distribution* on the parameters that reflects our initial impression of the distribution of these parameters.
- The denominator $P(\mathbf{t} = \mathbf{t}_0)$ is the total probability of the results that we obtained, and is the integral over the distribution of the parameters weighted by their prior probability:

$$P(\mathbf{t} = \mathbf{t}_0) = \int P(\mathbf{t} = \mathbf{t}_0 | t_*, \sigma^2) P(t_*, \sigma^2) dt_* d\sigma^2$$

4.1.1 Bayesian experiments with the normal distribution

To illustrate these Bayesian ideas, we'll consider the problem of measuring the temperature, but for simplicity let's assume that we fix the variance in our error measurements at 1 degree. Let's use the prior distribution on the true temperature that I proposed in Figure 4.1, which is a normal distribution with variance 15 "shifted" to be centered at 30:

$$P(t_*) = \left(\frac{1}{\sqrt{30\pi}} \right) e^{-(t_*-30)^2/30}.$$

The expected value $E[t_*]$ – the mean of this distribution – is 30.

Since the error in our measurements is normally distributed with variance 1, we have

$$P(t - t_*) = \left(\frac{1}{\sqrt{2\pi}} \right) e^{-(t-t_*)^2/2}$$

or as a function of the absolute temperature, we have

$$P(t) = \left(\frac{1}{\sqrt{2\pi}} \right) e^{-(t-t_*)^2/2}.$$

Now we make a bunch of measurements to obtain $\mathbf{t}_0 = (t_1, \dots, t_n)$. We have

$$P(\mathbf{t} = \mathbf{t}_0 | t_*) = \left(\frac{1}{\sqrt{2\pi}} \right)^n e^{-\|\mathbf{t}_0 - t_* \mathbf{e}\|^2/2}.$$

The total probability $T = P(\mathbf{t} = \mathbf{t}_0)$ is hard to calculate, so let's table that for a while. The posterior probability is

$$P(t_* | \mathbf{t} = \mathbf{t}_0) = \frac{1}{T} \left(\frac{1}{\sqrt{2\pi}} \right)^n e^{-\|\mathbf{t}_0 - t_* \mathbf{e}\|^2/2} \left(\frac{1}{\sqrt{30\pi}} \right) e^{-(t_*-30)^2/30}.$$

Leaving aside the multiplicative constants for the moment, consider the exponential

$$e^{-(\|\mathbf{t} - t_* \mathbf{e}\|^2/2 + (t_*-30)^2/30)}.$$

Since $\mathbf{t}$ is a vector of constants – it is a vector of our particular measurements – the exponent

$$\|\mathbf{t} - t_* \mathbf{e}\|^2/2 + (t_* - 30)^2/30 = (t_* - 30)^2/30 + \sum_i (t_i - t_*)^2/2$$

is a quadratic polynomial in t_* that simplifies:

$$(t_* - 30)^2/30 + \sum_i (t_i - t_*)^2/2 = At_*^2 + Bt_* + C.$$

Here

$$\begin{aligned} A &= \left(\frac{1}{30} + \frac{n}{2}\right), \\ B &= -2 - \sum_i t_i \\ C &= 30 + \frac{1}{2} \sum_i t_i^2. \end{aligned}$$

We can complete the square to write

$$At_*^2 + Bt_* + C = (t_* - U)^2/2V + K$$

where

$$U = \frac{2 + \sum_i t_i}{\frac{1}{15} + n}$$

and

$$V = \frac{1}{\frac{1}{15} + n}.$$

So up to constants that don't involve t_* , the posterior density is of the form

$$e^{-(t_* - U)^2/2V}$$

and since it is a probability density, the constants must work out to give total integral of 1. Therefore the posterior density is a normal distribution centered at U and with variance V . Here U is called the *posterior mean* and V the *posterior variance*.

To make this explicit, suppose $n = 5$ and we measured the following temperatures:

$$40, 41, 39, 37, 44$$

The mean of these observations is 40.2 and the variance is 5.4.

A calculation shows that the posterior mean is 40.1 and the posterior variance is 0.2. Comparing the prior with the posterior, we obtain the plot in Figure 4.3. The posterior has a sharp peak at 40.1 degrees. This value is just a bit smaller than the mean of the observed temperatures which is 40.2 degrees. This difference is caused by the prior – our prior distribution said the temperature was likely to be around 30 degrees, and so the prior pulls the observed mean a bit towards the prior mean taking into account past experience. Because the variance of the prior is large, it has a relatively small influence on the posterior.

The general version of the calculation above is summarized in this proposition.

Proposition: Suppose that our statistical model for an experiment proposes that the measurements are normally distributed around an (unknown) mean value of μ with a (fixed) variance σ^2 . Suppose further that our prior distribution on the unknown mean μ is normal with mean μ_0 and variance τ^2 . Suppose we make measurements

$$y_1, \dots, y_n$$

with mean $\bar{y}$. Then the posterior distribution of μ is again normal, with posterior variance

$$\tau'^2 = \frac{1}{\frac{1}{\tau^2} + \frac{n}{\sigma^2}}$$

and posterior mean

$$\mu' = \frac{\frac{\mu_0}{\tau^2} + \frac{n}{\sigma^2} \bar{y}}{\frac{1}{\tau^2} + \frac{n}{\sigma^2}}$$

So the posterior mean is a sort of weighted average of the sample mean and the prior mean; and as $n \rightarrow \infty$, the posterior mean approaches the sample mean – in other words, as you get more data, the prior has less and less influence on the results of the experiment.

4.1.2 Bayesian coin flipping

For our final example in this fast overview of ideas from probability, we consider the problem of deciding whether a coin is fair. Our experiment consists of N flips of a coin with unknown probability p of heads, so the data consists of the number h of heads out of the N flips. To apply Bayesian reasoning, we need a prior distribution on p . Let's first assume that we have no reason to prefer one value of p over another, and so we choose for our prior the uniform distribution on p between 0 and 1.

We wish to analyze $P(p|h)$, the probability distribution of p given h heads out of N flips. Bayes Theorem gives us:

$$P(p|h) = \frac{P(h|p)P(p)}{P(h)}$$

where

$$P(h|p) = \binom{N}{h} p^h (1-p)^{N-h}$$

and

$$P(h) = \int_{p=0}^1 P(h|p)P(p)dp = \binom{N}{h} \int_{p=0}^1 p^h (1-p)^{N-h} dp$$

is a constant which ensures that

$$\int_p P(p|h)dp = 1.$$

We see that the posterior distribution $P(p|h)$ is proportional to the polynomial function

$$P(p|h) \propto p^h (1-p)^{N-h}.$$

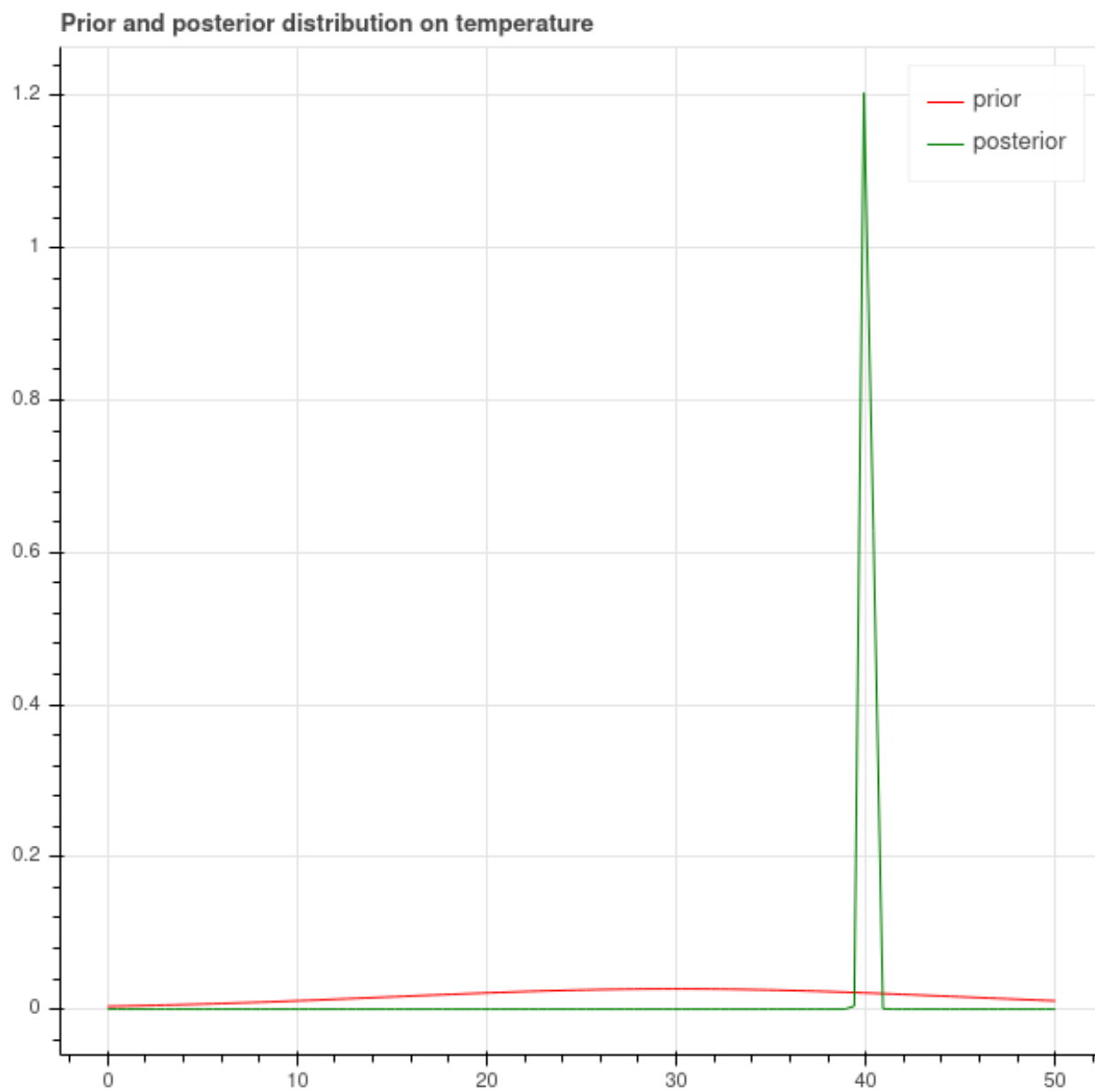


Figure 4.3: Prior and Posterior

4 Bayesian Inference

As in Section 3.6.1, we see that this function peaks at h/N . This is called the maximum *a posteriori* estimate for p .

Another way to summarize the posterior distribution $P(p|h)$ is to look at the expected value of p . This is called the *posterior mean* of p . To compute it, we need to know the normalization constant in the expression for $P(p|h)$, and for that we can take advantage of the properties of a special function $B(a, b)$ called the Beta-function:

$$B(a, b) = \int_{p=0}^1 p^{a-1}(1-p)^{b-1} dp.$$

Proposition: If a and b are integers, then $B(a, b) = \frac{a+b}{ab} \frac{1}{\binom{a+b}{a}}$.

Proof: Using integration by parts, one can show that

$$B(a, b) = \frac{a-1}{b} B(a-1, b+1)$$

and a simple calculation shows that

$$B(1, b) = \frac{1}{b}.$$

Let

$$H(a, b) = \frac{a+b}{ab} \frac{1}{\binom{a+b}{a}} = \frac{(a-1)!(b-1)!}{(a+b-1)!}$$

Then it's easy to check that H satisfies the same recurrences as $B(a, b)$, and that $H(1, b) = 1/b$. So the two functions agree by induction.

Using this Proposition, we see that

$$P(p|h) = \frac{p^h(1-p)^{N-h}}{B(h+1, N-h+1)}$$

and

$$E[p] = \frac{\int_{p=0}^1 p^{h+1}(1-p)^{N-h} dp}{B(h+1, N-h+1)} = \frac{B(h+2, N-h+1)}{B(h+1, N-h+1)}.$$

Sorting through this using the formula for $B(a, b)$ we obtain

$$E[p] = \frac{h+1}{N+2}.$$

So if we obtained 55 heads out of 100 flips, the maximum *a posteriori* estimate for p is .55, while the posterior mean is $56/102 = .549$ – just a bit less.

Now suppose that we had some reason to believe that our coin was fair. Then we can choose a prior probability distribution that expresses this. For example, we can choose

$$P(p) = \frac{1}{B(5, 5)} p^4(1-p)^4.$$

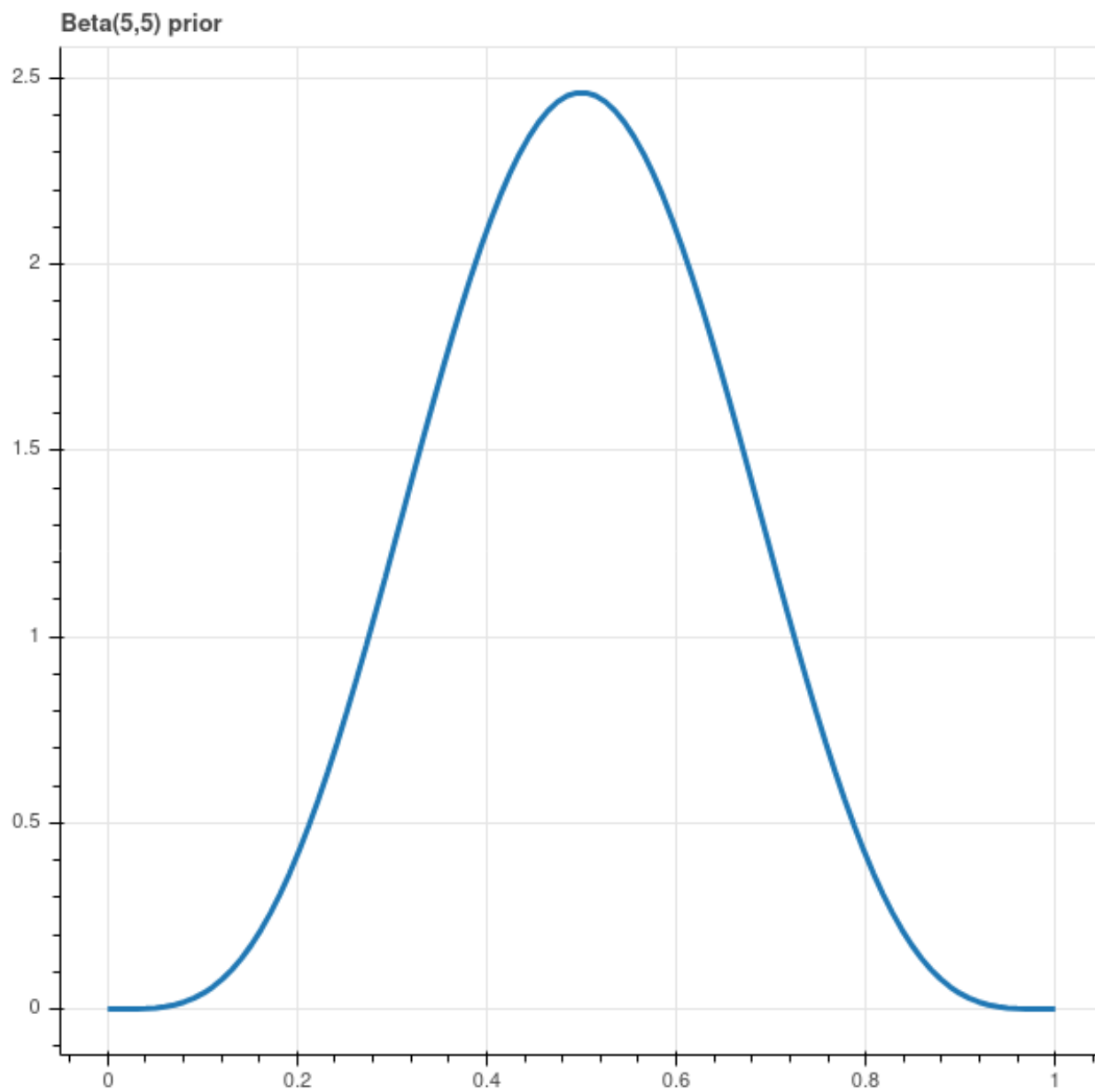


Figure 4.4: Beta(5,5) Prior

4 Bayesian Inference

Here we use the Beta function to guarantee that $\int_0^1 P(p)dp = 1$. We show this prior distribution in Figure 4.4.

If we redo our Bayes theorem calculation, we find that our posterior distribution is

$$P(p|h) \propto p^{h+4}(1-p)^{N-h+4}$$

and relying again on the Beta function for normalization we have

$$P(p|h) = \frac{1}{B(h+5, N-h+5)} p^{h+4}(1-p)^{N-h+4}$$

Here the maximum *a posteriori* estimate for p is $(h+4)/(N+8)$ while our posterior mean is

$$\frac{B(h+6, N-h+5)}{B(h+5, N-h+5)} = \frac{h+5}{N+10}.$$

In the situation of 55 heads out of 100, the maximum *a posteriori* estimate is .546 and the posterior mean is .545. These numbers have been pulled just a bit towards .5 because our prior knowledge makes us a little bit biased towards $p = .5$.

4.1.3 Bayesian Regression (or Ridge Regression)

In this chapter we return to our discussion of linear regression and introduce some Bayesian ideas. The combination will lead us to the notion of “ridge” regression, which is a type of linear regression that includes a prior distribution on the coefficients that indicates our preference for smaller rather than larger coefficients. Introduction of this prior leads to a form of linear regression that is more resilient in situations where the independent variables are less independent than we would hope.

Before introducing these Bayesian ideas, let us recall from Section 3.6.3 that ordinary least squares yields the parameters that give the “most likely” set of parameters for a model of the form

$$Y = XM + \epsilon$$

where the error ϵ is normally distributed with mean 0 and variance σ^2 , and the mean squared error becomes the maximum likelihood estimate of the variance σ^2 .

To put this into a Bayesian perspective, we notice that the linear regression model views $Y - XM$ as normally distributed given M . That is, we see the probability $P(Y - XM|M)$ as normal with variance σ^2 .

Then we introduce a prior distribution on the coefficients M , assuming that they, too, are normally distributed around zero with variance τ^2 . This means that *ab initio* we think that the coefficients are likely to be small.

From Bayes Theorem, we then have

$$P(M|Y, X) = \frac{P(Y, X|M)P(M)}{P(Y, X)}$$

and in distribution terms we have

$$P(M|Y, X) = Ae^{-\frac{\|Y-XM\|^2}{2\sigma^2}} e^{-\frac{\|M\|^2}{2\tau^2}}$$

where A is a normalizing constant.

The first thing to note from this expression is that the posterior distribution for the M parameters for regression is itself a normal distribution.

The maximum *a posteriori* estimate M_r for the parameters M occurs when $P(M|Y, X)$ is maximum, which we find by taking partial derivatives and setting them to zero. Using arguments similar to those we used for ordinary linear regression, we obtain the equation

$$(X^\top Y - (X^\top X)M_r)/\sigma^2 - M_r/\tau^2 = 0$$

or

$$(X^\top X + sI)M_r = X^\top Y \quad (4.1)$$

where $s = \sigma^2/\tau^2$.

Therefore the ridge coefficients are given by the equation

$$M_r = (X^\top X + sI)^{-1}X^\top Y \quad (4.2)$$

4.1.3.1 Practical aspects of ridge regression

Using ridge regression leads to a solution to the least squares problem in which the regression coefficients are biased towards being smaller. Beyond this, there are a number of implications of the technique which affect its use in practice.

First, we can put the Bayesian derivation of the ridge regression formulae in the background and focus our attention on Equation 4.2. We can treat the parameter s (which must be non-negative) as “adjustable”.

One important consideration when using ridge regression is that Equation 4.2 is not invariant if we scale X and Y by a constant. This is different from “plain” regression where we consider the equation $Y = XM$. In that case, rescaling X and Y by the same factor leaves the coefficients M alone. For this reason, ridge regression is typically used on centered, standardized coordinates. In other words, we replace each feature x_i by $(x_i - \mu_i)/\sigma_i$ where μ_i and σ_i are the sample mean and standard deviation of the i^{th} feature, and we replace our response variables y_i similarly by $(y_i - \mu)/\sigma$ where μ and σ are the mean and standard deviation of the y -values. Then we find M using Equation 4.2, perhaps experimenting with different values of s , using our centered and standardized variables.

To emphasize that we are using centered coordinates, we write X_0 for our data matrix instead of X . Recall that the matrix $X_0^\top X_0$ that enters into Equation 4.1 is ND_0 where D_0 is the covariance matrix. Therefore in ridge regression we have replaced ND_0 by $ND_0 + sI$. Since D_0 is a real symmetric matrix, as we’ve seen in Chapter 2 it is diagonalizable so that AD_0A^{-1} is diagonal for an orthogonal matrix A and has eigenvalues $\lambda_1 \geq \dots \geq \lambda_k$ which are the variances of the data along the principal directions.

One effect of using ridge regression is that the eigenvalues of $ND_0 + sI$ are always at least $s > 0$, so the use of ridge regression avoids the possibility that D_0 might not be invertible. In fact, a bit more is true. Numerical analysis tells us that when considering the problem of computing the inverse of a matrix, we should look at its *condition number*, which is the ratio λ_1/λ_k of the largest to the smallest eigenvalue.

If the condition number of a matrix is large, then results from numerical analysis show that it is *almost singular* and its inverse becomes very sensitive to small changes in the entries of the matrix. However, the eigenvalues of $ND_0 + sI$ are $N\lambda_i + s$ and so the condition number becomes $(N\lambda_1 + s)/(N\lambda_k + s)$. For larger values of λ , this condition number shrinks, and so the inverse of the matrix $ND_0 + sI$ becomes better behaved than ND_0 . In this way, ridge regression helps to improve the numerical stability of the linear regression algorithm.

A second way to look at Ridge regression is to go back to the discussion of the singular value decomposition of the matrix X_0 in section Section 2.4.2. There we showed that the SVD of X_0 yields an expression

$$X_0 = U\tilde{\Lambda}P^\top$$

where U and P are orthogonal matrices and Λ is an $N \times k$ matrix whose upper block is diagonal with eigenvalues $\sqrt{N\lambda_i}$. The rows of U gave us an orthonormal basis that allowed us to write the predicted vector $\hat{Y}$ as a projection:

$$\hat{Y} = \sum_{j=1}^k (u_j \cdot Y) u_j^\top.$$

If we repeat this calculation, but using the ridge regression formula, we obtain

$$\hat{Y}_r = X_0 M_r = U\tilde{\Lambda}P^\top (P\tilde{\Lambda}^\top U^\top U\tilde{\Lambda}P^\top + sI)^{-1} P\tilde{\Lambda}^\top U^\top Y.$$

Since P is orthogonal, $P^\top = P^{-1}$, so

$$P^\top (P\tilde{\Lambda}^2 P^\top + sI)^{-1} P = P^{-1} (P(\Lambda + sI)P^{-1})^{-1} P = (\Lambda + sI)^{-1}$$

and $\Lambda + sI$ is a $k \times k$ diagonal matrix with entries $N\lambda_i + s$.

Putting the pieces together we see that

$$\hat{Y}_r = U\tilde{\Lambda}(\Lambda + sI)^{-1}\tilde{\Lambda}U^\top Y.$$

In the language of orthogonal projection, this means that

$$\hat{Y}_r = \sum_{j=1}^k \frac{N\lambda_j}{N\lambda_j + s} (u_j \cdot Y) u_j^\top.$$

In other words, the predicted value computed by ridge regression is obtained by projecting Y into the space spanned by the feature vectors, but weighting the different principal components by $N\lambda_i/(N\lambda_i + s)$. With this weighting, the principal components with smaller variances are weighted less than those with larger variances. For this reason, ridge regression is sometimes called a *shrinkage* method.

5 The Naive Bayes classification method

5.1 Introduction

In our discussion of Bayes Theorem, we looked at a situation in which we had a population consisting of people infected with COVID-19 and people not infected, and we had a test that we could apply to determine which class an individual belonged to. Because our test was not 100 percent reliable, a positive test result didn't guarantee that a person was infected, and we used Bayes Theorem to evaluate how to interpret the positive test result. More specifically, our information about the test performance gave us the conditional probabilities of positive and negative test results given infection status – so for example we were given $P(+|\text{infected})$, the chance of getting a positive test assuming the person is infected – and we used Bayes Theorem to determine $P(\text{infected}|+)$, the chance that a person was infected given a positive test result.

The Naive Bayes classification method is a generalization of this idea. We have data that belongs to one of two classes, and based on the results of a series of tests, we wish to decide which class a particular data point belongs to. For one example, we are given a collection of product reviews from a website and we wish to classify those reviews as either “positive” or “negative.” This type of problem is called “sentiment analysis.” For another related example, we have a collection of emails or text messages and we wish to label those that are likely “spam” emails. In both of these examples, the “test” that we will apply is to look for the appearance or absence of certain key words that make the text more or less likely to belong to a certain class. For example, we might find that a movie review that contains the word “great” is more likely to be positive than negative, while a review that contains the word “boring” is more likely to be negative.

The reason for the word “naive” in the name of this method is that we will derive our probabilities from empirical data rather than from any deeper theory. For example, to find the probability that a negative movie review contains the word “boring”, we will look at a bunch of reviews that our experts have said are negative, and compute the proportion of those that contain the word boring. Indeed, to develop our family of tests, we will rely on a training set of already classified data from which we can determine estimates of probabilities that we need.

5.2 An example dataset

To illustrate the Naive Bayes algorithm, we will work with the “Sentiment Labelled Sentences Data Set” ([3]). This dataset contains 3 files, each containing 1000 documents labelled 0 or 1 for “negative” or “positive” sentiment. There are 500 of each type of document in each file. One file contains reviews of products from amazon.com; one contains yelp restaurant reviews, and one contains movie reviews from imdb.com.

5 The Naive Bayes classification method

Let's focus on the amazon reviews data. Here are some samples:

```
So there is no way for me to plug it in here
  in the US unless I go by a converter.    0
Good case, Excellent value. 1
Great for the jawbone. 1
Tied to charger for conversations lasting more than
  45 minutes.MAJOR PROBLEMS!! 0
The mic is great. 1
I have to jiggle the plug to get it to line up right to
  get decent volume. 0
If you have several dozen or several hundred contacts, then
  imagine the fun of sending each of them one by one. 0
If you are Razr owner...you must have this! 1
Needless to say, I wasted my money. 0
What a waste of money and time!. 0
```

As you can see, each line consists of a product review followed by a 0 or 1; in this file the review is separated from the label by a tab character.

5.3 Bernoulli tests

We will describe the “Bernoulli” version of a Naive Bayes classifier for this data. The building block of this method is a test based on a single word. For example, let's consider the word **great** among all of our amazon reviews. It turns out that **great** occurs 5 times in negative reviews and 92 times in positive reviews among our 1000 examples. So it seems that seeing the word **great** in a review makes it more likely to be positive. The appearances of **great** are summarized in Table 5.1 . We write $\sim\mathbf{great}$ for the case where **great** does *not* appear.

Table 5.1: Occurrences of **great** by type of review

	+	-	total
great	92	5	97
$\sim\mathbf{great}$	408	495	903
total	500	500	1000

In this data, positive and negative reviews are equally likely so $P(+) = P(-) = .5$. From this table, and Bayes Theorem, we obtain the empirical probabilities (or “naive” probabilities).

$$P(\mathbf{great}|+) = \frac{92}{500} = .184$$

and

$$P(\mathbf{great}) = \frac{97}{1000} = .097$$

Therefore

$$P(+|\mathbf{great}) = \frac{.184}{.097}(.5) = 0.948.$$

In other words, *if* you see the word **great** in a review, there's a 95% chance that the review is positive.

What if you *do not* see the word **great**? A similar calculation from the table yields

$$P(+|\sim \mathbf{great}) = \frac{408}{903} = .452$$

In other words, *not* seeing the word **great** gives a little evidence that the review is negative (there's a 55% chance it's negative) but it's not that conclusive.

The word **waste** is associated with negative reviews. Its statistics are summarized in Table 5.2 .

Table 5.2: Occurrences of **waste** by type of review

	+	-	total
waste	0	14	14
$\sim$ waste	500	486	986
total	500	500	1000

Based on this data, the “naive” probabilities we are interested in are:

$$\begin{aligned} P(+|\mathbf{waste}) &= 0 \\ P(+|\sim \mathbf{waste}) &= .51 \end{aligned}$$

In other words, if you see **waste** you definitely have a negative review, but if you don't, you're only slightly more likely to have a positive one.

What about combining these two tests? Or using even more words? We could analyze our data to count cases in which both **great** and **waste** occur, in which only one occurs, or in which neither occurs, within the two different categories of reviews, and then use those counts to estimate empirical probabilities of the joint events. But while this might be feasible with two words, if we want to use many words, the number of combinations quickly becomes huge. So instead, we make a basic, and probably false, assumption, but one that makes a simple analysis possible.

Assumption: We assume that the presence or absence of the words **great** and **waste** in a particular review (positive or negative) are independent events. More generally, given a collection of words $w_1, \dots, w_k$, we assume that their occurrences in a given review are independent events.

5 The Naive Bayes classification method

Independence means that we have

$$\begin{aligned} P(\mathbf{great}, \mathbf{waste}|\pm) &= P(\mathbf{great}|\pm)P(\mathbf{waste}|\pm) \\ P(\mathbf{great}, \sim \mathbf{waste}|\pm) &= P(\mathbf{great}|\pm)P(\sim \mathbf{waste}|\pm) \\ &\vdots \end{aligned}$$

So for example, if a document contains the word **great** and does *not* contain the word **waste**, then the probability of it being a positive review is:

$$P(+|\mathbf{great}, \sim \mathbf{waste}) = \frac{P(\mathbf{great}|+)P(\sim \mathbf{waste}|+)P(+)}{P(\mathbf{great}, \sim \mathbf{waste})}$$

while the probability of it being a negative review is

$$P(-|\mathbf{great}, \sim \mathbf{waste}) = \frac{P(\mathbf{great}|-)P(\sim \mathbf{waste}|-)P(-)}{P(\mathbf{great}, \sim \mathbf{waste})}$$

Rather than compute these probabilities (which involves working out the denominators), let's just compare them. Since they have the same denominators, we just need to compare numerators, which we call L for likelihood: Using the data from Table 5.1 and Table 5.2, we obtain:

$$L(+|\mathbf{great}, \sim \mathbf{waste}) = (.184)(1)(.5) = .092$$

and

$$L(-|\mathbf{great}, \sim \mathbf{waste}) = (.01)(.972)(.5) = .00486$$

so our data suggests strongly that this is a positive review.

5.4 Feature vectors

To generalize this, suppose that we have extracted keywords $w_1, \dots, w_k$ from our data and we have computed the empirical values $P(w_i|+)$ and $P(w_i|-)$ by counting the fraction of positive and negative reviews that contain the word w_i :

$$P(w_i|\pm) = \frac{\text{number of } \pm \text{ reviews that mention } w_i}{\text{number of } \pm \text{ reviews total}}$$

Notice that we only count *reviews*, not *occurrences*, so that if a word occurs multiple times in a review it only contributes 1 to the count. That's why this is called the *Bernoulli* Naive Bayes – we are thinking of each keyword as yielding a yes/no test on each review.

Given a review, we look to see whether each of our k keywords appears or does not. We encode this information as a vector of length k containing 0's and 1's indicating the absence or presence of the corresponding keyword. Let's call this vector the *feature vector* for the review.

For example, if our keywords are $w_1 = \mathbf{waste}$, $w_2 = \mathbf{great}$, and $w_3 = \mathbf{useless}$, and our review says

This phone is useless, useless, useless! What a waste!

then the associated feature vector is $f = (1, 0, 1)$.

For the purpose of classifying our reviews, we are going to forget entirely about the text of our reviews and work only with the feature vectors. From an abstract perspective, then, by choosing our k keywords, our “training set” of N labelled reviews can be replaced by an $N \times k$ matrix $X = (x_{ij})$ with entries 0 or 1, where $x_{ij} = 1$ if and only if the j^{th} keyword appears in the i^{th} review.

The labels of 0 or 1 for unfavorable or favorable reviews can also be packaged up into a $N \times 1$ vector Y that serves as our “target” variable.

Setting things up this way lets us express the computations of our probabilities $P(w_i|\pm)$ in vector form. In fact, $Y^\top X$ is the sum of the rows of X corresponding to positive reviews, and therefore, letting $N_\pm$ denote the number of $\pm$ reviews,

$$P_+ = \frac{1}{N_+} Y^\top X = \begin{bmatrix} P(w_1|+) & P(w_2|+) & \cdots & P(w_k|+) \end{bmatrix}.$$

Similarly, since Y and X have zero and one entries only, if we write $1 - Y$ for the matrix obtained by replacing every entry z by $1 - z$, we have:

$$P_- = \frac{1}{N_-} (1 - Y)^\top X = \begin{bmatrix} P(w_1|-) & P(w_2|-) & \cdots & P(w_k|-) \end{bmatrix}.$$

Note that the number of positive reviews is $N_+ = Y^\top Y$ and the number of negative ones is $N_- = N - N_+$. Since $P(+)$ is the fraction of positive reviews among all reviews, we can compute it as $P(+) = \frac{1}{N} Y^\top Y$, and $P(-) = 1 - P(+)$.

5.5 Likelihood

If a review has an associated feature vector $f = (f_1, \dots, f_k)$, then by independence the probability of that feature vector occurring within one of the $\pm$ classes is

$$P(f|\pm) = \prod_{i:f_i=1} P(w_i|\pm) \prod_{i:f_i=0} (1 - P(w_i|\pm))$$

which we can also write

$$P(f|\pm) = \prod_{i=1}^k P(w_i|\pm)^{f_i} (1 - P(w_i|\pm))^{(1-f_i)}. \quad (5.1)$$

These products aren’t practical to work with – they are often the product of many, many small numbers and are therefore really tiny. Therefore it’s much more practical to work with their logarithms.

$$\log P(f|\pm) = \sum_{i=1}^k f_i \log P(w_i|\pm) + (1 - f_i) \log(1 - P(w_i|\pm)) \quad (5.2)$$

5 The Naive Bayes classification method

If we have a group of N reviews organized in a matrix X , where each row is the feature vector associated to the corresponding review, then we can compute all of this at once. We'll write $\log P(X|\pm)$ as the column vector whose i^{th} entry is $\log P(f_i|\pm)$:

Here there are two different vector objects: $P_{\pm}$ is a $1 \times k$ vector of per-word probabilities ($P(w_j|\pm)$), while $P(X|\pm)$ is an $N \times 1$ vector of per-review likelihoods ($P(f_i|\pm)$). The matrix formula below uses $P_{\pm}$ as input and outputs the $N \times 1$ vector $P(X|\pm)$.

$$\log P(X|\pm) = X(\log P_{\pm})^{\top} + (1 - X)(\log(1 - P_{\pm}))^{\top}. \quad (5.3)$$

By Bayes Theorem, we can express the chance that our review with feature vector f is positive or negative by the formula:

$$\log P(\pm|f) = \log P(f|\pm) + \log P(\pm) - \log P(f)$$

where

$$P(\pm) = \frac{\text{the number of } \pm \text{ reviews}}{\text{total number of reviews}}$$

and $P(f)$ is the fraction of reviews with the given feature vector. (Note: in practice, some of these probabilities will be zero, and so the log will not be defined. A common practical approach to dealing with this is to introduce a “fake document” into both classes in which every vocabulary word appears – this guarantees that the frequency matrix will have no zeros in it).

A natural classification rule would be to say that a review is positive if $\log P(+|f) > \log P(-|f)$, and negative otherwise. In applying this, we can avoid computing $P(f)$ by just comparing $\log P(f|+) + \log P(+)$ and $\log P(f|-) + \log P(-)$ computed using Equation 5.2. Then we say:

- a review is positive if $\log P(f|+) + \log P(+)$ and negative otherwise.

Again we can exploit the matrix structure to do this for a bunch of reviews at once. Using Equation 5.3 and the vectors $P_{\pm}$ we can compute column vectors corresponding to both sides of our decision inequality and subtract them. The positive entries indicate positive reviews, and the negative ones, negative reviews.

5.6 The Bag of Words

In our analysis above, we thought of the presence or absence of certain key words as a set of independent tests that provided evidence of whether our review was positive or negative. This approach is suited to short pieces of text, but what about longer documents? In that case, we might want to consider not just the presence or absence of words, but the frequency with which they appear. Multinomial Naive Bayes, based on the “bag of words” model, is a classification method similar to Bernoulli Naive Bayes but which takes term frequency into account.

Let's consider, as above, the problem of classifying documents into one of two classes. We assume that we have a set of keywords $w_1, \dots, w_k$. For each class $\pm$, we have a set of probabilities $P(w_i|\pm)$ with the property that

$$\sum_{i=1}^k P(w_i|\pm) = 1.$$

The “bag of words” model says that we construct a document of length N in, say, the $+$ class by independently drawing a word N times from the set $w_1, \dots, w_k$ with probabilities $P(w_i|+)$. The name “bag of words” comes from thinking of each class as having an associated bag containing the words $w_1, \dots, w_k$ with relative frequencies given by the probabilities, and generating a document by repeatedly drawing a word from the bag.

In the Multinomial Naive Bayes method, we estimate the probabilities $P(w_i|\pm)$ by counting the number of times each word occurs in a document of the given class:

$$P(w_i|\pm) = \frac{\text{number of times word } i \text{ occurs in } \pm \text{ documents}}{\text{total number of words in } \pm \text{ documents}}$$

This is the “naive” part of the algorithm. Package up these probabilities in vectors:

$$P_{\pm} = [P(w_1|\pm) \quad \dots \quad P(w_k|\pm)].$$

As in the Bernoulli case, we often add a fake document to each class where all of the words occur once, in order to avoid having zero frequencies when we take a logarithm later.

Now, given a document, we associate a feature vector $\mathbf{f}$ whose i^{th} entry is the frequency with which word i appears in that document. The probability of obtaining a particular document with feature vector $\mathbf{f} = (f_1, \dots, f_k)$ from the bag of words associated with class $\pm$ is given by the “multinomial” distribution:

$$P(\mathbf{f}|\pm) = \frac{N!}{f_1! f_2! \dots f_k!} \prod_{i=1}^k P(w_i|\pm)^{f_i}$$

which generalizes the binomial distribution to multiple choices. The constant will prove irrelevant, so let's call the interesting part $L_{\pm}$:

$$L(\mathbf{f}|\pm) = \prod_{i=1}^k P(w_i|\pm)^{f_i}$$

From Bayes Theorem, we have

$$P(\pm|\mathbf{f}) = \frac{P(\mathbf{f}|\pm)P(\pm)}{P(\mathbf{f})}$$

where $P(\pm)$ is estimated by the fraction of documents in each class.

We classify our document by considering $P(\pm|\mathbf{f})$ and concluding:

- a document with feature vector $\mathbf{f}$ is in class $+$ if $\log P(+|\mathbf{f}) > \log P(-|\mathbf{f})$.

5 The Naive Bayes classification method

In this comparison, both the constant (the multinomial coefficient) and the denominator cancel out, so we only need to compare $\log L(\mathbf{f}|+) + \log P(+)$ with $\log L(\mathbf{f}|-) + \log P(-)$. We have

$$\log L(\mathbf{f}|\pm) = \sum_{i=1}^k f_i \log P(w_i|\pm)$$

or, in vector form,

$$\log L(\mathbf{f}|\pm) = \mathbf{f} \log P_{\pm}^{\top}$$

Therefore, just as in the Bernoulli case, we can package up our documents as an $N \times k$ data matrix X , where position ij gives the number of times word j occurs in document i . Then we can compute the vector

$$\hat{Y} = X \log P_{+}^{\top} + \log P(+) - X \log P_{-}^{\top} - \log P(-)$$

and assign those documents where $\hat{Y} > 0$ to the $+$ class and the rest to the $-$ class.

5.7 Other applications

We developed the Naive Bayes method for sentiment analysis, but once we chose a set of keywords, our training data was reduced to an $N \times k$ matrix X of 0/1 entries, together with an $N \times 1$ target column vector Y . Then our classification problem is to decide whether a given vector of k entries, all 0 or 1, is more likely to carry a 0 or 1 label. All of the parameters we needed for Naive Bayes – the various probabilities – can be extracted from the matrix X .

For example, suppose we have a collection of images represented as black-and-white pixels in a grid that belong to one of two classes. We might have 28×28 bitmaps of handwritten zeros and ones that are labelled, and we wish to construct a classifier that can decide whether a new 28×28 bitmap is a zero or one. An example of such a bitmap is given in Figure 5.1. We can view each 28×28 bitmap as a vector of length 784 with 0/1 entries and apply the same approach outlined above. However, there are other methods that are more commonly used for this problem, such as logistic regression and neural networks.

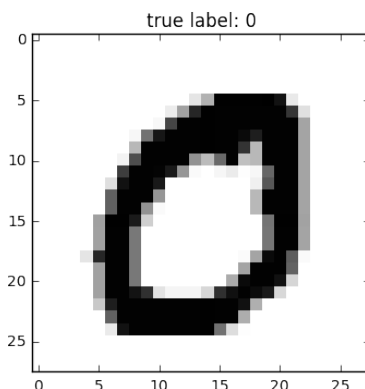


Figure 5.1: Handwritten 0

6 Gradient Descent

6.1 Introduction

A common mathematical theme throughout machine learning is the problem of finding the minimum or maximum value of a function. For example, in linear regression, we find the “best-fitting” linear function by identifying the parameters that minimize the mean squared error. In principal component analysis, we try to identify the scores which have the greatest variation for the given set of data, and for this we needed to maximize a function using Lagrange multipliers. In later lectures, we will see many more examples where we construct the “best” function for a particular task by minimizing some kind of error between our constructed function and the true observed values.

In our discussion of PCA and linear regression, we were able to give analytic formulae for the solution to our problems. These solutions involved (in the case of linear regression) inverting a matrix, and in the case of PCA, finding eigenvalues and eigenvectors. These are elegant mathematical results, but at that time we left open the question of how to actually *compute* these quantities of interest in an efficient way. In this section, we will discuss the technique known as gradient descent, which is perhaps the simplest approach to minimizing a function using calculus, and which is at the foundation of many practical machine learning algorithms.

6.2 The Key Idea

Suppose that we have a function $f(x_1, \dots, x_k)$ and we wish to find its minimum value. In Calculus classes, we are taught to take the derivatives of the function and set them equal to zero, but for anything other than the simplest functions this problem is not solvable in practice. In real life, we use iterative methods to find the minimum of the function f .

The main tool in this approach is the gradient of a function. Recall from multivariable calculus that, given a function $f(x_1, \dots, x_k)$, the *gradient* ∇f of f is the vector made up of the partial derivatives of f :

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_k} \end{bmatrix} \quad (6.1)$$

The gradient ∇f is an example of a *vector field*. It is a vector whose entries are functions. If we evaluate those functions at a point x , we obtain a vector $(\nabla f)(x)$.

We can think of a vector field as a way to attach a vector $(\nabla f)(x)$ to each point $x \in \mathbf{R}^k$.

Proposition: Let $f(x_1, \dots, x_k)$ be a function and let ∇f be its gradient. Then at each point x in $\mathbf{R}^k$, the gradient $(\nabla f)(x)$ is a vector that points in the direction in which f is increasing most rapidly from x and $(-\nabla f)(x)$ points in the direction in which it decreases most rapidly. If $(\nabla f)(x) = 0$, then x is a critical point of f .

This fact arises from thinking about the *directional derivative* of a function.

The directional derivative $D_v f$ measures the rate of change of f as one moves with velocity vector v from a chosen point $a \in \mathbf{R}^k$ and it is defined as

$$D_v f(a) = \frac{d}{dt} f(a + tv)|_{t=0}$$

If $a = (a_1, \dots, a_k)$ and $v = (v_1, \dots, v_k)$, then

$$f(a + tv) = f(a_1 + tv_1, \dots, a_k + tv_k).$$

Since

$$\frac{d}{dt}(a_i + tv_i) = v_i$$

we can compute from the chain rule that

$$D_v f(a) = \sum_{i=1}^k \frac{\partial f}{\partial x_i}(a) v_i = (\nabla f)(a) \cdot v$$

where ∇f is the gradient of f as in Equation 6.1.

If we travel from a point a with velocity vector v , the rate of change of f along our path will depend on how fast we travel as well as the direction we choose. In other words, $D_v f$ depends on the magnitude of v , as well as its direction. To remove this dependence on the magnitude of v , we scale v to be a unit vector.

Now

$$(\nabla f)(a) \cdot v = \|(\nabla f)(a)\| \|v\| \cos \theta = \|(\nabla f)(a)\| \cos \theta$$

where θ is the angle between v and $(\nabla f)(a)$. Clearly this dot product is maximized when v is parallel to $(\nabla f)(a)$ so $\cos \theta = 1$. If v is opposite to $(\nabla f)(a)$, then $\cos \theta = -1$ and the dot product is minimized.

6.3 The Algorithm

To exploit the fact that the gradient points in the direction of most rapid increase of our function f , we adopt the following strategy. Starting from a point x , compute the gradient $(\nabla f)(x)$ of f . Take a small step in the direction of the gradient – that should increase the value of f . Then do it again, and again; each time, you move in the direction of increasing f , but at some point the gradient becomes very small and you stop moving much. At that moment, you stop. You have reached the highest value of f in the immediate vicinity. This is called “gradient ascent” to a local maximum.

If we want to *minimize*, not maximize, our function, then we want to move *opposite* to the gradient in small steps. This is the more common formulation.

6.3.1 Gradient Descent Algorithm

Given a function $f : \mathbb{R}^k \rightarrow \mathbb{R}$, to find a point where it is minimized, choose:

- a starting point $c^{(0)}$,
- a small constant ν (called the *learning rate*)
- and a small constant ϵ (the *tolerance*).

Iteratively compute

$$c^{(n+1)} = c^{(n)} - \nu(\nabla f)(c^{(n)})$$

until $\|c^{(n+1)} - c^{(n)}\| < \epsilon$.

Then $c^{(n+1)}$ is an (approximate) critical point of f .

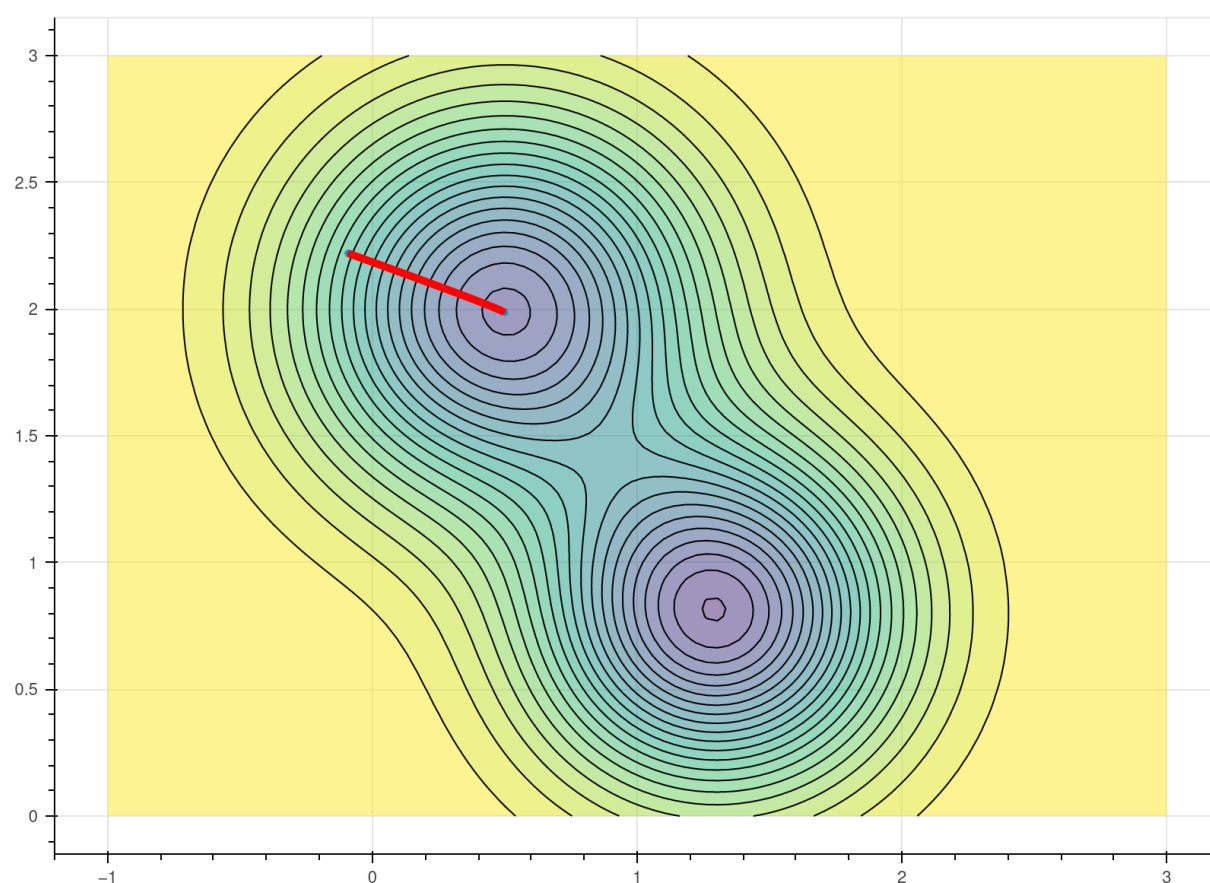


Figure 6.1: Gradient Descent Illustrated

The behavior of gradient descent, at least when all goes well, is illustrated in Figure 6.1 for the function

$$f(x, y) = 1.3e^{-2.5((x-1.3)^2+(y-0.8)^2)} - 1.2e^{-2((x-1.8)^2+(y-1.3)^2)}.$$

Figure 6.1 is a contour plot, with the black lines at constant height and the colors indicating the height of the function. This function has two “pits” or “wells” indicated by the darker, “cooler”

6 Gradient Descent

colored regions. The red line shows the gradient descent algorithm heading “downhill” towards a nearby local minimum.

To get a little more perspective on gradient descent, consider the one-dimensional case, with

$$f(x) = 3x^4 + 4x^3 - 12x^2 + 5.$$

This is a quartic polynomial whose graph has two local minima and a local maximum, depicted in Figure 6.2.

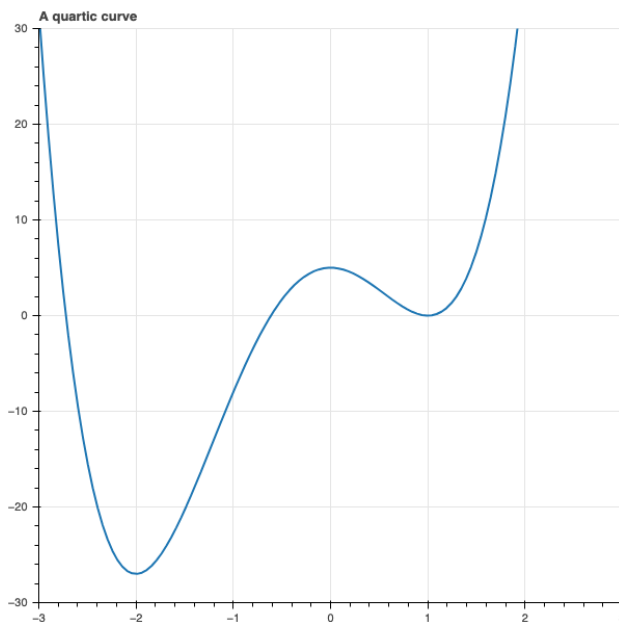


Figure 6.2: A quartic polynomial

In this case the gradient is just the derivative

$$f'(x) = 12x^3 + 12x^2 - 24x$$

and the iteration is

$$c^{(n+1)} = c^{(n)} - 12\nu((c^{(n)})^3 + (c^{(n)})^2 - 2c^{(n)}).$$

From this simple example we can see the power and also the pitfalls of this method. Suppose we choose $c^{(0)} = .5$, $\nu = .01$, and do 30 iterations of the main loop in our algorithm. The result is shown in Figure 6.3 .

As we hope, the red dots quickly descend to the bottom of the “valley” at the point $x = 1$. However, this valley is only a *local minimum* of the function; the true minimum is at $x = -2$. Gradient descent can’t see that far away point and so we don’t find the true minimum of the function. One way to handle this is to *run gradient descent multiple times with random starting coordinates* and then look for the minimum value it finds among all of these tries.

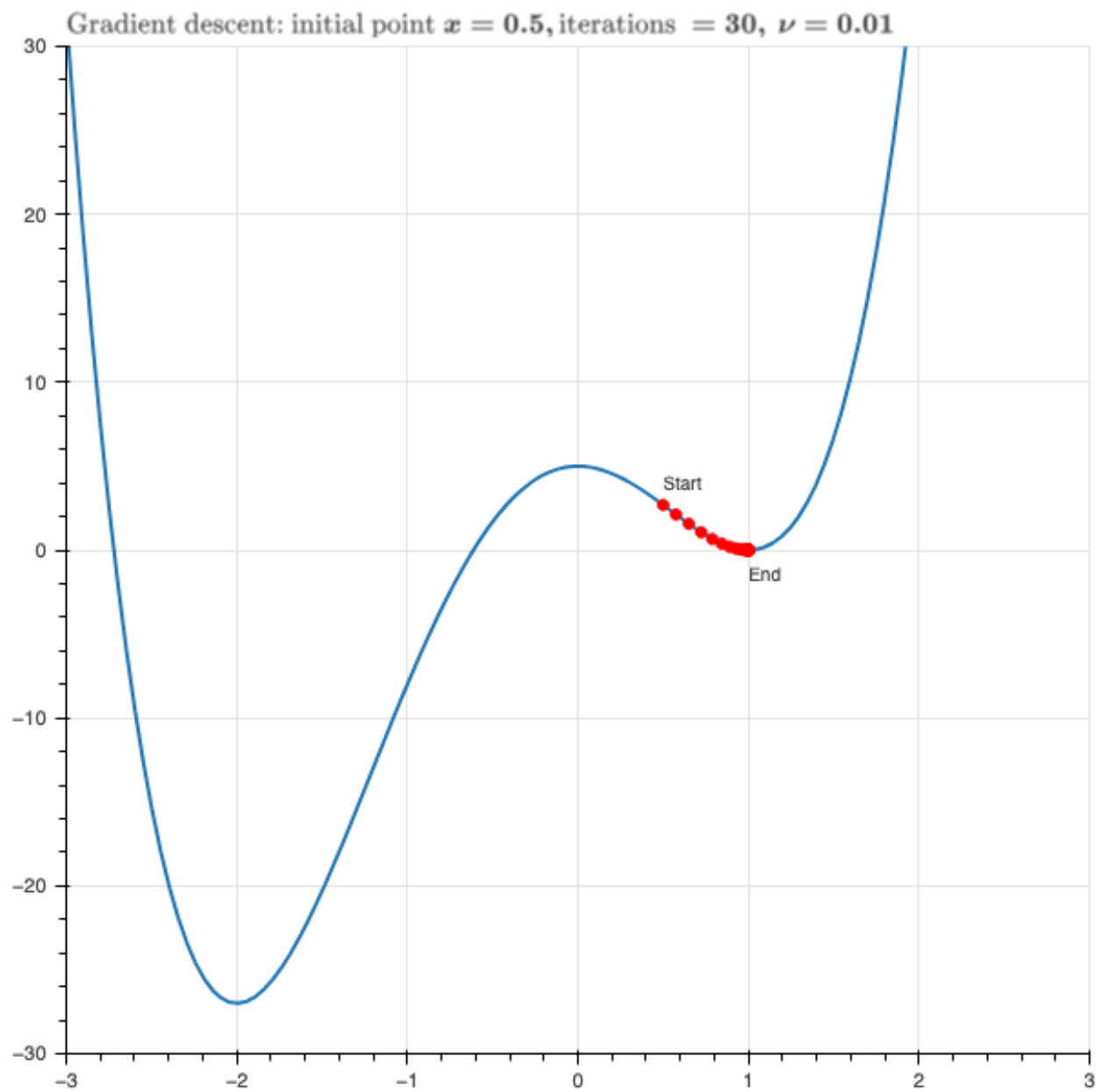


Figure 6.3: Gradient descent to a local minimum

6.4 Linear Regression via Gradient Descent

In our discussion of Linear Regression in Chapter 1, we used Calculus to find the values of the parameters that minimized the squared difference between the predicted and desired values. If our features were stored in the matrix X (with an additional column of 1's) and our target values in the vector Y , then we showed that the optimal parameters M were given by

$$M = D^{-1}X^{\top}Y$$

where $D = X^{\top}X$. This set of parameters minimizes the squared error:

$$L = \|Y - XM\|^2.$$

See Equation 1.7 and Equation 1.8.

One problem with this approach is the need to invert the matrix D , which is a serious computation in its own right. We can avoid relying on matrix inversion by approaching this problem numerically via gradient descent, using the computation of the gradient in Equation 1.5:

$$\nabla E = \begin{bmatrix} \frac{\partial}{\partial M_1} L \\ \frac{\partial}{\partial M_2} L \\ \vdots \\ \frac{\partial}{\partial M_{k+1}} L \end{bmatrix} = -2X^{\top}Y + 2X^{\top}XM$$

The gradient descent algorithm looks like this.

6.4.1 Gradient Descent Algorithm for Linear Regression

Set $M^{(0)}$ to a random vector in $\mathbb{R}^{k+1}$ for an initial guess and choose a learning rate parameter ν . Compute $A = X^{\top}Y$ (an element of $\mathbb{R}^{k+1}$) and $D = X^{\top}X$ (a $(k+1) \times (k+1)$ matrix).

Iteratively compute

$$M^{(j+1)} = M^{(j)} - \nu(-2A + 2DM^{(j)})$$

until a stopping condition is met. For example, stop if:

- the squared error $\|Y - XM^{(j)}\|^2$ is sufficiently small, or
- the squared error decreases by less than some tolerance on each iteration, or
- the entries of $M^{(j)}$ change by less than some tolerance.

Notice that this algorithm does not need computation of D^{-1} .

6.5 Stochastic Gradient Descent

Using the numerical approach to linear regression avoids inverting the matrix $D = X^\top X$, but still leaves us the task of computing it. Typically X has many rows, and so this computation is time-intensive. We would like to avoid having to use *all* of the data for each iteration of our algorithm.

Stochastic gradient descent is a method for numerical optimization that does not require use of all the data on each iteration. Instead, it uses each data point sequentially. Let's look at this in the context of linear regression. Suppose we have an estimate M for the parameters. We take one data point x — a single row of the data matrix X — and the associated target value y . The error for this particular point is $(y - xM)^2$.

The gradient of the squared error (SE), viewed as a function of the weights M , at this particular point is

$$\nabla SE = -2x^\top(y - xM)$$

Notice that $y - xM$ is just a scalar, so this is a scalar multiple of the vector x .

Now we iterate over the data, adjusting the parameters M by this per-sample gradient. Each pass through the entire set is sometimes called an “epoch.”

6.5.1 Stochastic Gradient Descent for Linear Regression

Set $M^{(0)}$ to a random starting vector in $\mathbb{R}^{k+1}$ and choose a learning rate parameter ν .

For each sample (x_i, y_i) consisting of a row of the data matrix X and its corresponding target value, adjust the parameters by the gradient of the error associated with this point:

$$M^{(j+1)} = M^{(j)} - \nu(-2x_i^\top(y_i - x_i M^{(j)})) = M^{(j)} + 2\nu x_i^\top(y_i - x_i M^{(j)})$$

Here $x_i^\top$ and $M^{(j)}$ are column vectors and $y_i - x_i M^{(j)}$ is a scalar.

Run through the data set multiple times and track the error $(y_i - x_i M^{(j)})^2$ for each pair (x_i, y_i) . These errors will bounce around but trend overall downward. When the error becomes smaller than a threshold, or when the difference between successive errors becomes smaller than a threshold, stop.

Note: To minimize the bias introduced by the particular order in which you process the data, it's often worthwhile to shuffle the order in which you consider the points (x, y) in each epoch.

7 Logistic Regression

Suppose that we are trying to convince customers to buy our product by showing them advertising. Our experience teaches us that there is no deterministic relationship between how often a potential customer sees one of our ads and whether or not they purchase our product; nevertheless, it is the case that as they see more ads they become more likely to make a purchase. Logistic regression is a statistical model that captures this idea.

To formulate this situation mathematically, let's imagine that we are trying to model a random event that depends on a parameter. The random event might be a user deciding to make a purchase from a website, which, in our very simple model, depends on how many times the user saw an advertisement for the product in question. But we could imagine other situations where the chance of an event happening depends on a parameter. For example, we could imagine that a student's score on a certain test depends on how much studying they do, with the likelihood of passing the test increasing with the amount of studying.

To construct this model, we assume that the probability of a certain event p is related to some parameter x by the following relationship:

$$\log \frac{p}{1-p} = ax + b \quad (7.1)$$

where a and b are constants. The quantity $\frac{p}{1-p}$ is the “odds” of the event occurring. We often use this quantity colloquially; if the chance of our team winning a football game is 1 in 3, then we would say the odds of a win are 1-to-2, which we can interpret as meaning they are twice as likely to lose as to win. The quantity $\log \frac{p}{1-p}$ is, for obvious reasons, called the log-odds of the event. The logistic model in Equation 7.1 means that an increase by 1 in the parameter x increases the log-odds of the event happening by a .

The assumption in Equation 7.1 can be written

$$\frac{p}{1-p} = e^{ax+b}$$

and we interpret this as telling us that if the parameter x increases by 1, the odds of our event happening go up by a factor of e^a . So, to be even more concrete, if $a = \log 2$, then our logistic model would say that an increase of 1 in our parameter x doubles the odds of our event taking place.

In terms of the probability p , Equation 7.1 can be rewritten

$$p = \frac{1}{1 + e^{-ax-b}}$$

7 Logistic Regression

This proposed relationship between the probability p and the parameter x is called the *logistic model*. The function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

is called the *logistic function* and yields an S-shaped curve.

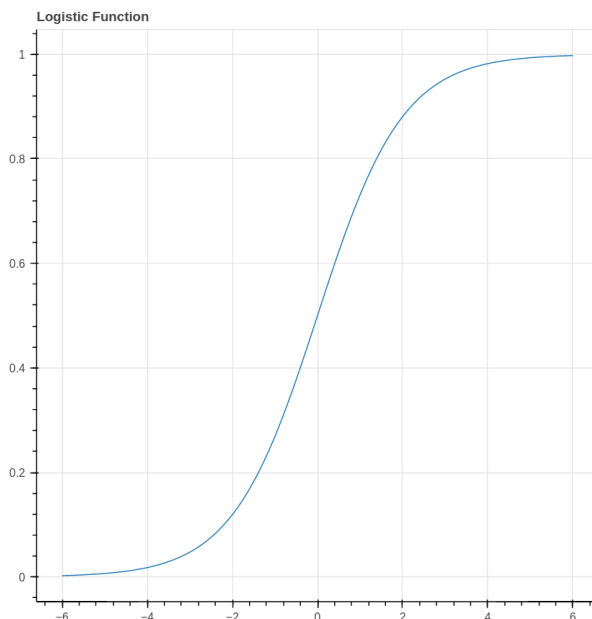


Figure 7.1: Logistic Curve

To put the logistic model in perspective, let's choose some explicit parameters and look at what data arising from such a model would look like. Imagine therefore that $a = \log 2$ and $b = 0$, so that the probability of the event we are interested in occurring is given by the formula

$$p(x) = \frac{1}{1 + e^{-(\log 2)x}} = \frac{1}{1 + (.5)^x}.$$

Our data consists of counts of how often our event happened for a range of values of x . To generate this data, we can pick x values from the set $\{-3, -2, -1, 0, 1, 2, 3\}$ yielding probabilities $\{.11, .20, .33, .50, .67, .80, .89\}$. Now our data consists, for each value of x , of the result of 100 independent Bernoulli trials with probability $p(x)$. For example, we might find that our event occurred $\{10, 18, 38, 50, 69, 78, 86\}$ times respectively for each of the x values. As you can see, the event occurs more frequently when x is large, although the number of occurrences is still random.

7.1 Likelihood and Logistic Regression

In applications, our goal is to choose the parameters of a logistic model to accurately predict the likelihood of the event under study occurring as a function of the measured parameter. Let's imagine that we collected the data that we generated above, without knowing that its source was a

logistic model. So Table 7.1 shows the number of times the event occurred, for each of the measured values of the x parameter.

Table 7.1: Sample Data

x	-3	-2	-1	0	1	2	3
Occurrences (out of 100)	10	18	38	50	69	78	86

Our objective now is to find a logistic model which best explains this data. Concretely, we need to estimate the coefficients a and b that yield

$$p(x) = \frac{1}{1 + e^{-ax-b}} \quad (7.2)$$

where the resulting probabilities best estimate the data. As we have seen, this notion of “best” can have different interpretations. For example, we could approach this from a Bayesian point of view, adopt a prior distribution on the parameters a and b , and use the data to obtain this prior and obtain a posterior distribution on a and b . For this first look at logistic regression, we will instead adopt a “maximum likelihood” notion of “best” and ask what is the most likely choice of a and b to yield this data.

To apply the maximum likelihood approach, we need to ask “for (fixed, but unknown) values of a and b , what is the likelihood that a logistic model with those parameters would yield the data we have collected?” Each column in Table 7.1 represents 100 Bernoulli trials with a fixed probability $p(x)$. So, for example, the chance q of obtaining 10 positive results with $x = -3$ is given by

$$q(-3) = Cp(-3)^{10}(1 - p(-3))^{90}$$

where C is a constant (it would be a binomial coefficient). Combining this for different values of x , we see that the likelihood of the data is the product

$$L(a, b) = C'p(-3)^{10}(1 - p(-3))^{90}p(-2)^{18}(1 - p(-2))^{82} \cdots p(3)^{86}(1 - p(3))^{14}$$

where C' is another constant. Each $p(x)$ is a function of the parameters a and b , so all together this is a function of those two parameters. Our goal is to maximize it.

One step that simplifies matters is to consider the logarithm of the likelihood:

$$\log L(a, b) = \sum_{i=1}^7 [c_i \log(p(x_i)) + (100 - c_i) \log(1 - p(x_i))] + C''$$

where C'' is yet another constant. Since our ultimate goal is to maximize this, the value of C'' is irrelevant and we can drop it.

7.1.1 Another point of view on logistic regression

In Table 7.1 we summarize the results of our experiments in groups by the value of the x parameter. We can think of the data somewhat differently, by instead considering each event separately, corresponding to a parameter value x and an outcome 0 or 1. From this point of view the data summarized in Table 7.1 would correspond to a vector with 700 rows. The first 100 rows (corresponding to the first column of the table) would have first entry -3 , the next 100 would have -2 , and so on. So our parameter values form a vector X . Meanwhile, the outcomes form a vector Y with entries 0 or 1.

More generally, imagine we are studying our advertising data and, for each potential customer, we record how many times they saw our ad. We create a vector X whose entries are these numbers. Then we create another vector Y , of the same length, whose entries are either 0 or 1 depending on whether or not the customer purchased our product.

One way to think about logistic regression in this setting is that we are trying to fit a function that, given the value x_i , tries to yield the corresponding value y_i . However, instead of finding a deterministic function, as we did in linear regression, instead we try to fit a logistic function that captures the likelihood that the y -value is a 1 given the x -value. This “curve-fitting perspective” is why this is considered a regression problem.

If, as above, we think of each row of the matrix as an independent trial, then the chance that $y_i = 1$ is $p(x_i)$ and the chance that $y_i = 0$ is $1 - p(x_i)$, where $p(x)$ is given by the logistic function as in Equation 7.2. The likelihood of the results we obtained is therefore:

$$L(a, b) = C \prod_{i=1}^N p(x_i)^{y_i} (1 - p(x_i))^{(1-y_i)}$$

where C is a constant and we are exploiting the trick that, since y_i is either zero or one, $1 - y_i$ is correspondingly one or zero. Thus only $p(x_i)$ or $1 - p(x_i)$ occurs in each term of the product. If we group the terms according to x_i we obtain our earlier formula for $L(a, b)$.

This expression yields an apparently similar formula for the log-likelihood (up to an irrelevant constant):

$$\log L(X, a, b) = \sum_{i=1}^N y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i)).$$

Using vector notation, this can be further simplified, where again we drop irrelevant constants:

$$\log L(X, a, b) = Y \cdot \log p(X) + (1 - Y) \cdot \log(1 - p(X)).$$

To be absolutely concrete, in this formula, $p(X)$ is a vector

$$p(X) = [p(x_i)]_{i=1}^N = \left[\frac{1}{1 + e^{-ax_i - b}} \right]_{i=1}^N$$

so its entries are functions of the unknown parameters a and b .

We might naively try to maximize this by taking the derivatives with respect to a and b and setting them to zero, but this turns out to be impractical. So we need a different approach to finding the

parameters a and b which maximize this likelihood function. We will return to this problem later, but before we do so we will look at some generalizations and broader applications of the logistic model.

7.1.2 Logistic regression with multiple features

The next generalization we can consider of the logistic model is the situation where the log-odds of our event of interest depend linearly on multiple parameters. In other words, we have

$$\log \frac{p}{1-p} = m_1 x_1 + m_2 x_2 + \dots + m_k x_k + b$$

where the m_i and b are constants. Under this model, notice that *the incremental effects of changes to the different parameters x_i have independent effects on the probability*. So, for example, if x_1 were the number of times our potential customer saw an online advertisement and x_2 were the number of times they saw a print advertisement, by adopting this model we are assuming that the impact of seeing more online ads is completely unrelated to the impact of seeing more print ads.

The probability is again given by a sigmoid function

$$p(x_1, \dots, x_k) = \frac{1}{1 + e^{-(\sum_{i=1}^k m_i x_i + b)}}$$

This model has an $N \times k$ feature matrix whose rows are the values $x_1, \dots, x_k$ for each sample. The outcome of our experiment is recorded in an $N \times 1$ column vector Y whose entries are 0 or 1. The likelihood function is formally equivalent to what we computed in the case of a single feature, but it will be useful to be a bit careful about vector notation.

Following the same pattern we adopted for linear regression, let X be the $N \times (k+1)$ matrix whose first k columns contain the values x_i for each sample, and whose last column is all 1. Rename the intercept parameter as m_{k+1} and organize these parameters into a $(k+1) \times 1$ matrix M . Then

$$p(X) = \sigma(XM)$$

and our likelihood becomes

$$\log L(M) = Y \cdot \log \sigma(XM) + (1 - Y) \cdot \log(1 - \sigma(XM)). \quad (7.3)$$

7.2 Finding the maximum likelihood solution by gradient descent

Given a set of features X and targets Y for a logistic model, we now want to find the values M so that the log-likelihood of the model for those parameters, given the data, is maximized. While in linear regression we could find a nice closed form solution to this problem, the presence of the non-linear function $\sigma(x)$ in the likelihood makes that impossible for logistic regression. Instead, we will use the gradient descent method from Chapter 6 .

7.3 Gradient Descent and Logistic Regression

We can use gradient descent, as discussed in Chapter 6, to find the maximum likelihood set of parameters for our logistic model. As we saw earlier, in Equation 7.3, we have the log likelihood function

$$\log L(M) = Y \cdot \log \sigma(XM) + (1 - Y) \cdot \log(1 - \sigma(XM))$$

where Y is the vector of target 0/1 values, X is our $N \times (k + 1)$ data matrix whose last column is all ones, and M is the $k + 1 \times 1$ column vector of unknown parameters. Since gradient descent is naturally a *minimizing* algorithm, we will minimize the function $-\log L(M)$.

The key piece of information that we need is the gradient $-\nabla \log L$, where the variables are the entries of M . The complicating feature is the presence of the nonlinear function σ , so let's start with a simple observation about this function.

Lemma: The logistic function $\sigma(x)$ satisfies the differential equation

$$\frac{d\sigma}{dx} = \sigma(x)(1 - \sigma(x)).$$

Proof: Since

$$\begin{aligned}\sigma(x) &= \frac{1}{1 + e^{-x}}, \\ 1 - \sigma(x) &= \frac{e^{-x}}{1 + e^{-x}}.\end{aligned}$$

Then we calculate

$$\begin{aligned}\frac{d\sigma}{dx} &= \left(\frac{1}{1 + e^{-x}} \right)^2 e^{-x} \\ &= \left(\frac{1}{1 + e^{-x}} \right) \left(\frac{e^{-x}}{1 + e^{-x}} \right) \\ &= \sigma(x)(1 - \sigma(x))\end{aligned}$$

which is what we claimed.

We apply this differential equation to compute the gradient of $\log L$.

Theorem 7.1. Proposition: The gradient $-\nabla \log L(M)$ is given by

$$-\nabla \log L(M) = X^\top (\sigma(XM) - Y).$$

Notice that the right side of this equation yields a $(k + 1) \times 1$ column vector. The entries of this vector are the partial derivatives with respect to the coefficients m_i for $i = 1, \dots, k + 1$.

Proof: This is yet another exercise in the chain rule and keeping track of indices. Let's first look at the term $Y \cdot \log \sigma(XM)$. Writing it out, we have

$$Y \cdot \log \sigma(XM) = \sum_{i=1}^N y_i \log \sigma \left(\sum_{j=1}^{k+1} x_{ij} m_j \right).$$

Applying $\partial/\partial m_s$ to this yields

$$\sum_{i=1}^N y_i (1 - \sigma(\sum_{j=1}^{k+1} x_{ij} m_j)) x_{is}$$

where we've used the chain rule and the differential equation for σ discussed above. At the same time, we can apply $\partial/\partial m_s$ to the second term $(1 - Y) \cdot \log(1 - \sigma(XM))$ and obtain

$$- \sum_{i=1}^N (1 - y_i) \sigma(\sum_{j=1}^{k+1} x_{ij} m_j) x_{is}.$$

The term $\sum_{i=1}^N y_i \sigma(\sum_{j=1}^{k+1} x_{ij} m_j) x_{is}$ cancels, yielding

$$\frac{\partial \log L(M)}{\partial m_s} = - \sum_{i=1}^N (y_i - \sigma(\sum_{j=1}^{k+1} x_{ij} m_j)) x_{is}.$$

Since our weights M are naturally a $(k+1) \times 1$ column vector, looked at properly this is our desired formula:

$$-\nabla \log L(M) = X^\top (\sigma(XM) - Y).$$

Since the right side is an $(k+1) \times N$ matrix times an $N \times 1$ column vector, the result is a $(k+1) \times 1$ column vector whose entries are the partial derivatives of $-\log L(M)$ with respect to the weights m_s .

7.3.1 Gradient Descent on our synthetic data

Now we can apply gradient descent to find a maximum likelihood logistic model for the sample data that we generated from the logistic model and reported in Table 7.1. With the probability given as

$$p(x) = \frac{1}{1 + e^{-ax-b}}$$

we make an initial guess of $a = 1$ and $b = 0$, set a learning rate $\nu = .001$, and run the gradient descent algorithm for 30 iterations. We plot the negative log-likelihood for this algorithm on the left in Figure 7.2, where we see that it drops swiftly to a minimum value. The corresponding parameter values are $a = .6717$ and $b = -.0076$, and the fit of the corresponding logistic curve to the observed data is shown on the right in Figure 7.2.

The parameters used to generate the data are close to this; they were $a = \log(2) = .6931$ and $b = 0$.

7.3.2 Gradient Descent and Logistic Regression on “real” data

We conclude this first look at logistic regression and gradient descent by analyzing some simple real data. This dataset consists of about 2200 customers who patronize a certain food store. Among the features in the data set is a field giving the total dollars spent at the store by a customer; we

7 Logistic Regression

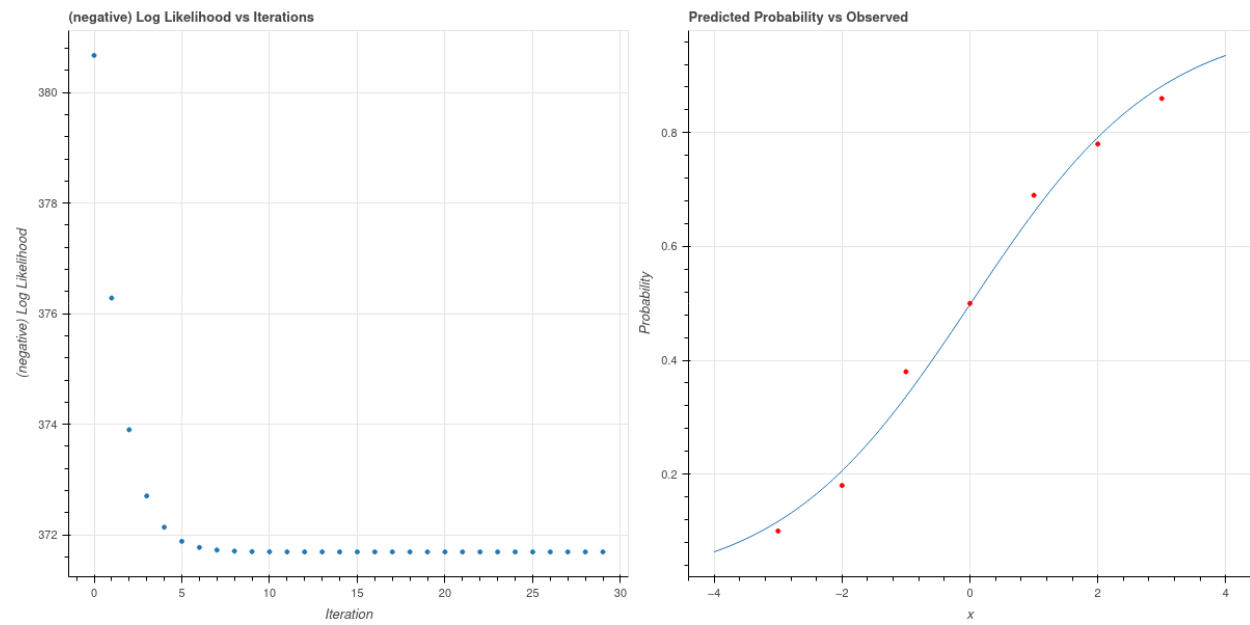


Figure 7.2: Max Likelihood Gradient Descent for Logistic Fitting

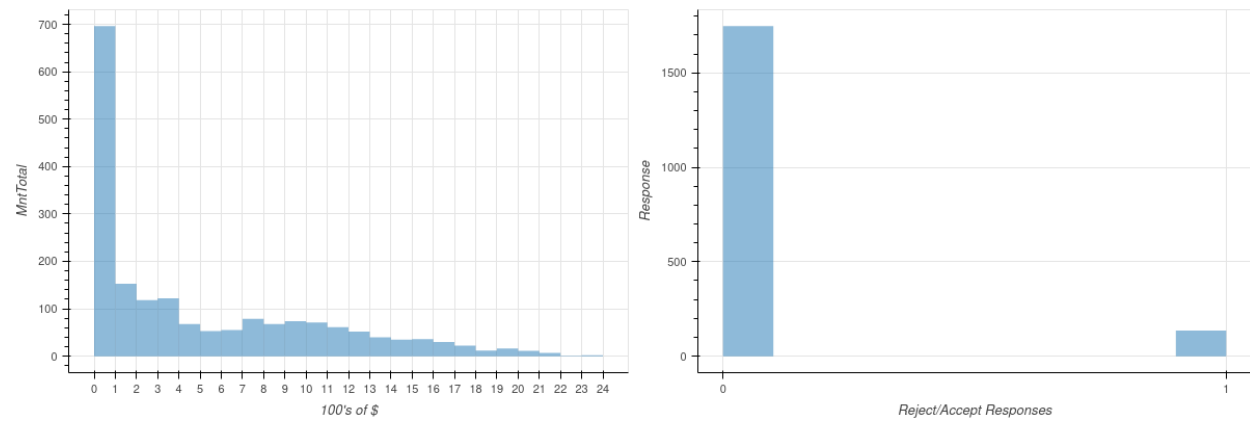


Figure 7.3: Food Marketing Data: Histograms of Expenditures and Response

will study that feature and its relationship to the question of whether or not the customer accepted a special offer from the store. (see [4] for the original data source).

The two plots in Figure 7.3 summarize the data. The first plot is a histogram showing the amounts spent by the customers; the second shows the distribution of responses.

We would like to know how expenditures increase the likelihood of customers accepting our offer. We therefore fit a logistic model to the data. The result is shown in Figure 7.4.

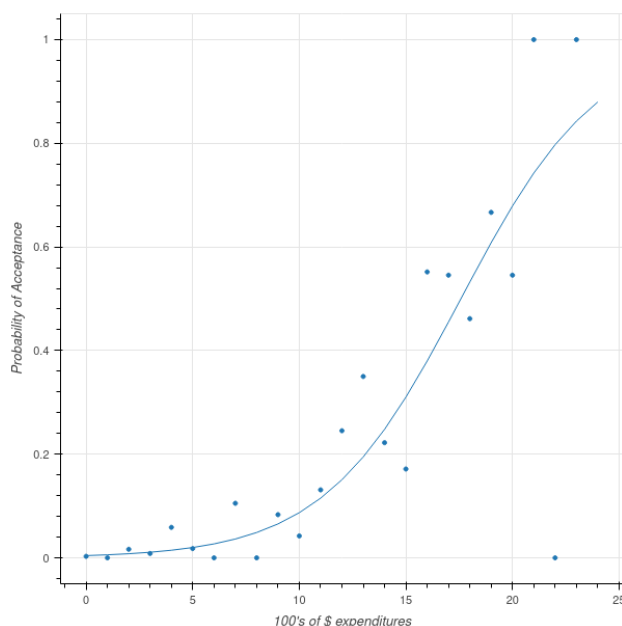


Figure 7.4: Logistic Model for Food Marketing

7.4 Logistic Regression and Classification

Beyond the kind of probability prediction that we have discussed up to this point, logistic regression is one of the most powerful techniques for attacking the classification problem. Let's start our discussion with a sample problem that is a simplified version of one of the most famous machine learning benchmark problems, the MNIST (Modified National Institute of Science and Technology) dataset of handwritten numerals. This dataset consists of 60000 labelled grayscale images of handwritten digits from 0 to 9. Each image is stored as a 28×28 array of integers from 0 to 255. Each cell of the array corresponds to a "pixel" in the image, and the contents of that cell is a grayscale value. See [5] for a more detailed description of how the dataset was constructed.

In Figure 7.5 is a picture of a handwritten "1" from the MNIST dataset.

Classification Problem for MNIST: Given a 28×28 array of grayscale values, determine which digit is represented.

At first glance, this does not look like a logistic regression problem. To make the connection clearer, let's simplify the problem and imagine that our database contains only labelled images of zeros

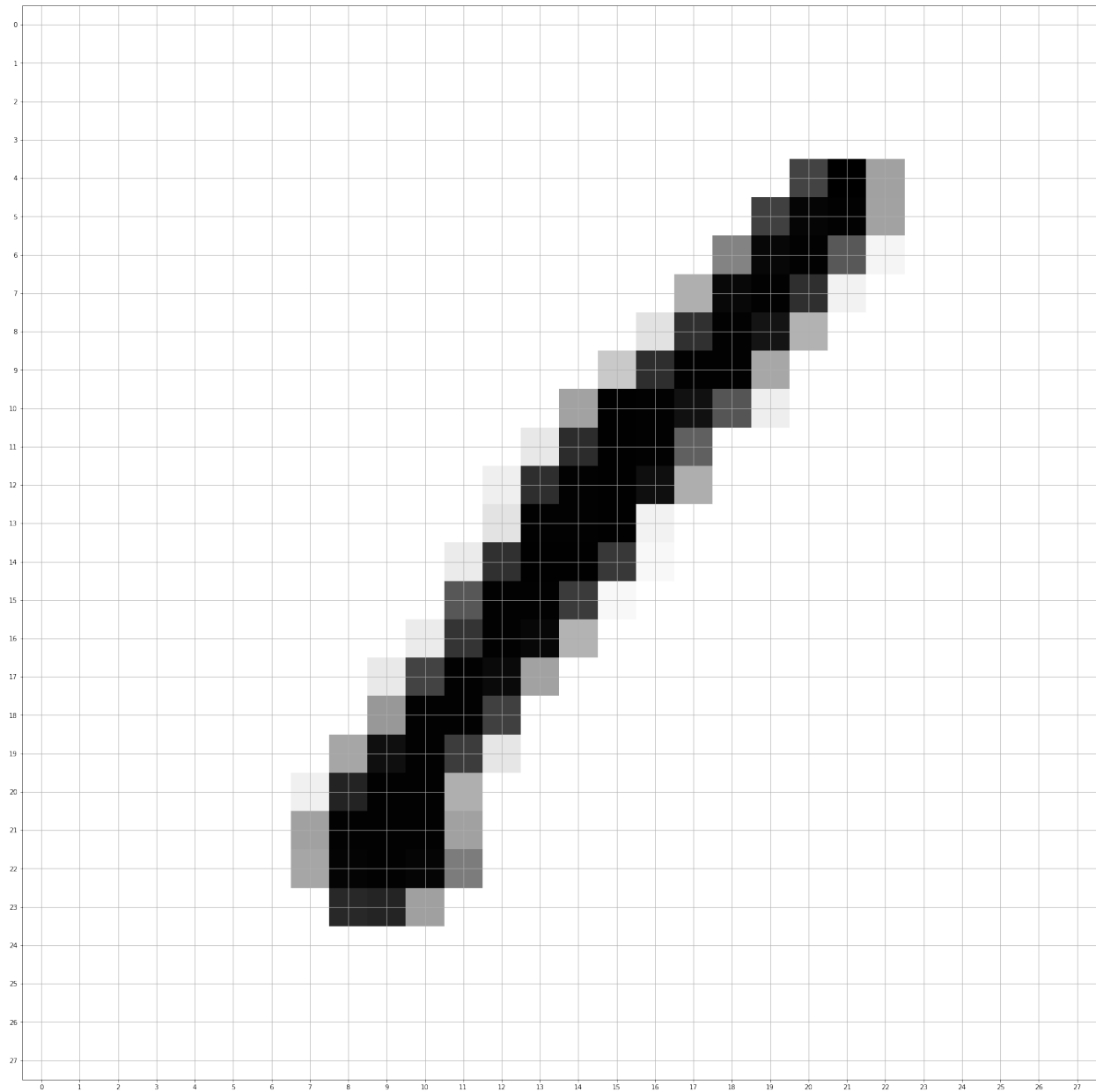


Figure 7.5: Handwritten One from MNIST

and ones – we’ll worry about how to handle the full problem later. So now our task is to determine which images are zeros, and which are ones.

Our approach will be to view each image as a vector of length $784 = 28 * 28$ by stringing the pixel values row by row into a one-dimensional vector, which following our conventions yields a matrix of size $N \times 784$ where N is the number of images. Since we may also need an “intercept”, we add a column of 1’s to our images yielding a data matrix X of size $N \times 785$. The labels y form a column vector of size N containing zeros and ones.

We will also simplify the data by converting the gray-scale images to monochrome by converting gray levels up to 128 as “white” and beyond 128 as “black”.

The logistic regression approach asks us to find the “best” vector M so that, for a given image vector x (extended by adding a one at the end), the function

$$p(x) = \frac{1}{1 + e^{-xM}}$$

is close to 1 if x represents a one, and is close to zero if x represents zero. Essentially we think of $p(x)$ as giving the probability that the vector x represents an image of a one. If we want a definite choice, then we can set a threshold value p_{th} and say that the image x is a one if $p(x) > p_{\text{th}}$ and zero otherwise. The natural choice of $p_{\text{th}} = .5$ amounts to saying that we choose the more likely of the two options under the model.

Since we are applying the logistic model we are assuming:

- that the value of each pixel in the image contributes something towards the chance of the total image being one;
- and the different pixels have independent, and additive effects on the odds of getting a one.

If we take this point of view, then we can ask for the vector M that is *most likely* to account for the labellings, and we can use our maximum likelihood gradient descent method to find M .

This approach is surprisingly effective. With the MNIST zeros and ones, and the gradient descent method discussed above, one can easily find M so that the logistic model predicts the correct classification with accuracy in the high 90% range.

7.4.1 Weights as filters

One interesting aspect of using logistic regression on images for classification is that we can interpret the optimum set of coefficients M as a kind of filter for our images. Remember that M is a vector with 785 entries, the last of which is an “intercept”.

The logistic model says that, for an image vector x , the log-odds that the image is a one is given by

$$\log \frac{p}{1-p} = \sum_{i=1}^{784} M_i x_i + M_{785}.$$

This means that if the value of M_i is positive, then large values in the i^{th} pixel *increase* the chance that our image is a one; while if M_i is negative, large values *decrease* the chance. However, the

values x_i are the gray scale “darkness” of the image, so the entries of M emphasize or de-emphasize dark pixels according to whether that dark pixel is more or less likely to occur in a one compared to a zero.

This observation allows us to interpret the weights M as a kind of “filter” for the image. In fact, if we rescale the entries of M (omitting the intercept) so that they lie between 0 and 255, we can arrange them as a 28×28 array and plot them as an image. The result of doing this for a selection of MNIST zeros and ones is shown on the left in Figure 7.6. The red (or positive) weights in the middle of the image tell us that if those pixels are dark, the image is more likely to be a one; the blue (or negative) weights scattered farther out tell us that if those pixels are dark, the image is more likely to be a zero.

What’s important to notice here is that we did not design this “filter” by hand, based on our understanding of the differences between a handwritten zero and one; instead, the algorithm “learned” the “best” filter to optimize its ability to distinguish these digits.

Here’s another example. Suppose we redo the MNIST problem above, but we try to distinguish 3’s from 8’s.

We have about 4500 of each digit, and we label the 3’s with zero and the 8’s with one. Then we use our maximum likelihood optimization. In this case, the filter is shown on the bottom in Figure 7.6.

7.5 Multiclass Logistic Regression

One could attack the problem of classifying the ten distinct classes of digits by, for example, labelling all of the zeros as class zero and everything else as class one, and finding a set of weights that distinguishes zero from everything else. Then, in turn, one could do the same for each of the other digits. Given an unknown image, this would yield a set of probabilities from which one could choose the most likely class. This type of classification is called “one vs rest”, for obvious reasons. It seems more natural, however, to construct a model that, given an image, assigns a probability that it belongs to each of the different possibilities. It is this type of multiclass logistic regression that we will study now.

Our goal is to build a model that, given an unknown image, returns a vector of ten probabilities, each of which we can interpret as the chance that our unknown image is in fact of a particular digit. If we *know* the image’s class, then its probability vector *should* be nine zeros with a single one in the position corresponding to the digit. So, for example, if our image is of a two, then the vector of probabilities

$$[p_1 \ p_2 \ p_3 \ \cdots \ p_9 \ p_{10}] = [0 \ 0 \ 1 \ \cdots \ 0 \ 0]$$

where p_j is the probability that our image is the digit $j - 1$. Notice also that the probabilities p_j must sum to one. We encode the class membership of our samples by constructing an $N \times r$ matrix Y , each row of which has a one in column j if that sample belongs to class j , and zeros elsewhere. This type of representation is sometimes called “one-hot” encoding.

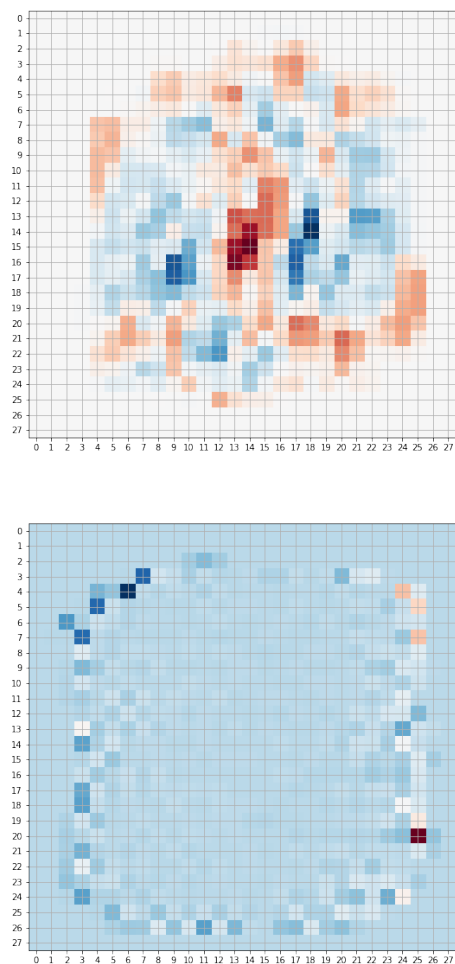


Figure 7.6: Rescaled weights (blue is negative). Top: 0 vs 1. Bottom: 3 vs 8.

So let's assume we have N data points, each with k features, and a one-hot encoded, $N \times r$ matrix of labels Y encoding the data into r classes. As usual, we add an “extra” feature, which is the constant 1 for each sample, to account for the “intercept”. So our data matrix will be $N \times (k+1)$.

Our goal will be to find a $(k+1) \times r$ matrix of “weights” M so that, for each sample, we compute r values, given by the rows of the matrix XM . These r values are linear functions of the features, but we need probabilities. In the one-dimensional case, we used the logistic function σ to convert our linear function to probabilities. In this higher dimensional case we use a generalization of σ called the “softmax” function.

Definition: Let $F : \mathbf{R}^r \rightarrow \mathbf{R}$ be the function

$$F(z_1, \dots, z_r) = \sum_{j=1}^r e^{z_j}$$

and let $\sigma : \mathbf{R}^r \rightarrow \mathbf{R}^r$ be the function

$$\sigma(z_1, \dots, z_r) = \begin{bmatrix} \frac{e^{z_1}}{F} & \dots & \frac{e^{z_r}}{F} \end{bmatrix}.$$

Notice that the coordinates of the vector $\sigma(z_1, \dots, z_r)$ are all between 0 and 1, and their sum is one.

Our multiclass logistic model will say that the probability vector that gives the probabilities that a particular sample belongs to a particular class is given by the rows of the matrix $\sigma(XM)$, where $\sigma(XM)$ means applying the function σ to each row of the $N \times r$ matrix XM . For later computation, if:

- $x = X[i, :]$ is the $k+1$ -entry feature vector of a single sample – a row of the data matrix X
- $m_j = M[:, j]$ is the $k+1$ -entry column vector corresponding to the j^{th} column of M ,

then the probability vector $[p_t]_{t=1}^r$ has entries

$$p_t(x; M) = \frac{e^{x \cdot m_t}}{\sum_{s=1}^r e^{x \cdot m_s}}.$$

7.5.1 Multiclass logistic regression - the likelihood

The probability vector $[p_t(x; M)]$ encodes the probabilities that the sample with feature vector x belongs to each of the possible classes. That is,

$$p_j(x; M) = \text{the chance that } x \text{ is in class } j.$$

We have captured the class membership of the samples in an $N \times r$ matrix Y which is “one-hot” encoded. Each row of this matrix has a zero in each place, except in the “correct” class, where it is one. Let $y = Y[i, :]$ be the i^{th} row of this matrix, so it is an r -entry row vector which is 1 in the position giving the “correct” class for our sample x .

So we can represent the chance that sample i belongs to class j as

$$P(\text{ sample } i \text{ in class } j) = \prod_{s=1}^r p_s(x; M)^{y_s}.$$

Taking the logarithm, we find

$$\log P = \sum_{s=1}^r y_s \log p_s(x; M). \quad (7.4)$$

Since each sample is independent, the total likelihood is the product of these probabilities, and the log-likelihood the corresponding sum:

$$\log L(M) = \sum_{X,Y} \sum_{s=1}^r y_s \log p_s(x; M).$$

where the sum is over the N rows of X and Y . This is equivalent to the matrix expression

$$\log L(M) = \text{trace}(Y^\top \log P) = \text{trace}(Y^\top \log \sigma(XM))$$

This is the multiclass generalization of Equation 7.3. To see the connection, notice that, in the case where we have only two classes, $y_2 = 1 - y_1$ and $p_2(x; M) = 1 - p_1(x; M)$, so this sum is the same as in the two class situation.

7.5.2 Multiclass logistic regression - the gradient.

To find the “best-fitting” multiclass logistic model by gradient descent, we need an expression for the gradient of the likelihood $L(M)$. As with all of these calculations, this is an exercise in the chain rule. We start with the formula

$$p_s(x; M) = \frac{e^{x \cdot m_s}}{\sum_{t=1}^r e^{x \cdot m_t}}$$

The gradient of this is made up of the derivatives with respect to the m_{bq} where $b = 1, \dots, k+1$ and $q = 1, \dots, r$ so it’s natural to think of this gradient as a $(k+1) \times r$ matrix, the same shape as M . Remember that each m_s is the s^{th} column of M so is made up of m_{bs} for $b = 1, \dots, k+1$.

Looking at

$$\frac{\partial p_s}{\partial m_{bq}}$$

there are two cases to consider. The first is when q and s are different, so the numerator of p_s doesn’t involve m_{bq} . In this case the derivative is

$$\frac{\partial p_s}{\partial m_{bq}} = -\frac{e^{x \cdot m_s} e^{x \cdot m_q} x_b}{(\sum_{t=1}^r e^{x \cdot m_t})^2} = -p_s p_q x_b$$

7 Logistic Regression

In vector terms:

$$\left[\frac{\partial p_s}{\partial m_{bq}}\right]_{b=1}^{k+1} = -p_q p_s [x_b]_{b=1}^{k+1}$$

as an equality of $k + 1$ -entry row vectors. This can be written more simply as a vector equation:

$$\frac{\partial p_s}{\partial m_q} = -p_q p_s x^\top. \quad (q \neq s).$$

When $q = s$, we have

$$\frac{\partial p_s}{\partial m_{bs}} = \frac{e^{x \cdot m_s} x_b}{\sum_{t=1}^r e^{x \cdot m_t}} - \frac{e^{x \cdot m_s} e^{x \cdot m_s} x_b}{(\sum_{t=1}^r e^{x \cdot m_t})^2} = p_s(1 - p_s)x_b$$

or in vector terms

$$\frac{\partial p_s}{\partial m_s} = p_s(1 - p_s)x^\top.$$

Important: The gradient on the left is properly seen as a column vector (because m_s is a column of the matrix M , with $k + 1$ entries), and since x is a row of the data matrix, to keep the indices straight, we need $x^\top$ on the right.

Now we can use these formulae together with the expression for $\log L(M)$ to obtain the gradient. Using the vector form, we have

$$\frac{\partial \log L(M)}{\partial m_q} = \sum_{X,Y} \sum_{s=1}^r y_s \frac{\partial \log p_s}{\partial m_q}.$$

Using our computations above, the chain rule, and the derivative of the logarithm, this is the sum

$$\frac{\partial \log L(M)}{\partial m_q} = \sum_{X,Y} \sum_{s=1}^r y_s (I_{qs} - p_q) x^\top$$

where $I_{qs} = 1$ if $q = s$ and zero otherwise.

Now $y_s I_{qs}$ is zero unless $s = q$, and the sum $\sum_{s=1}^r y_s = 1$, so this simplifies further to

$$\frac{\partial \log L(M)}{\partial m_q} = \sum_{X,Y} (y_q - p_q) x^\top.$$

This is equivalent to the matrix expression

$$\nabla \log L(M) = X^\top (Y - P) = X^\top (Y - \sigma(XM)). \quad (7.5)$$

Compare Equation 7.5 to Theorem 7.1 and we see that the form is identical whether in the two-class or multi-class case if we set things up properly.

7.5.3 Multiclass Gradient Descent

Given:

- an $N \times (k + 1)$ data matrix X whose last column is all 1,
- an $N \times r$ matrix Y that “one-hot” encodes the labels of the classification problem;
- a random $(k + 1) \times r$ matrix M of initial guesses for the parameters;
- a “learning rate” ν ,

Iterate:

$$M^{(j+1)} = M^{(j)} + \nu X^T (Y - \sigma(XM^{(j)}))$$

until M changes by less than some tolerance.

7.6 Stochastic Gradient Descent for Logistic Regression

The gradient descent process for logistic regression suffers from the same limitation as for linear regression that we discussed in Section 6.5. Each iteration requires us to compute the matrix product $X^T(Y - \sigma(XM^{(j)}))$ which uses all of the data and is therefore time- and even more importantly memory-intensive.

Therefore logistic regression is also a candidate for stochastic gradient descent, in which we iterate over each data point x and associated target y and make adjustments point by point. To do this we use the gradient from Equation 7.5 in the much simpler case where X is a single row of the data matrix and Y is a row vector that is zero except for a one in the spot corresponding to the target class for X .

8 Support Vector Machines

8.1 Introduction

Suppose that we are given a collection of data made up of samples from two different classes, and we would like to develop an algorithm that can distinguish between the two classes. For example, given a picture that is either a dog or a cat, we'd like to be able to say which of the pictures are dogs, and which are cats. For another example, we might want to be able to distinguish “real” emails from “spam.” This type of problem is called a *classification* problem.

Typically, one approaches a classification problem by beginning with a large set of data for which you know the classes, and you use that data to *train* an algorithm to correctly distinguish the classes for the test cases where you already know the answer. For example, you start with a few thousand pictures labelled “dog” and “cat” and you build your algorithm so that it does a good job distinguishing the dogs from the cats in this initial set of *training data*. Then you apply your algorithm to pictures that aren't labelled and rely on the predictions you get, hoping that whatever let your algorithm distinguish between the particular examples will generalize to allow it to correctly classify images that aren't pre-labelled.

Because classification is such a central problem, there are many approaches to it. We will see several of them through the course of these lectures. We will begin with a particular classification algorithm called “Support Vector Machines” (SVM) that is based on linear algebra. The SVM algorithm is widely used in practice and has a beautiful geometric interpretation, so it will serve as a good beginning for later discussion of more complicated classification algorithms.

Incidentally, I'm not sure why this algorithm is called a “machine”; the algorithm was introduced in the paper [6] where it is called the “Optimal Margin Classifier” and as we shall see that is a much better name for it.

My presentation of this material was heavily influenced by the beautiful paper [7].

8.2 A simple example

Let us begin our discussion with a very simple dataset (see [8] and [9]). This data consists of various measurements of physical characteristics of 344 penguins of 3 different species: Gentoo, Adelie, and Chinstrap. If we focus our attention for the moment on the Adelie and Gentoo species, and plot their body mass against their culmen depth, we obtain the following scatterplot.

Incidentally, a bird's *culmen* is the upper ridge of their beak, and the *culmen depth* is a measure of the thickness of the beak. There's a nice picture at [9] for the penguin enthusiasts.

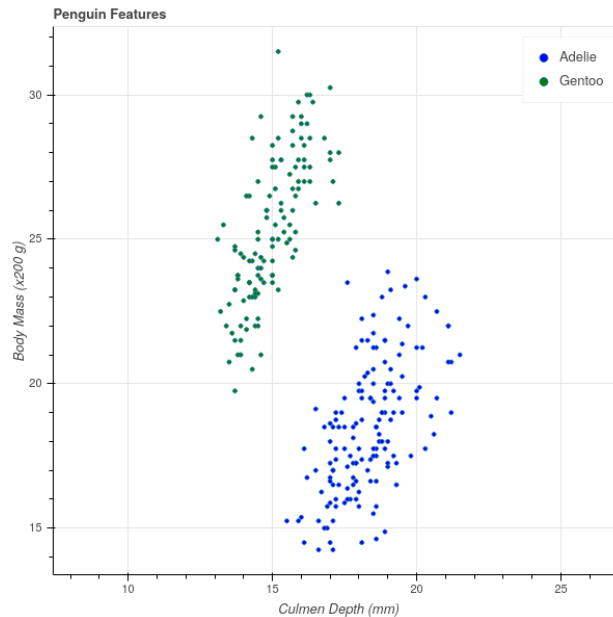


Figure 8.1: Penguin Scatterplot

A striking feature of this scatter plot is that there is a clear separation between the clusters of Adelie and Gentoo penguins. Adelie penguins have deeper culmens and less body mass than Gentoo penguins. These characteristics seem like they should provide a way to classify a penguin between these two species based on these two measurements.

One way to express the separation between these two clusters is to observe that one can draw a line on the graph with the property that all of the Adelie penguins lie on one side of that line and all of the Gentoo penguins lie on the other. In Figure 8.2 I've drawn in such a line (which I found by eyeballing the picture in Figure 8.1). The line has the equation

$$Y = 1.25X + 2.$$

The fact that all of the Gentoo penguins lie above this line means that, for the Gentoo penguins, their body mass in grams is at least 400 more than 250 times their culmen depth in mm. (Note that the y axis of the graph is scaled by 200 grams).

$$\text{Gentoo mass} > 250(\text{Gentoo culmen depth}) + 400$$

while

$$\text{Adelie mass} < 250(\text{Adelie culmen depth}) + 400.$$

Now, if we measure a penguin caught in the wild, we can compute $250(\text{culmen depth}) + 400$ for that penguin and if this number is greater than the penguin's mass, we say it's an Adelie; otherwise, a Gentoo. Based on the experimental data we've collected – the *training* data – this seems likely to work pretty well.

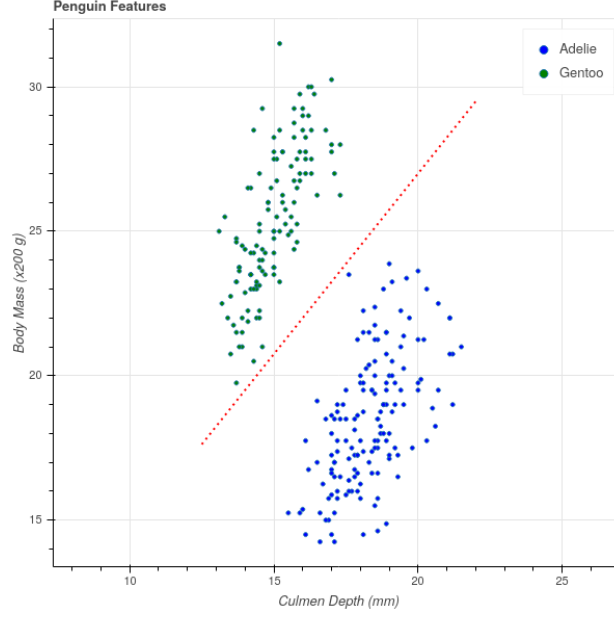


Figure 8.2: Penguins with Separating Line

8.3 The general case

To generalize this approach, let's imagine now that we have n samples and k features (or measurements) for each sample. As before, we can represent this data as an $n \times k$ data matrix X . In the penguin example, our data matrix would be 344×2 , with one row for each penguin and the columns representing the mass and the culmen depth. In addition to this numerical data, we have a classification that assigns each row to one of two classes. Let's represent the classes by a $n \times 1$ vector Y , where $y_i = +1$ if the i^{th} sample is in one class, and $y_i = -1$ if that i^{th} sample is in the other. Our goal is to predict Y based on X – but unlike in linear regression, Y takes on the values of ± 1 .

In the penguin case, we were able to find a line that separated the two classes and then classify points by which side of the line the point was on. We can generalize this notion to higher dimensions. Before attacking that generalization, let's recall a few facts about the generalization to $\mathbf{R}^k$ of the idea of a line.

8.3.1 Hyperplanes

The correct generalization of a line given by an equation $w_1x_1 + w_2x_2 + b = 0$ in $\mathbf{R}^2$ is an equation $f(x) = 0$ where $f(x)$ is a degree one polynomial

$$f(x) = f(x_1, \dots, x_k) = w_1x_1 + w_2x_2 + \dots + w_kx_k + b \quad (8.1)$$

It's easier to understand the geometry of an equation like $f(x) = 0$ in Equation 8.1 if we think of the coefficients w_i as forming a *nonzero* vector $w = (w_1, \dots, w_k)$ in $\mathbf{R}^k$ and writing the formula for

$f(x)$ as

$$f(x) = w \cdot x + b$$

Lemma: Let $f(x) = w \cdot x + b$ with $w \in \mathbf{R}^k$ a nonzero vector and b a constant in $\mathbf{R}$.

- The inequalities $f(x) > 0$ and $f(x) < 0$ divide up $\mathbf{R}^k$ into two disjoint subsets (called half spaces), in the way that a line in $\mathbf{R}^2$ divides the plane in half.
- The vector w is normal vector to the hyperplane $f(x) = 0$. Concretely this means that if p and q are any two points in that hyperplane, then $w \cdot (p - q) = 0$.
- Let $p = (u_1, \dots, u_k)$ be a point in $\mathbf{R}^k$. Then the perpendicular distance D from p to the hyperplane $f(x) = 0$ is

$$D = \frac{f(p)}{\|w\|}$$

Proof: The first part is clear since the inequalities are mutually exclusive. For the second part, suppose that p and q satisfy $f(x) = 0$. Then $w \cdot p + b = w \cdot q + b = 0$. Subtracting these two equations gives $w \cdot (p - q) = 0$, so $p - q$ is orthogonal to w .

For the third part, consider Figure 8.3. The point q is an arbitrary point on the hyperplane defined by the equation $w \cdot x + b = 0$. The distance from the hyperplane to p is measured along the dotted line perpendicular to the hyperplane. The dot product $w \cdot (p - q) = \|w\| \|p - q\| \cos(\theta)$ where θ is the angle between $p - q$ and w – which is complementary to the angle between $p - q$ and the hyperplane. The distance D is therefore

$$D = \frac{w \cdot (p - q)}{\|w\|}.$$

However, since q lies on the hyperplane, we know that $w \cdot q + b = 0$ so $w \cdot q = -b$. Therefore $w \cdot (p - q) = w \cdot p + b = f(p)$, which is the formula we seek.

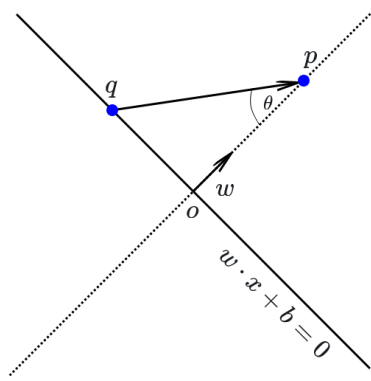


Figure 8.3: Distance to a Hyperplane

8.3.2 Linear separability and Margins

Now we can return to our classification scheme. The following definition generalizes our two dimensional picture from the penguin data.

Definition: Suppose that we have an $n \times k$ data matrix X and a set of labels Y that assign the n samples to one of two classes. Then the labelled data is said to be *linearly separable* if there is a vector w and a constant b so that, if $f(x) = w \cdot x + b$, then $f(x) > 0$ whenever $x = (x_1, \dots, x_k)$ is a row of X – a sample – belonging to the +1 class, and $f(x) < 0$ whenever x belongs to the -1 class. The solutions to the equation $f(x) = 0$ in this situation form a hyperplane that is called a *separating hyperplane* for the data.

In the situation where our data falls into two classes that are linearly separable, our classification strategy is to find a separating hyperplane f for our training data. Then, given a point x whose class we don't know, we can evaluate $f(x)$ and assign x to a class depending on whether $f(x) > 0$ or $f(x) < 0$.

This definition begs two questions about a particular dataset:

1. How do we tell if the two classes are linearly separable?
2. If the two sets are linearly separable, there are infinitely many separating hyperplanes. To see this, look back at the penguin example and notice that we can ‘wiggle’ the red line a little bit and it will still separate the two sets. Which is the ‘best’ separating hyperplane?

Let's try to make the first of these two questions concrete. We have two sets of points A and B in $\mathbf{R}^k$, and we want to (try to) find a vector w and a constant b so that $f(x) = w \cdot x + b$ takes strictly positive values for $x \in A$ and strictly negative ones for $x \in B$. Let's approach the problem by first choosing w and then asking whether there is a b that will work. In the two dimensional case, this is equivalent to choosing the slope of our line, and then asking if we can find an intercept so that the line passes between the two classes.

In algebraic terms, we are trying to solve the following system of inequalities: given w , find b so that:

$$w \cdot x + b > 0 \text{ for all } x \text{ in } A$$

and

$$w \cdot x + b < 0 \text{ for all } x \text{ in } B.$$

This is only going to be possible if there is a gap between the smallest value of $w \cdot x$ for $x \in A$ and the largest value of $w \cdot x$ for $x \in B$. In other words, given w there is a b so that $f(x) = w \cdot x + b$ separates A and B if

$$\max_{x \in B} w \cdot x < \min_{x \in A} w \cdot x.$$

If this holds, then choose b so that $-b$ lies in this open interval and you will obtain a separating hyperplane.

Proposition: The sets A and B are linearly separable if there is a w so that

$$\max_{x \in B} w \cdot x < \min_{x \in A} w \cdot x$$

If this inequality holds for some w , and $-b$ within this open interval, then $f(x) = w \cdot x + b$ is a separating hyperplane for A and B .

*Figure 8.4 is an illustration of this argument for a subset of the penguin data. Here, we have fixed $w = (1.25, -1)$ coming from the line $y = 1.25x + 2$ that we eyeballed earlier. For each Gentoo (green) point x_i , we computed $-b = w \cdot x_i$ and drew the line $f(x) = w \cdot x - w \cdot x_i$ giving a family of parallel lines through each of the green points. Similarly for each Adelle (blue) point we drew the corresponding line. The maximum value of $w \cdot x$ for the blue points turned out to be 1.998 and the minimum value of $w \cdot x$ for the green points turned out to be 2.003. Thus we have two lines with a gap between them, and any parallel line in that gap will separate the two sets.

Finally, among all the lines *with this particular w* , it seems that the **best** separating line is the one running right down the middle of the gap between the boundary lines. Any other line in the gap will be closer to either the blue or green set than the midpoint line is.

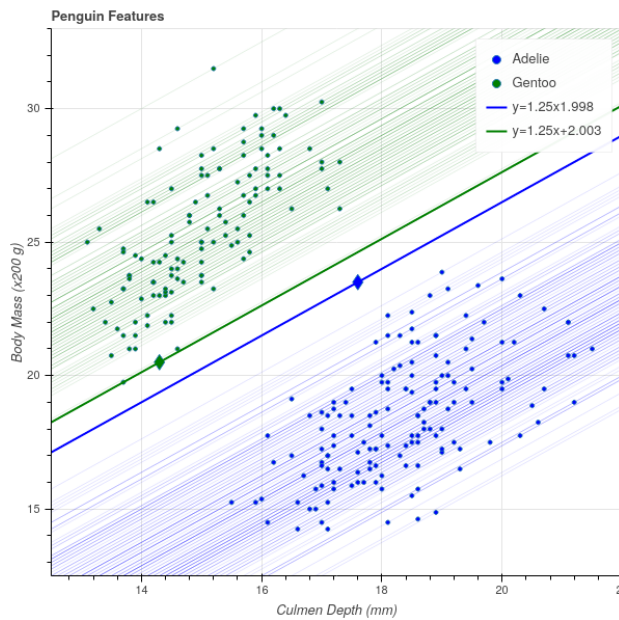


Figure 8.4: Lines in Penguin Data for $w = (1.25, -1)$

Let's put all of this together and see if we can make sense of it in general.

Suppose that A^+ and A^- are finite point sets in $\mathbf{R}^k$ and $w \in \mathbf{R}^k$ such that

$$B^-(w) = \max_{x \in A^-} w \cdot x < \min_{x \in A^+} w \cdot x = B^+(w).$$

Let x^- be a point in A^- with $w \cdot x^- = B^-(w)$ and x^+ be a point in A with $w \cdot x^+ = B^+(w)$. The two hyperplanes $f^\pm(x) = w \cdot x - B^\pm$ have the property that:

$$f^+(x) \geq 0 \text{ for } x \in A^+ \text{ and } f^+(x) < 0 \text{ for } x \in A^-$$

and

$$f^-(x) \leq 0 \text{ for } x \in A^- \text{ and } f^-(x) > 0 \text{ for } x \in A^+$$

Hyperplanes like f^+ and f^- , which “just touch” a set of points, are called supporting hyperplanes.

Definition: Let A be a set of points in $\mathbf{R}^k$. A hyperplane $f(x) = w \cdot x + b = 0$ is called a *supporting hyperplane* for A if $f(x) \geq 0$ for all $x \in A$ and $f(x) = 0$ for at least one point in A , or if $f(x) \leq 0$ for all $x \in A$ and $f(x) = 0$ for at least one point in A .

The gap between the two supporting hyperplanes f^+ and f^- is called the *margin* between A and B for w .

Definition: Let f^+ and f^- be as in the discussion above for point sets A^+ and A^- and vector w . Then the orthogonal distance between the two hyperplanes f^+ and f^- is called the geometric margin $\tau_w(A^+, A^-)$ (along w) between A^+ and A^- . We have

$$\tau_w(A^+, A^-) = \frac{B^+(w) - B^-(w)}{\|w\|}.$$

Now we can propose an answer to our second question about the best classifying hyperplane.

Definition: The *optimal margin* $\tau(A^+, A^-)$ between A^+ and A^- is the largest value of τ_w over all possible w for which $B^-(w) < B^+(w)$:

$$\tau(A^+, A^-) = \max_w \tau_w(A^+, A^-).$$

If w is such that $\tau_w = \tau$, then the hyperplane $f(x) = w \cdot x - \frac{B^+ + B^-}{2}$ is the *optimal margin classifying hyperplane*.

The optimal classifying hyperplane runs “down the middle” of the gap between the two supporting hyperplanes f^+ and f^- that give the sides of the optimal margin.

We can make one more observation about the maximal margin. If we find a vector w so that $f^+(x) = w \cdot x - B^+$ and $f^-(x) = w \cdot x - B^-$ are the two supporting hyperplanes such that the gap between them is the optimal margin, then this gap gives us an estimate on how close together the points in A^+ and A^- can be. This is visible in Figure 8.4, where it’s clear that to get from a blue point to a green one, you have to cross the gap between the two supporting hyperplanes.

Proposition: The closest distance between points in A^+ and A^- is greater than or equal to the optimal margin:

$$\min_{p \in A^+, q \in A^-} \|p - q\| \geq \tau(A^+, A^-)$$

Proof: We have $f^+(p) = w \cdot p - B^+ \geq 0$ and $f^-(q) = w \cdot q - B^- \leq 0$. These two inequalities imply that

$$w \cdot (p - q) \geq B^+ - B^- > 0.$$

Therefore

$$\|p - q\| \|w\| \geq |w \cdot (p - q)| \geq |B^+ - B^-|$$

and so

$$\|p - q\| \geq \frac{B^+ - B^-}{\|w\|} = \tau(A^+, A^-)$$

If this inequality were always *strict* – that is, if the optimal margin equalled the minimum distance between points in the two clusters – then this would give us an approach to finding this optimal margin.

Unfortunately, that isn't the case. In Figure 8.5, we show a very simple case involving only six points in total in which the distance between the closest points in A^+ and A^- is larger than the optimal margin.

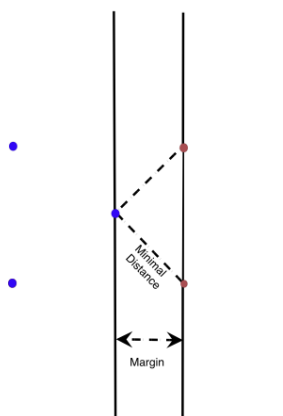


Figure 8.5: Shortest distance between $+$ and $-$ points can be greater than the optimal margin

At least now our problem is clear. Given our two point sets A^+ and A^- , find w so that $\tau_w(A^+, A^-)$ is maximal among all w where $B^-(w) < B^+(w)$. This is an optimization problem, but unlike the optimization problems that arose in our discussions of linear regression and principal component analysis, it does not have a closed form solution. We will need to find an algorithm to determine w by successive approximations. Developing that algorithm will require thinking about a new concept known as *convexity*.

8.4 Convexity, Convex Hulls, and Margins

In this section we introduce the notion of a *convex set* and the particular case of the *convex hull* of a finite set of points. As we will see, these ideas will give us a different interpretation of the margin between two sets and will eventually lead to an algorithm for finding the optimal margin classifier.

Definition: A subset U of $\mathbf{R}^k$ is *convex* if, for any pair of points p and q in U , every point t on the line segment joining p and q also belongs to U . In vector form, for every $0 \leq s \leq 1$, the point $t(s) = (1-s)p + sq$ belongs to U . (Note that $t(0) = p$, $t(1) = q$, and so $t(s)$ traces out the segment joining p to q .)

*Figure 8.6 illustrates the difference between convex sets and non-convex ones.

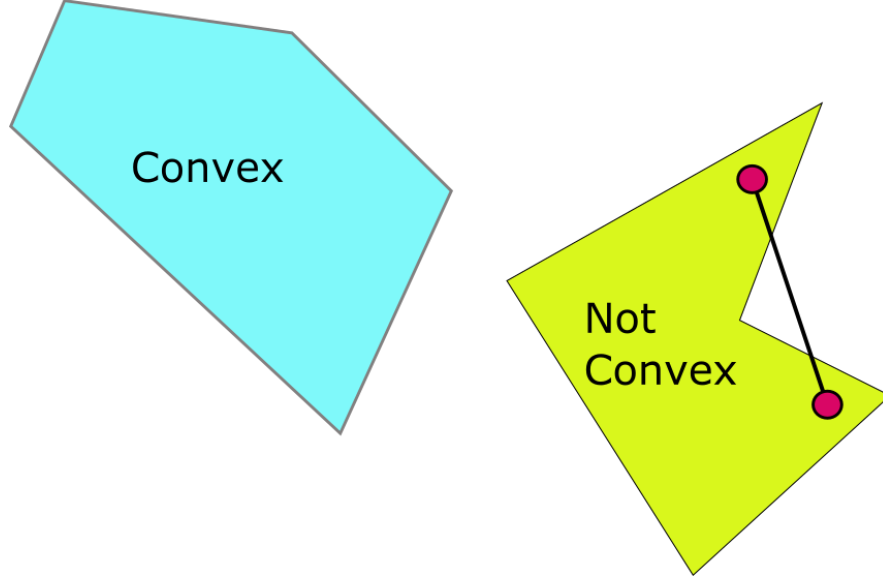


Figure 8.6: Convex vs Non-Convex Sets

The key idea from convexity that we will need to solve our optimization problem and find the optimal margin is the idea of the *convex hull* of a finite set of points in $\mathbf{R}^k$.

Definition: Let $S = \{q_1, \dots, q_N\}$ be a finite set of N points in $\mathbf{R}^k$. The *convex hull* $C(S)$ of S is the set of points

$$p = \sum_{i=1}^N \lambda_i q_i$$

as $\lambda_1, \dots, \lambda_N$ runs over all positive real numbers such that

$$\sum_{i=1}^N \lambda_i = 1.$$

There are a variety of ways to think about the convex hull $C(S)$ of a set of points S , but perhaps the most useful is that it is the smallest convex set that contains all of the points of S . That is the content of the next lemma.

Lemma: $C(S)$ is convex. Furthermore, let U be any convex set containing all of the points of S . Then U contains $C(S)$.

Proof: To show that $C(S)$ is convex, we apply the definition. Let p_1 and p_2 be two points in $C(S)$, so that let $p_j = \sum_{i=1}^N \lambda_i^{(j)} q_i$ where $\sum_{i=1}^N \lambda_i^{(j)} = 1$ for $j = 1, 2$. Then a little algebra shows that

$$(1-s)p_1 + sp_2 = \sum_{i=1}^N (s\lambda_i^{(1)} + (1-s)\lambda_i^{(2)})q_i$$

and $\sum_{i=1}^N (s\lambda_i^{(1)} + (1-s)\lambda_i^{(2)}) = 1$. Therefore all of the points $(1-s)p_1 + sp_2$ belong to $C(S)$, and therefore $C(S)$ is convex.

For the second part, we proceed by induction. Let U be a convex set containing S . Then by the definition of convexity, U contains all sums $\lambda_i q_i + \lambda_j q_j$ where $\lambda_i + \lambda_j = 1$. Now suppose that U contains all the sums $\sum_{i=1}^N \lambda_i q_i$ where exactly $m - 1$ of the λ_i are non-zero for some $m < N$. Consider a sum

$$q = \sum_{i=1}^N \lambda_i q_i$$

with exactly m of the $\lambda_i \neq 0$. For simplicity let's assume that $\lambda_i \neq 0$ for $i = 1, \dots, m$. Now let $T = \sum_{i=1}^{m-1} \lambda_i$ and set

$$q' = \sum_{i=1}^{m-1} \frac{\lambda_i}{T} q_i.$$

This point q' belongs to U by the inductive hypothesis. Also, $(1 - T) = \lambda_m$. Therefore by convexity of U ,

$$q = (1 - T)q_m + Tq'$$

also belongs to U . It follows that all of $C(S)$ belongs to U .

In Figure 8.7 we show our penguin data together with the convex hull of points corresponding to the two types of penguins. Notice that the boundary of each convex hull is a finite collection of line segments that join the “outermost” points in the point set.

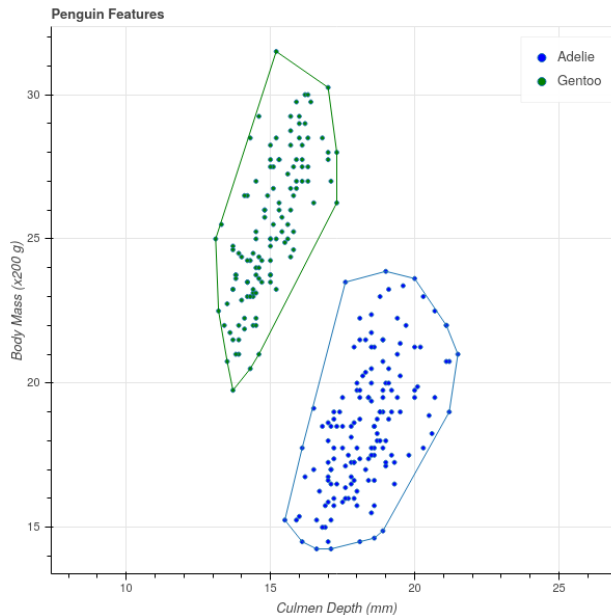


Figure 8.7: The Convex Hull

One very simple example of a convex set is a half-plane. More specifically, if $f(x) = w \cdot x + b = 0$ is a hyperplane, then the two “sides” of the hyperplane, meaning the subsets $\{x : f(x) \geq 0\}$ and $\{x : f(x) \leq 0\}$, are both convex. (This is exercise 1 in Section 8.7).

As a result of this observation, and the Lemma above, we can conclude that if $f(x) = w \cdot x + b = 0$ is a supporting hyperplane for the set S – meaning that either $f(x) \geq 0$ for all $x \in S$, or $f(x) \leq 0$

for all $x \in S$, with at least one point $x \in S$ such that $f(x) = 0$ – then $f(x) = 0$ is a supporting hyperplane for the entire convex hull. After all, if $f(x) \geq 0$ for all points $x \in S$, then S is contained in the convex set of points where $f(x) \geq 0$, and therefore $C(S)$ is contained in that set as well.

Interestingly, however, the converse is true as well – the supporting hyperplanes of $C(S)$ are exactly the same as those for S .

Lemma: Let S be a finite set of points in $\mathbf{R}^k$ and let $f(x) = w \cdot x + b = 0$ be a supporting hyperplane for $C(S)$. Then $f(x)$ is a supporting hyperplane for S .

Proof: Suppose $f(x) = 0$ is a supporting hyperplane for $C(S)$. Let's assume that $f(x) \geq 0$ for all $x \in C(S)$ and $f(x^*) = 0$ for a point $x^* \in C(S)$, since the case where $f(x) \leq 0$ is identical. Since $S \subset C(S)$, we have $f(x) \geq 0$ for all $x \in S$. To show that $f(x) = 0$ is a supporting hyperplane, we need to know that $f(x) = 0$ for at least one point $x \in S$.

Let x' be the point in S where $f(x')$ is minimal among all $x \in S$. Note that $f(x') \geq 0$. Then the hyperplane $g(x) = f(x) - f(x')$ has the property that $g(x) \geq 0$ on all of S , and $g(x') = 0$. Since the halfplane $g(x) \geq 0$ is convex and contains all of S , we have $C(S)$ contained in that halfplane. So, on the one hand we have $g(x^*) = f(x^*) - f(x') \geq 0$. On the other hand $f(x^*) = 0$, so $-f(x') \geq 0$, so $f(x') \leq 0$. Since $f(x')$ is also greater or equal to zero, we have $f(x') = 0$, and so we have found a point of S on the hyperplane $f(x) = 0$. Therefore $f(x) = 0$ is also a supporting hyperplane for S .

This argument can be used to give an alternative description of $C(S)$ as the intersection of all halfplanes containing S arising from supporting hyperplanes for S . This is exercise 2 in Section 8.7. It also has as a corollary that $C(S)$ is a closed set.

Lemma: $C(S)$ is compact.

Proof: Exercise 2 in Section 8.7 shows that it is the intersection of closed sets in $\mathbf{R}^k$, so it is closed. Exercise 3 shows that $C(S)$ is bounded. Thus it is compact.

Now let's go back to our optimal margin problem, so that we have linearly separable sets of points A^+ and A^- . Recall that we showed that the optimal margin was at most the minimal distance between points in A^+ and A^- , but that there could be a gap between the minimal distance and the optimal margin – see Figure 8.5 for a reminder.

It turns out that by considering the minimal distance between $C(A^+)$ and $C(A^-)$, we can “close this gap.” The following proposition shows that we can change the problem of finding the optimal margin into the problem of finding the closest distance between the convex hulls of $C(A^+)$ and $C(A^-)$. The following proposition generalizes the Proposition at the end of Section 8.3.2.

Proposition: Let A^+ and A^- be linearly separable sets in $\mathbf{R}^k$. Let $p \in C(A^+)$ and $q \in C(A^-)$ be any two points. Then

$$\|p - q\| \geq \tau(A^+, A^-).$$

Proof: As in the earlier proof, choose supporting hyperplanes $f^+(x) = w \cdot x - B^+ = 0$ and $f^-(x) = w \cdot x - B^-$ for A^+ and A^- . By our discussion above, these are also supporting hyperplanes for $C(A^+)$ and $C(A^-)$. Therefore if $p \in C(A^+)$ and $q \in C(A^-)$, we have $w \cdot p - B^+ \geq 0$ and $w \cdot q - B^- \leq 0$. As before

$$w \cdot (p - q) \geq B^+ - B^- > 0$$

and so

$$\|p - q\| \geq \frac{B^+ - B^-}{\|w\|} = \tau_w(A^+, A^-)$$

Since this holds for any w , we have the result for $\tau(A^+, A^-)$.

The reason this result is useful is that, as we've seen, if we restrict p and q to A^+ and A^- , then there can be a gap between the minimal distance and the optimal margin. If we allow p and q to range over the convex hulls of these sets, then that gap disappears.

One other consequence of this is that if A^+ and A^- are linearly separable then their convex hulls are disjoint.

Corollary: If A^+ and A^- are linearly separable then $\|p - q\| > 0$ for all $p \in C(A^+)$ and $q \in C(A^-)$

Proof: The sets are linearly separable precisely when $\tau > 0$.

Our strategy now is to show that if p and q are points in $C(A^+)$ and $C(A^-)$ respectively that are at minimal distance D , and if we set $w = p - q$, then we obtain supporting hyperplanes with margin equal to $\|p - q\|$. Since this margin is the *largest possible margin*, this w must be the optimal w . This transforms the problem of finding the optimal margin into the problem of finding the closest points in the convex hulls.

Lemma: Let

$$D = \min_{p \in C(A^+), q \in C(A^-)} \|p - q\|.$$

Then there are points $p^* \in C(A^+)$ and $q^* \in C(A^-)$ with $\|p^* - q^*\| = D$. If p_1^*, q_1^* and p_2^*, q_2^* are two pairs of points satisfying this condition, then $p_1^* - q_1^* = p_2^* - q_2^*$.

Proof: Consider the set of differences

$$V = \{p - q : p \in C(A^+), q \in C(A^-)\}.$$

- V is compact. This is because it is the image of the compact set $C(A^+) \times C(A^-)$ in $\mathbf{R}^k \times \mathbf{R}^k$ under the continuous map $h(x, y) = x - y$.
- the function $d(v) = \|v\|$ is continuous and satisfies $d(v) \geq D > 0$ for all $v \in V$.

Since d is a continuous function on a compact set, it attains its minimum D and so there is a $v = p^* - q^*$ with $d(v) = D$.

Now suppose that there are two distinct points $v_1 = p_1^* - q_1^*$ and $v_2 = p_2^* - q_2^*$ with $d(v_1) = d(v_2) = D$. Consider the line segment

$$t(s) = (1 - s)v_1 + sv_2 \text{ where } 0 \leq s \leq 1$$

joining v_1 and v_2 .

Now

$$t(s) = ((1 - s)p_1^* + sp_2^*) - ((1 - s)q_1^* + sq_2^*).$$

Both terms in this difference belong to $C(A^+)$ and $C(A^-)$ respectively, regardless of s , by convexity, and therefore $t(s)$ belongs to V for all $0 \leq s \leq 1$.

This little argument shows that V is convex. In geometric terms, v_1 and v_2 are two points in the set V equidistant from the origin and the segment joining them is a chord of a circle; as Figure 8.8 shows, in that situation there must be a point on the line segment joining them that's closer to the origin than they are. Since all the points on that segment are in V by convexity, this would contradict the assumption that v_1 is the closet point in V to the origin.

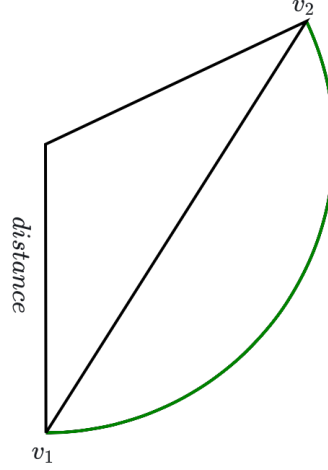


Figure 8.8: Chord of a circle

In algebraic terms, since D is the minimal value of $\|v\|$ for all $v \in V$, we must have $t(s) \geq D$. On the other hand

$$\frac{d}{ds} \|t(s)\|^2 = \frac{d}{ds} (t(s) \cdot t(s)) = t(s) \cdot \frac{dt(s)}{ds} = t(s) \cdot (v_2 - v_1).$$

Therefore

$$\frac{d}{ds} \|t(s)\|^2|_{s=0} = v_1 \cdot (v_2 - v_1) = v_1 \cdot v_2 - \|v_1\|^2 \leq 0$$

since $v_1 \cdot v_2 \leq D^2$ and $\|v_1\|^2 = D^2$. If $v_1 \cdot v_2 < D^2$, then this derivative would be negative, which would mean that there is a value of s where $t(s)$ would be less than D . Since that can't happen, we conclude that $v_1 \cdot v_2 = D^2$ which means that $v_1 = v_2$ — the vectors have the same magnitude D and are parallel. This establishes uniqueness.

Note: The essential ideas of this argument show that a compact convex set in $\mathbf{R}^k$ has a unique point closest to the origin. The convex set in this instance,

$$V = \{p - q : p \in C(A^+), q \in C(A^-)\},$$

is called the difference $C(A^+) - C(A^-)$, and it is generally true that the difference of convex sets is convex.

Now we can conclude this line of argument.

Theorem: Let p and q be points in $C(A^+)$ and $C(A^-)$ respectively are such that $\|p - q\|$ is minimal among all such pairs. Let $w = p - q$ and set $B^+ = w \cdot p$ and $B^- = w \cdot q$. Then $f^+(x) = w \cdot x - B^+ = 0$ and $f^-(x) = w \cdot x - B^-$ are supporting hyperplanes for $C(A^+)$ and $C(A^-)$ respectively and the associated margin

$$\tau_w(A^+, A^-) = \frac{B^+ - B^-}{\|w\|} = \|p - q\|$$

is optimal.

Proof: First we show that $f^+(x) = 0$ is a supporting hyperplane for $C(A^+)$. Suppose not. Then there is a point $p' \in C(A^+)$ such that $f^+(p') < 0$. Consider the line segment $t(s) = (1 - s)p + sp'$ running from p to p' . By convexity it is entirely contained in $C(A^+)$. Now look at the distance from points on this segment to q :

$$D(s) = \|t(s) - q\|^2.$$

We have

$$\left. \frac{dD(s)}{ds} \right|_{s=0} = 2(p - q) \cdot (p' - p) = 2w \cdot (p' - p) = 2[(f^+(p') + B^+) - (f^+(p) + B^+)]$$

so

$$\left. \frac{dD(s)}{ds} \right|_{s=0} = 2(f^+(p') - f^+(p)) < 0$$

since $f^+(p) = 0$. This means that $D(s)$ is decreasing along $t(s)$ and so there is a point s' along $t(s)$ where $\|t(s') - q\| < D$. This contradicts the fact that D is the minimal distance. The same argument shows that $f^-(x) = 0$ is also a supporting hyperplane.

Now the margin for this w is

$$\tau_w(A^+, A^-) = \frac{w \cdot (p - q)}{\|w\|} = \|p - q\| = D$$

and as w varies we know this is the largest possible τ that can occur. Thus this is the maximal margin.

*Figure 8.9 shows how considering the closest point in the convex hulls “fixes” the problem that we saw in Figure 8.5. The closest point occurs at a point on the boundary of the convex hull that is not one of the points in A^+ or A^- .

8.5 Finding the Optimal Margin Classifier

Now that we have translated our problem into geometry, we can attempt to develop an algorithm for solving it. To recap, we have two sets of points

$$A^+ = \{x_1^+, \dots, x_{n_+}^+\}$$

and

$$A^- = \{x_1^-, \dots, x_{n_-}^-\}$$

in $\mathbf{R}^k$ that are linearly separable.

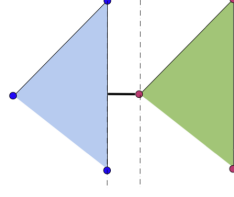


Figure 8.9: Closest distance between convex hulls gives optimal margin

We wish to find points $p \in C(A^+)$ and $q \in C(A^-)$ such that

$$\|p - q\| = \min_{p' \in C(A^+), q' \in C(A^-)} \|p' - q'\|.$$

Using the definition of the convex hull we can express this more concretely. Since $p \in C(A^+)$, there are coefficients $\lambda_i^+ \geq 0$ for $i = 1, \dots, n_+$ and $\lambda_i^- \geq 0$ for $i = 1, \dots, n_-$ so that

$$p = \sum_{i=1}^{n_+} \lambda_i^+ x_i^+$$

$$q = \sum_{i=1}^{n_-} \lambda_i^- x_i^-$$

where $\sum_{i=1}^{n_\pm} \lambda_i^\pm = 1$.

We can summarize this as follows:

Optimization Problem 1: Write $\lambda^\pm = (\lambda_1^\pm, \dots, \lambda_{n_\pm}^\pm)$. Define

$$w(\lambda^+, \lambda^-) = \sum_{i=1}^{n_+} \lambda_i^+ x_i^+ - \sum_{i=1}^{n_-} \lambda_i^- x_i^-$$

To find the supporting hyperplanes that define the optimal margin between A^+ and A^- , find λ^+ and λ^- such that $\|w(\lambda^+, \lambda^-)\|^2$ is minimal among all such w where all $\lambda_i^\pm \geq 0$ and $\sum_{i=1}^{n_\pm} \lambda_i^\pm = 1$.

This is an example of a *constrained optimization problem*. It's worth observing that the *objective function* $\|w(\lambda^+, \lambda^-)\|^2$ is just a quadratic function in the $\lambda^\pm$. Indeed we can expand

$$\|w(\lambda^+, \lambda^-)\|^2 = \left(\sum_{i=1}^{n_+} \lambda_i^+ x_i^+ - \sum_{i=1}^{n_-} \lambda_i^- x_i^- \right) \cdot \left(\sum_{i=1}^{n_+} \lambda_i^+ x_i^+ - \sum_{i=1}^{n_-} \lambda_i^- x_i^- \right)$$

to obtain

$$\|w(\lambda^+, \lambda^-)\|^2 = R - 2S + T$$

where

$$\begin{aligned}
R &= \sum_{i=1}^{n_+} \sum_{j=1}^{n_+} \lambda_i^+ \lambda_j^+ (x_i^+ \cdot x_j^+) \\
S &= \sum_{i=1}^{n_+} \sum_{j=1}^{n_-} \lambda_i^+ \lambda_j^- (x_i^+ \cdot x_j^-) \\
T &= \sum_{i=1}^{n_-} \sum_{j=1}^{n_-} \lambda_i^- \lambda_j^- (x_i^- \cdot x_j^-)
\end{aligned} \tag{8.2}$$

Thus the function we are trying to minimize is relatively simple.

On the other hand, unlike optimization problems we have seen earlier in these lectures, in which we can apply Lagrange multipliers, in this case some of the constraints are inequalities – namely the requirement that all of the $\lambda^\pm \geq 0$ – rather than equalities. There is an extensive theory of such problems that derives from the idea of Lagrange multipliers. However, in these notes, we will not dive into that theory but will instead construct an algorithm for solving the problem directly.

8.5.1 Relaxing the constraints

Our first step in attacking this problem is to adjust our constraints and our objective function slightly so that the problem becomes easier to attack.

Optimization Problem 2: This is a slight revision of problem 1 above. We minimize:

$$Q(\lambda^+, \lambda^-) = \|w(\lambda^+, \lambda^-)\|^2 - \sum_{i=1}^{n_+} \lambda_i^+ - \sum_{i=1}^{n_-} \lambda_i^-$$

subject to the constraints that all $\lambda_i^\pm \geq 0$ and

$$\alpha = \sum_{i=1}^{n_+} \lambda_i^+ = \sum_{i=1}^{n_-} \lambda_i^-.$$

Problem 2 is like problem 1, except we don't require the sums of the $\lambda_i^\pm$ to be one, but only that they be equal to each other; and we modify the objective function slightly. It turns out that the solution to this optimization problem easily yields the solution to our original one.

Lemma: Suppose λ^+ and λ^- satisfy the constraints of problem 2 and yield the minimal value for the objective function $Q(\lambda^+, \lambda^-)$. Then $\alpha \neq 0$. Rescale the $\lambda^\pm$ to have sum equal to one by dividing by α , yielding $\tau^\pm = (1/\alpha)\lambda^\pm$. Then $w(\tau^+, \tau^-)$ is a solution to optimization problem 1.

Proof: To show that $\alpha \neq 0$, suppose that $\lambda_i^\pm = 0$ for all $i \neq 1$ and $\lambda = \lambda_1^+ = \lambda_1^-$. The one-variable quadratic function $Q(\lambda)$ takes its minimum value at $\lambda = 1/\|x_1^+ - x_1^-\|^2$ and its value at that point is negative. Therefore the minimum value of Q is negative, which means $\alpha \neq 0$ at that minimum point.

For the equivalence, notice that $\tau^\pm$ still satisfy the constraints of problem 2. Therefore

$$Q(\lambda^+, \lambda^-) = \|w(\lambda^+, \lambda^-)\|^2 - 2\alpha \leq \|w(\tau^+, \tau^-)\|^2 - 2.$$

On the other hand, suppose that $\sigma^\pm$ are a solution to problem 1. Then

$$\|w(\sigma^+, \sigma^-)\|^2 \leq \|w(\tau^+, \tau^-)\|^2.$$

Therefore

$$\alpha^2 \|w(\sigma^+, \sigma^-)\|^2 = \|w(\alpha\sigma^+, \alpha\sigma^-)\|^2 \leq \|w(\lambda^+, \lambda^-)\|^2$$

and finally

$$\|w(\alpha\sigma^+, \alpha\sigma^-)\|^2 - 2\alpha \leq Q(\lambda^+, \lambda^-) = \|w(\alpha\tau^+, \alpha\tau^-)\|^2 - 2\alpha.$$

Since Q is the minimal value, we have

$$\alpha^2 \|w(\sigma^+, \sigma^-)\|^2 = \alpha^2 \|w(\tau^+, \tau^-)\|^2$$

so that indeed $w(\tau^+, \tau^-)$ gives a solution to Problem 1.

8.5.2 Sequential Minimal Optimization

Now we outline an algorithm for solving Problem 2 that is called Sequential Minimal Optimization that was introduced by John Platt in 1998 (See [10] and Chapter 12 of [11]). The algorithm is based on the principle of “gradient ascent”, where we exploit the fact that the negative gradient of a function points in the direction of its most rapid decrease and we take small steps in the direction of the negative gradient until we reach the minimum.

However, in this case simplify this idea a little. Recall that the objective function $Q(\lambda^+, \lambda^-)$ is a quadratic function in the λ 's and that we need to preserve the condition that $\sum \lambda_i^+ = \sum \lambda_i^-$. So our approach is going to be to take, one at a time, a pair λ_i^+ and λ_j^- and change them *together* so that the equality of the sums is preserved and the change reduces the value of the objective function. Iterating this will take us to a minimum.

So, for example, let's look at λ_i^+ and λ_j^- and, for the moment, think of all of the other λ 's as constants. Then our objective function reduces to a quadratic function of these two variables that looks something like:

$$Q(\lambda_i^+, \lambda_j^-) = a(\lambda_i^+)^2 + b\lambda_i^+\lambda_j^- + c(\lambda_j^-)^2 + d\lambda_i^+ + e\lambda_j^- + f.$$

The constraints that remain are $\lambda^\pm \geq 0$, and we are going to try to minimize Q by changing λ_i^+ and λ_j^- *by the same amount* δ . Furthermore, since we still must have $\lambda_i^+ + \delta \geq 0$ and $\lambda_j^- + \delta \geq 0$, we have

$$\delta \geq M = \max\{-\lambda_i^+, -\lambda_j^-\} \quad (8.3)$$

In terms of this single variable δ , our optimization problem becomes the job of finding the minimum of a quadratic polynomial in one variable subject to the constraint in Equation 8.3. This is easy! There are two cases: the critical point of the quadratic is to the left of M , in which case the minimum value occurs at M ; or the critical point of the quadratic is to the right of M , in which case the critical point occurs there. This is illustrated in Figure 8.10.

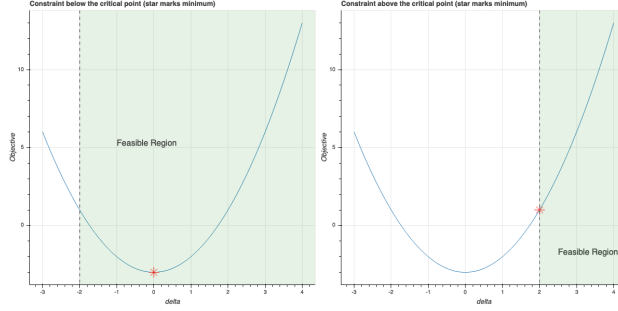


Figure 8.10: Minimizing the 1-variable quadratic objective function

Computationally, let's write

$$w_{\delta,i,j}(\lambda^+, \lambda^-) = w(\lambda^+, \lambda^-) + \delta(x_i^+ - x_j^-).$$

Then

$$\frac{d}{d\delta}(\|w_{\delta,i,j}(\lambda^+, \lambda^-)\|^2 - 2\alpha) = 2w_{\delta,i,j}(\lambda^+, \lambda^-) \cdot (x_i^+ - x_j^-) - 2$$

and using the definition of $w_{\delta,i,j}$ we obtain the following formula for the critical value of δ by setting this derivative to zero:

$$\delta_{i,j} = \frac{(1 - w(\lambda^+, \lambda^-) \cdot (x_i^+ - x_j^-))}{\|x_i^+ - x_j^-\|^2}$$

Using this information we can describe the SMO algorithm.

Algorithm (SMO, see [10]):

Given: Two linearly separable sets of points $A^+ = \{x_1^+, \dots, x_{n_+}^+\}$ and $A^- = \{x_1^-, \dots, x_{n_-}^-\}$ in $\mathbf{R}^k$.

Find: Points p and q belonging to $C(A^+)$ and $C(A^-)$ respectively such that

$$\|p - q\|^2 = \min_{p' \in C(A^+), q' \in C(A^-)} \|p' - q'\|^2$$

Initialization: Set $\lambda_i^+ = \frac{1}{n_+}$ for $i = 1, \dots, n_+$ and $\lambda_i^- = \frac{1}{n_-}$ for $i = 1, \dots, n_-$. Set

$$p(\lambda^+) = \sum_{i=1}^{n_+} \lambda_i^+ x_i^+$$

and

$$q(\lambda^-) = \sum_{i=1}^{n_-} \lambda_i^- x_i^-$$

Notice that $w(\lambda^+, \lambda^-) = p(\lambda^+) - q(\lambda^-)$. Let $\alpha = \sum_{i=1}^{n_+} \lambda_i^+ = \sum_{i=1}^{n_-} \lambda_i^-$. These sums will remain equal to each other throughout the operation of the algorithm.

Repeat the following steps until maximum value of δ^* computed in each iteration is smaller than some tolerance (so that the change in all of the λ 's is very small):

- For each pair i, j with $1 \leq i \leq n_+$ and $1 \leq j \leq n_-$, compute

$$M_{i,j} = \max\{-\lambda_i^+, -\lambda_j^-\}$$

and

$$\delta_{i,j} = \frac{1 - (p(\lambda^+) - q(\lambda^-)) \cdot (x_i^+ - x_j^-)}{\|x_i^+ - x_j^-\|^2}.$$

If $\delta_{i,j} \geq M$ then set $\delta^* = \delta_{i,j}$; otherwise set $\delta^* = M$. Then update the $\lambda^\pm$ by the equations:

$$\begin{aligned}\lambda_i^+ &= \lambda_i^+ + \delta_{i,j}^* \\ \lambda_j^- &= \lambda_j^- + \delta_{i,j}^*\end{aligned}$$

When this algorithm finishes, $p \approx p(\lambda^+)$ and $q \approx q(\lambda^-)$ will be very good approximations to the desired closest points.

Recall that if we set $w = p - q$, then the optimal margin classifier is

$$f(x) = w \cdot x - \frac{B^+ + B^-}{2} = 0$$

where $B^+ = w \cdot p$ and $B^- = w \cdot q$. Since $w = p - q$ we can simplify this to obtain

$$f(x) = (p - q) \cdot x - \frac{\|p\|^2 - \|q\|^2}{2} = 0.$$

In Figure 8.11, we show the result of applying this algorithm to the penguin data and illustrate the closest points as found by an implementation of the SMO algorithm, together with the optimal classifying line.

Bearing in mind that the y-axis is scaled by a factor of 200, we obtain the following rule for distinguishing between Adelie and Gentoo penguins – if the culmen depth and body mass put you above the red line, you are a Gentoo penguin, otherwise you are an Adelie.

8.6 Inseparable Sets

Not surprisingly, real life is often more complicated than the penguin example we've discussed at length in these notes. In particular, sometimes we have to work with sets that are not linearly separable. Instead, we might have two point clouds, the bulk of which are separable, but because of some outliers there is no hyperplane we can draw that separates the two sets into two halfplanes.

Fortunately, all is not lost. There are two common ways to address this problem, and while we won't take the time to develop the theory behind them, we can at least outline how they work.

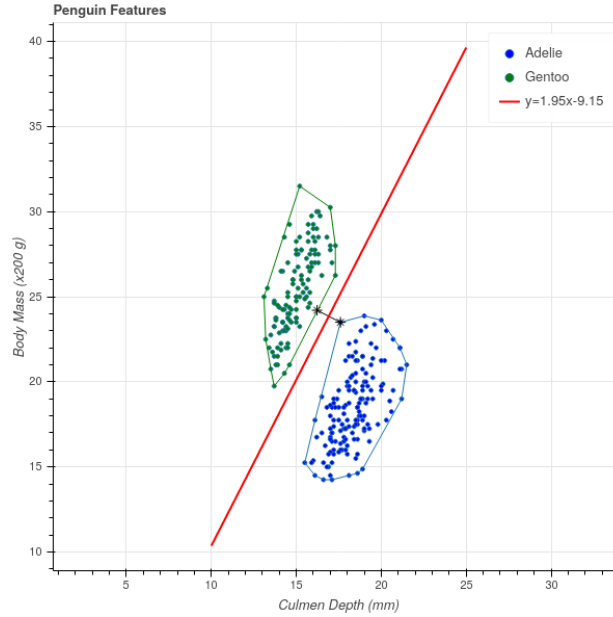


Figure 8.11: Closest points in convex hulls of penguin data

8.6.1 Best Separating Hyperplanes

If our sets are not linearly separable, then their convex hulls overlap and so our technique for finding the closest points of the convex hulls won't work. In this case, we can “shrink” the convex hull by considering combinations of points $\sum_i \lambda_i x_i$ where $\sum \lambda_i = 1$ and $C \geq \lambda_i \geq 0$ for some $C \leq 1$. For C small enough, reduced convex hulls will be linearly separable – although some outlier points from each class will lie outside of them – and we can find hyperplane that separates the reduced hulls.

In practice, this means we allow a few points to lie on the “wrong side” of the hyperplane. Our tolerance for these mistakes depends on C , but we can include C in the optimization problem to try to find the smallest C that “works”.

8.6.2 Nonlinear kernels

The second option is to look not for separating hyperplanes but instead for separating curves – perhaps polynomials or even more exotic curves. This can be achieved by taking advantage of the form of Equation 8.2. As you see there, the only way the points $x_i^\pm$ enter in to the function being minimized is through the inner products $x_i^\pm \cdot x_j^\pm$. We can adopt a different inner product than the usual Euclidean one, and reconsider the problem using this different inner product. This amounts to embedding our points in a higher dimensional space where they are more likely to be linearly separable. Again, we will not pursue the mathematics of this further in these notes.

8.7 Exercises

1. Prove that, if $f(x) = w \cdot x + b = 0$ is a hyperplane in $\mathbf{R}^k$, then the two “sides” of this hyperplane, consisting of the points where $f(x) \geq 0$ and $f(x) \leq 0$, are both convex sets.
2. Prove that $C(S)$ is the intersection of all the halfplanes $f(x) \geq 0$ as $f(x) = w \cdot x + b$ runs through all supporting hyperplanes for S where $f(x) \geq 0$ for all $p \in S$.
3. Prove that $C(S)$ is bounded. Hint: show that S is contained in a sphere of sufficiently large radius centered at zero, and then that $C(S)$ is contained in that sphere as well.
4. Confirm the final formula for the optimal margin classifier at the end of the lecture.

9 Neural Networks

9.1 Introduction

Many of the most impressive achievements in machine learning stem from the development of “artificial neural networks.” The earliest ideas for building algorithms based on models of neurons go back to the 1940s and peaked in the late 1950s and early 1960s with the development of the “perceptron,” an early example of what we now call a multi-layer neural net. Hardware limitations, along with a tendency to overstate the power of these early techniques, led to the abandonment of these ideas for nearly 50 years until Geoffrey Hinton and others returned to them, supported by dramatic improvements in computing power, specialized hardware such as GPUs, and several crucial advances in optimization algorithms. Since then neural networks have demonstrated an amazing ability to “learn” and have overcome challenges in image recognition and other classic problems in artificial intelligence, culminating in the invention of the attention mechanism, the transformer, and the modern LLM.

9.2 Basics of Neural Networks

At its heart, a neural network is a function F that is built out of two simple components:

- linear maps (matrices)
- simple non-linear maps (activation functions).

The function F is the composition of these types of functions so that $F(x) = \cdots \sigma_2 \circ M_2 \circ \sigma_1 \circ M_1$.

The structure of the different layers, the shape of the matrices M_i , and the choice of non-linear transition functions make up the *architecture* of the neural network.

Suppose we want to build a neural network that recognizes pictures of cats. We begin with a large collection of images (a training set) x_i and labels y_i indicating whether x_i is a “cat” (1) or “not-cat” (0). Our goal is to learn a function F that assigns to each image a probability between 0 and 1 measuring how likely it is that the picture is a cat.

To do this, we choose an architecture for our neural network F and initialize its weight matrices randomly. Then we compute $F(x_i)$ and compare it to y_i . Using an optimization algorithm, we adjust the weights until $F(x_i)$ is close to 1 when $y_i = 1$ and close to 0 when $y_i = 0$. Eventually, we obtain a function that, hopefully, can recognize images that were not in its training set, assigning high probabilities to pictures of cats and low probabilities to pictures of non-cats.

In fact, both linear and logistic regression fit into this framework and are very simple examples of neural networks. In the case of linear regression, we have training data (x_i, y_i) and our goal

is to find a matrix M so that the function $Y = MX$ is a good approximation to the relationship $y_i \approx Mx_i$. The error is measured by the mean squared error, and we can find M either analytically or by gradient descent. In this case the neural network is purely linear, with no nonlinear maps involved.

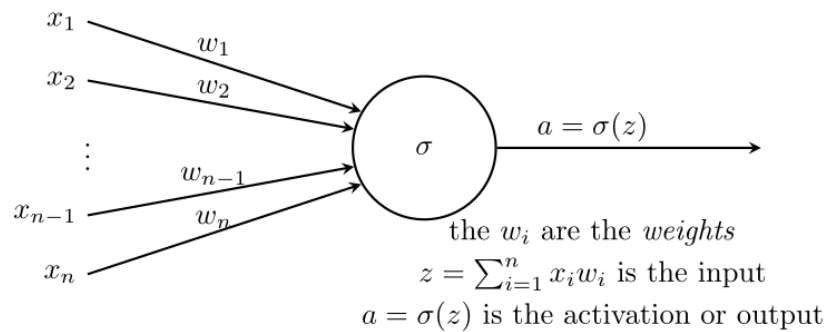
In the case of (simple) logistic regression, we have a collection of data (x_i, y_i) where y_i is 0 or 1 and the x_i are vectors of length n . Here we use the logistic function σ together with a weight vector $w \in \mathbb{R}^n$ and consider $F(x_i) = \sigma(w \cdot x_i)$. The error we measure is the negative log-likelihood of the data given the probabilities $F(x_i)$, which works out to

$$L = - \sum_i (y_i \log F(x_i) + (1 - y_i) \log(1 - F(x_i)))$$

and we find w by iteratively minimizing this L .

9.3 Graphical Representation of Neural Networks

It is traditional, when working with neural networks, to take advantage of a graphical representation of the structure of the network. Figure 9.1 shows the fundamental element of such a graphical representation—a single “neuron.” Here, the inputs x_i flow into the neuron along the arrows from the left, where they are multiplied by the weights w_i . Then these values $x_i w_i$ are summed, yielding $z = \sum_i x_i w_i$, and the nonlinear “activation” function σ is applied; the result is $a = \sigma(z)$.



$$\sigma(x) = \frac{1}{1+e^{-x}} \text{ or } \sigma(t) = \tanh(x) \text{ or } \sigma(t) = \begin{cases} t & t \geq 0 \\ 0 & t < 0 \end{cases} \text{ or others}$$

Figure 9.1: A single neuron

A full-fledged neural network is built up from “layers.” Each layer consists of a collection of neurons with input connections from the previous layer and output connections to the next layer. This structure is illustrated in Figure 9.2.

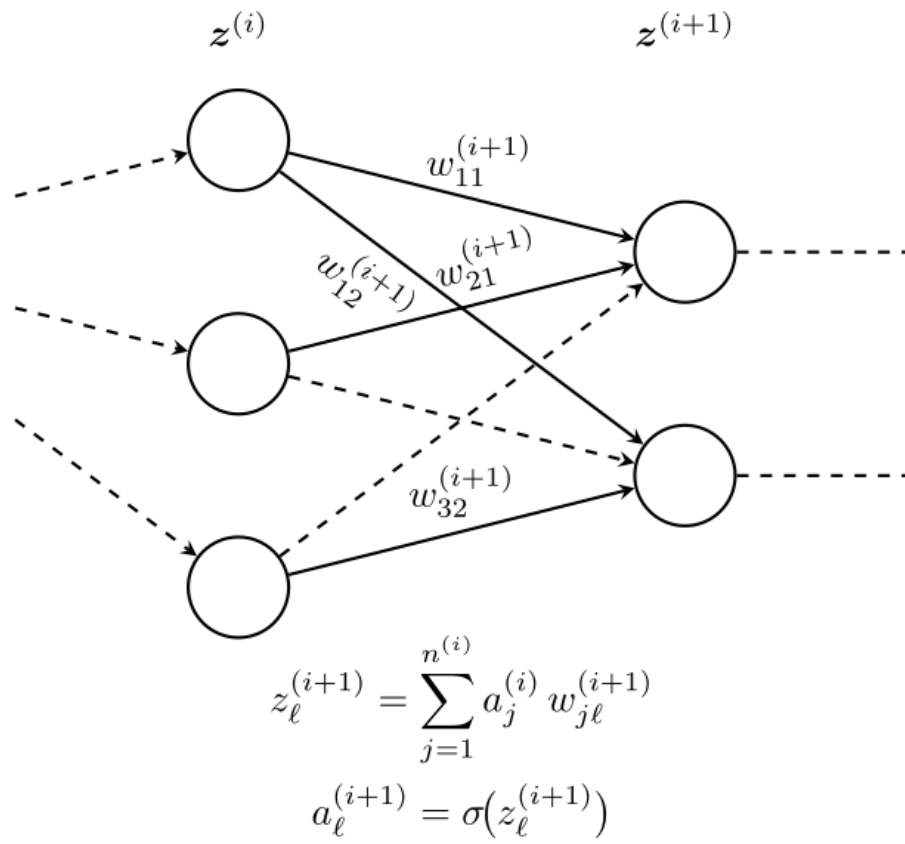
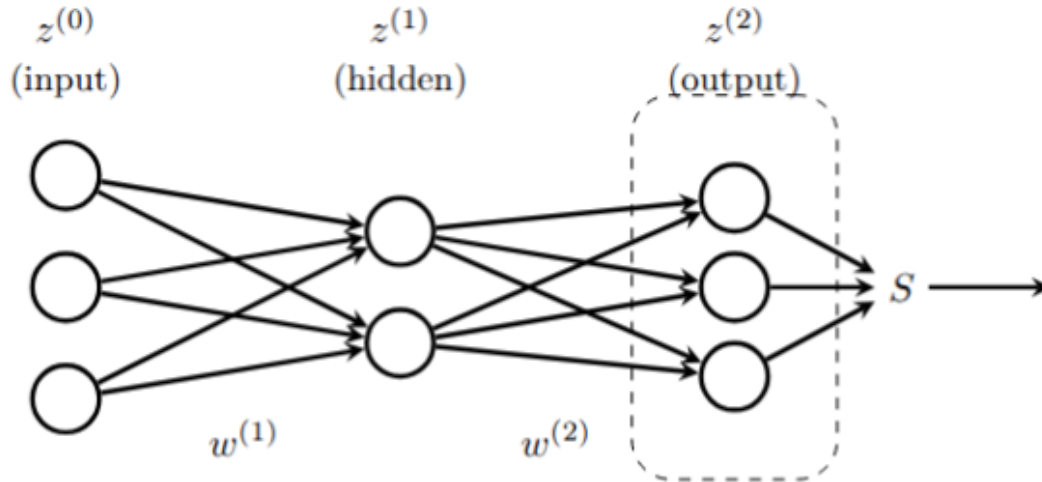


Figure 9.2: Layers

A multi-layer neural network with specified weights and activation functions defines a function called the “inference” or “feed-forward” function. Consider the simple example shown in Figure 9.3.



$z^{(0)}$	1×3 matrix
$w^{(1)}$	3×2 matrix
$w^{(2)}$	2×3 matrix

Figure 9.3: A Simple Network

The input layer has 3 components, which we can represent as a 1×3 row vector with entries $(z_1^{(0)}, z_2^{(0)}, z_3^{(0)})$. The middle “hidden layer” has two nodes. The 6 weights $w_{ij}^{(1)}$ connecting node $z_i^{(0)}$ to $z_j^{(1)}$ form a 3×2 matrix $W^{(1)}$ where

$$z_j^{(1)} = \sum_{i=1}^3 z_i^{(0)} w_{ij}^{(1)}.$$

The outputs of the hidden layer are obtained by applying the activation function σ to each of the $z_j^{(1)}$:

$$a_j^{(1)} = \sigma(z_j^{(1)})$$

for $j = 1, 2$. These activations are the inputs to the final layer, which has 3 nodes. The weights $w_{jk}^{(2)}$ connecting the hidden layer to the output layer form a 2×3 matrix $W^{(2)}$, where

$$z_k^{(2)} = \sum_{j=1}^2 a_j^{(1)} w_{jk}^{(2)}.$$

The last step is to apply an output function to the vector $z^{(2)}$. While activation functions are typically applied element-wise, the output function often uses all of the values in the layer.

Putting this together, the feed-forward function F looks like

$$F(z^{(0)}) = S(\sigma(z^{(0)}W^{(1)})W^{(2)})$$

Here, the function σ is applied element-wise to the vector $z^{(0)}W^{(1)}$, while the output function S is applied to the entire vector $z^{(2)}$.

9.3.0.1 Linear Regression as a Neural Network

A simple linear regression problem takes an N -dimensional input vector x (which we write as a $1 \times N$ row vector) and produces an M -dimensional output vector y (which we write as a $1 \times M$ row vector) by multiplying by a weight matrix W so that $y = xW$.

This is a neural network with no activation function, just a single layer with weight matrix W and the identity as the output function.

9.3.0.2 Logistic Regression as a Neural Network

While linear regression can be represented as a trivial neural network, logistic regression is a more interesting example. Consider the problem of multi-class logistic regression, where the input vector is an N -dimensional row vector x and the output is a probability distribution over M classes, represented as an M -dimensional row vector y with non-negative entries summing to one.

From our earlier work, we know that this model relies on an $N \times M$ weight matrix, and the output of the logistic model is $F(x) = S(xW)$, where S is the softmax function defined by

$$S(z)_i = \frac{e^{z_i}}{\sum_{j=1}^M e^{z_j}}.$$

The graphical representation of this neural network is shown in Figure 9.5.

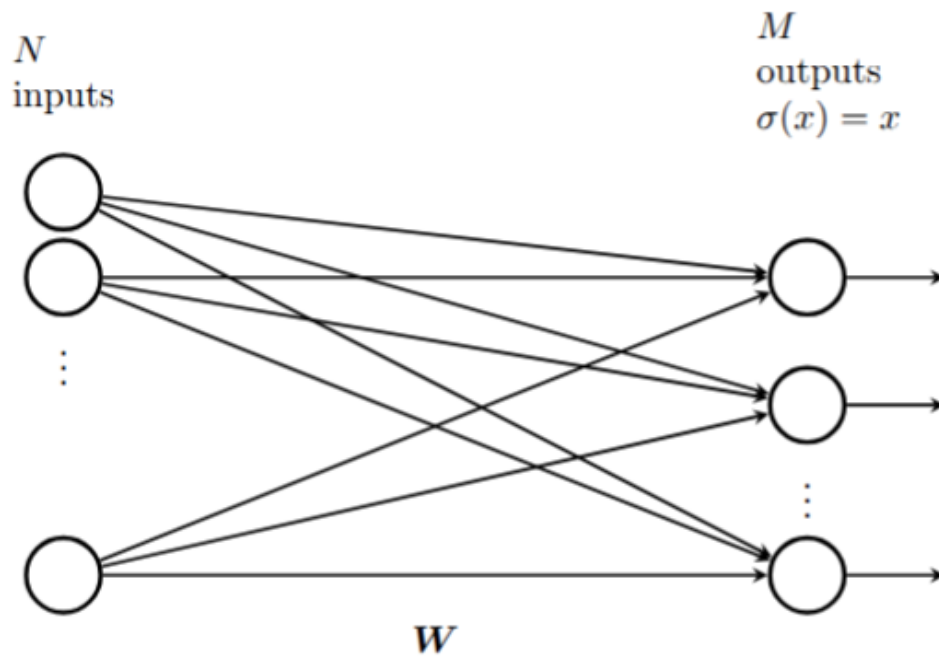


Figure 9.4: Linear Regression as a Neural Network

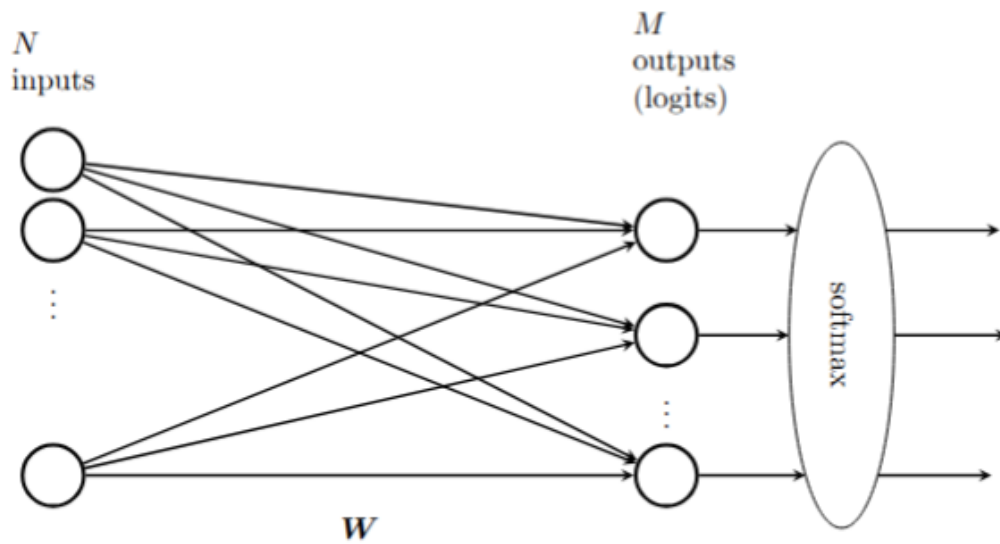


Figure 9.5: Logistic Regression as a Neural Network

9.4 Loss functions and training

Given a large collection of data $(x^{[i]}, y^{[i]})$, where x is the input and y the target output, the goal of training a neural network F_W on this data is to adjust the weights W in F_W so that $F(x^{[i]})$ is “approximately” $y^{[i]}$. To make sense of this, we need to quantify how close $F(x^{[i]})$ is to $y^{[i]}$ by means of a function L_W called the “loss function.”

The choice of a loss function is based on the ultimate purpose of the neural network. We have seen two widely used examples. The first, which arises in linear regression, is the “mean squared error”. On a single data point $(x^{[i]}, y^{[i]})$, this loss function is

$$L_W(F_W(x^{[i]}), y^{[i]}) = \|F_W(x^{[i]}) - y^{[i]}\|^2$$

and the overall loss is

$$L_W = \frac{1}{M} \sum_{i=1}^M \|F_W(x^{[i]}) - y^{[i]}\|^2.$$

In the case of multi-class classification (with, say, n classes), the loss function is usually the “cross entropy”. In this case the output vectors $y^{[i]}$ are $(y_1^{[i]}, \dots, y_n^{[i]})$ where the $y_j^{[i]}$ are all zero except for a 1 in the j th position where the proper class assignment is class j . (This is called one-hot encoding). The output layer of a classification network consists of $(z_1, \dots, z_n)$ which are passed through the softmax function yielding

$$\left(\frac{e^{z_1}}{H}, \dots, \frac{e^{z_n}}{H}\right)$$

where $H = \sum_{j=1}^n e^{z_j}$. If we write $p_j = e^{z_j}/H$, then the loss for a single data point $(x^{[i]}, y^{[i]})$ is

$$L_W(x^{[i]}, y^{[i]}) = - \sum_{j=1}^n y_j^{[i]} \log(p_j)$$

and the total loss would be

$$L_W = \frac{1}{M} \sum_{i=1}^M L_W(x^{[i]}, y^{[i]})$$

It is important to recognize that the loss *is a function of the weights* of the network; the data (x, y) are fixed.

The goal of training is to minimize L_W by varying W . From a mathematical point of view, we do this by gradient descent. That is to say, for the fixed collection of data, we iteratively calculate $\partial L_W / \partial W_{kl}^{(j)}$ for all the weights $W_{kl}^{(j)}$ in the network, and then make a small adjustment

$$W_{kl}^{(j)} = W_{kl}^{(j)} - \lambda \frac{\partial L_W}{\partial W_{kl}^{(j)}}$$

until the loss function changes by less than some threshold amount on each iteration.

Computing the partial derivatives $\partial L_W / \partial W_{kl}^{(j)}$ is a miraculous application of the chain rule that exploits the architecture of the neural network. The algorithm for this computation is called “backpropagation” and we will discuss it in the next section.

9.5 Backpropagation

Our neural network is made up of n layers, with the output of the final n th layer serving as input to the loss function. The nodes at the j th layer have values $z_k^{(j)}$. The idea behind backpropagation is, for each data point $(x^{[i]}, y^{[i]})$, to compute vectors

$$\delta_k^{(j)} = \frac{\partial L_W}{\partial z_k^{(j)}}$$

inductively starting with $j = n$ and working backwards. From these $\delta^{(j)}$ we can then compute the partials $\partial L_W / \partial W_{rs}^{(j)}$.

Since the total loss is accumulated as a sum over the data points, we can sum the $\delta_k^{(j)}$ obtained from successive data points to compute the gradient of the loss over all the data or over some portion of the data, and use these accumulated gradients to adjust the weights.

The term “backpropagation” comes from the fact that we compute the δ starting at the output end of the network and working backwards toward the input end, in contrast with the inference or forward pass which goes from the input to the output.

9.5.1 Backpropagation: first step

The first step of the backpropagation algorithm comes from the output layer of the neural network. The elements of the last layer $z^{(n)}$ are fed directly into the loss function L_W as shown in Figure 9.6. Therefore,

$$\delta_j^{(n)} = \frac{\partial L_W}{\partial z_j^{(n)}}$$

depends on the loss function L_W . Let’s look at the two cases we considered above separately.

9.5.1.1 Mean Squared Error

In this case, as we’ve seen, the loss function is given by the squared Euclidean distance between the output vector $z^{(n)}$ and the target vector $y^{[i]}$.

$$L_W(x^{[i]}, y^{[i]}) = \|z^{(n)} - y^{[i]}\|^2$$

and therefore the derivative is just

$$\frac{\partial L_W}{\partial z_j^{(n)}} = 2(z_j^{(n)} - y_j^{[i]}).$$

In other words, since we are going to adjust the scaling anyway when we do gradient descent, we can set

$$\delta^{(n)} = z^{(n)} - y^{[i]}.$$

9.5.1.2 Cross Entropy

In the classification problem we saw that

$$L_W(x^{[i]}, y^{[i]}) = - \sum_j y_j^{[i]} \log \frac{e^{z_j^{(n)}}}{H}$$

where $H = \sum_j e^{z_j^{(n)}}$. Therefore

$$\delta_j^{(n)} = \frac{\partial L_W}{\partial z_j^{(n)}} = -y_j^{[i]} + \frac{\partial \log H}{\partial z_j^{(n)}}$$

which gives

$$\delta_j^{(n)} = -y_j^{[i]} + p_j$$

where

$$p_j = \frac{e^{z_j^{(n)}}}{H}$$

Putting this together yields

$$\delta^{(n)} = -y^{[i]} + p$$

where p is the vector of probabilities with entries $p_j = e^{z_j^{(n)}}/H$.

9.5.2 Backpropagation - inductive steps

To continue the backpropagation steps, we are going to compute $\delta^{(i-1)}$ from $\delta^{(i)}$ for $i < n$ inductively. Again, the key is the chain rule:

$$\delta_k^{(i-1)} = \frac{\partial L_W}{\partial z_k^{(i-1)}} = \sum_t \frac{\partial L_W}{\partial z_t^{(i)}} \frac{\partial z_t^{(i)}}{\partial z_k^{(i-1)}}$$

The structure of the neural network tells us that

$$z_t^{(i)} = \sum_j \sigma(z_j^{(i-1)}) w_{jt}^{(i)}. \quad (9.1)$$

Therefore

$$\frac{\partial z_t^{(i)}}{\partial z_k^{(i-1)}} = \sigma'(z_k^{(i-1)}) w_{kt}^{(i)}.$$

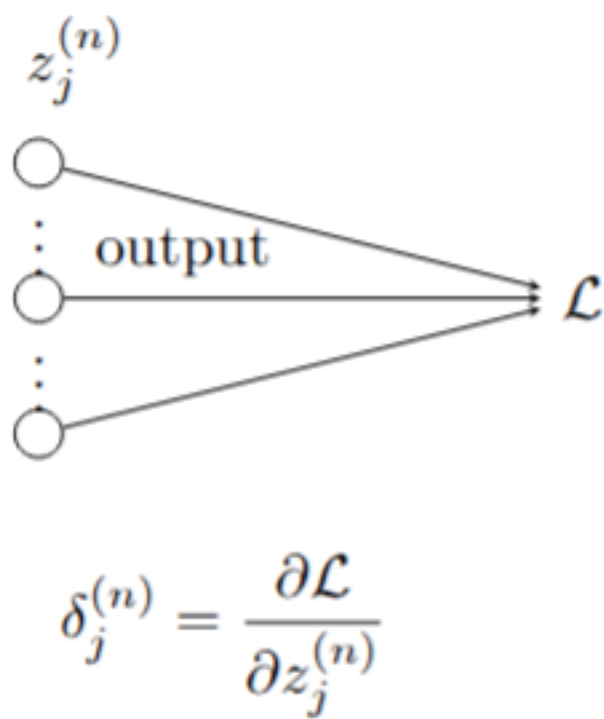


Figure 9.6: Final Layer with Loss

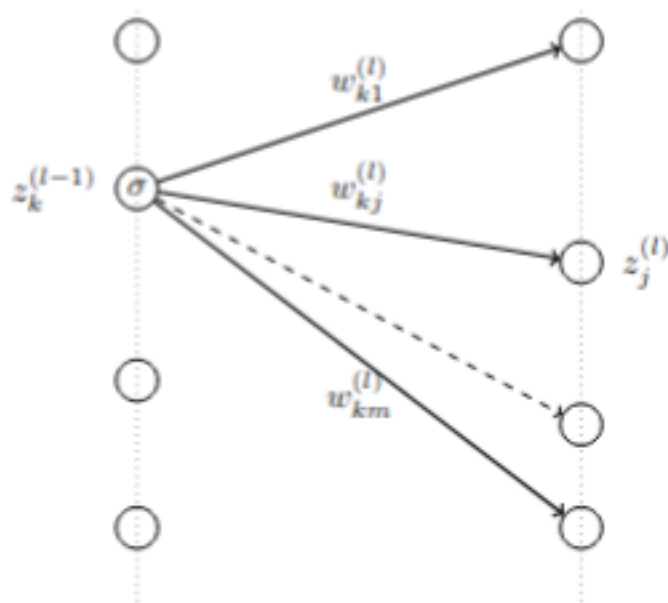


Figure 9.7: Backpropagation – interior

Putting this in the chain rule sum yields

$$\frac{\partial L_W}{\partial z_k^{(i-1)}} = \sum_t \frac{\partial L_W}{\partial z_t^{(i)}} \sigma'(z_k^{(i-1)}) w_{kt}^{(i)}.$$

In other words

$$\delta_k^{(i-1)} = \sigma'(z_k^{(i-1)}) w_{kt}^{(i)} \delta_t^{(i)}$$

or, in vector form,

$$\delta^{(i-1)} = \sigma'(z^{(i-1)}) W^{(i)} \delta^{(i)}$$

where the multiplication by the vector $(\sigma'(z_j^{(i-1)}))$ is done componentwise.

9.5.3 Backpropagation - putting it together

As we've seen, by a backward pass through the network we can compute the $\delta^{(i)}$, which are essentially the gradients of L_W with respect to the variables making up the i th layer.

Of course, what we *really* want are the gradients with respect to the weights, and that requires one more use of the chain rule. We see that

$$\frac{\partial L_W}{\partial w_{jk}^{(i)}} = \sum \frac{\partial L_W}{\partial z_k^{(i)}} \frac{\partial z_k^{(i)}}{\partial w_{jk}^{(i)}}$$

and from Equation 9.1 we have

$$\frac{\partial z_k^{(i)}}{\partial w_{jk}^{(i)}} = \sigma(z_j^{(i-1)}).$$

As a result,

$$\frac{\partial L_W}{\partial w_{jk}^{(i)}} = \delta_k^{(i)} \sigma(z_j^{(i-1)}).$$

The gradient $\nabla W^{(i)}$ of $W^{(i)}$ is therefore a matrix whose jk entry is given by

$$\frac{\partial L_W}{\partial w_{jk}^{(i)}} = \sigma(z_j^{(i-1)}) \delta_k^{(i)}.$$

This is sometimes called the outer product of the vectors $\sigma(z^{(i-1)})$ and $\delta^{(i)}$ or just the matrix product of the column vector $\delta^{(i)}$ and the row vector $\sigma(z^{(i-1)})$ (in that order).

9.5.4 Training

The training process for a neural network uses successive forward (inference) passes and backward (backpropagation) passes to iteratively move the weights of the network in a way that reduces the loss function.

More specifically, begin by setting all the $\delta^{(i)}$ and $\nabla W^{(i)}$ to zero, and initialize the weights of the network randomly. Then, for each (x, y) in the dataset:

- make a forward pass through the network, computing and saving the inputs $z^{(i)}$ for each layer.
- make a backward pass through the network, computing the $\delta^{(i)}$ and the $\nabla W^{(i)}$ at each layer using the backpropagation formulae. Accumulate the $\delta^{(i)}$ and $\nabla W^{(i)}$ from each pass into variables $\delta_*^{(i)}$ and $\nabla W_*^{(i)}$.
- periodically (after each iteration, after a fixed number of data points, or after a complete pass through the data), update the weights using your chosen version of gradient descent; the simplest approach is:

$$W^{(i)} \leftarrow W^{(i)} - \lambda \nabla W_*^{(i)}$$

for a learning rate λ . Then reset all of the accumulators $\delta_*^{(i)}$ and $\nabla W_*^{(i)}$ to zero.

Once every data point in the dataset has been considered, you have completed one *training epoch*. Repeat until the loss stops decreasing meaningfully.

9.6 Convolution

In this section, we will look at convolution, an important idea in mathematics generally that plays a key role in neural networks, especially in image recognition. Before looking at its application there we will first talk about it more generally.

Let's start with a function $f(t)$ of one variable that represents some underlying trend together with random fluctuations. For example, $f(t)$ might be the price of the stock of a company that is overall growing but that bounces up and down in response to short-term events. We show a sample graph of such a function in Figure 9.8 .

We are interested in identifying the underlying trend, and filtering out the noise. One standard method for doing this is to use a “moving average” – that is, to replace each value $f(t)$ by the average of the values of f in a window near t . More formally we choose a step size δ and a “window size” k and let

$$\hat{f}(t) = \frac{1}{2k+1} \sum_{i=-k}^k f(t+i\delta)$$

We have to make some decisions here about what to do at the beginning and end of the domain of f , but this is mainly a detail so let's ignore this problem for now. In many applications, we have f given to us as a discrete series of numbers, such as the daily prices of a stock, and in that case the moving average is just the average over some number of days around the given day.

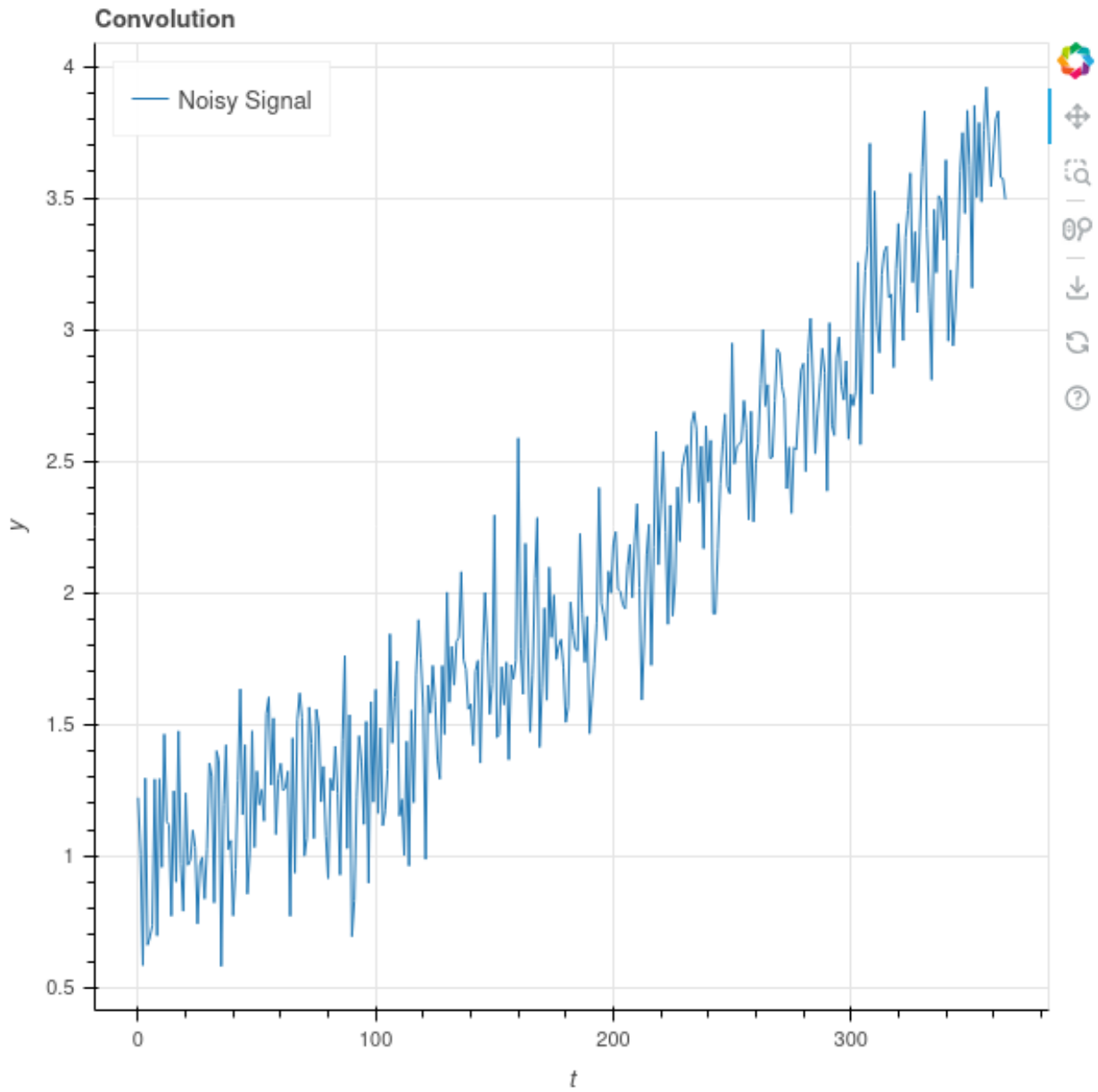


Figure 9.8: A Noisy Signal

The effect of the moving average is, in general, to shrink the random variability in the signal, since the average of a random signal will, in general, have lower variance than the original random signal.

In our graph above, if we take $\delta = 1$ and $k = 5$, we get the result shown in Figure 9.9 . As you can see, the red line bounces around much less than the blue one.

This idea of a “moving average” isn’t limited to one dimensional data; it can be applied to data in any dimension. For example, suppose our data is an image represented by a 28×28 array of numbers between 0 and 255. Each array cell is a pixel, and the number is how dark that pixel is, from white (255) to black (0).

Figure 9.10 is such an image.

In this situation, our “moving average” can mean replacing each cell by the average value of that cell and its 8 neighbors. For boundary cells, we need to make some kind of choice – for example, we can compute the average of neighboring cells that lie within the 28×28 array. Applying this to the image above “smooths” out the image. We can see the effect of this in Figure 9.11 .

Convolution is essentially a generalization of the moving average, where replace the simple average with an arbitrary weighted average. That is, we take another function $h(t)$, which we think of as a “filter”, and we use the values of h to weight the values of f in our average. The transform of f under this type of convolution is written $h \star f$, and in the general continuous case it’s an integral:

$$(h \star f)(t) = \int_{s=-\infty}^{\infty} \sum h(s)f(t-s)dt$$

In machine learning, we have discrete data, and so we can simplify this a bit. Our function h , our “filter”, is a function $h(k)$ of integers, and

$$(h \star f)(t) = \sum_{s=-\infty}^{\infty} h(s)f(t+s\delta)$$

where δ is a step size. To make this a proper “average” we need to normalize this by assuming that the total integral

$$\int_{-\infty}^{\infty} h(t)dt = 1$$

From this fancier point of view, the moving average over a window of length, say, 5, is the case where $h(k) = 1/5$ for $k = -2, -1, 0, 1, 2$ and is zero elsewhere. Then

$$(h \star f)(t) = \frac{f(t-2\delta) + f(t-\delta) + f(t) + f(t+\delta) + f(t+2\delta)}{5}$$

which is the usual average.

As an example, consider the technique called “exponential smoothing”, in which we replace the value of f at a time t with the average of some window of its past values, but we weight those past values with an exponentially decaying factor λ . We let

$$h(k) = \lambda^k(\lambda - 1)/(\lambda^{N+1} - 1)$$

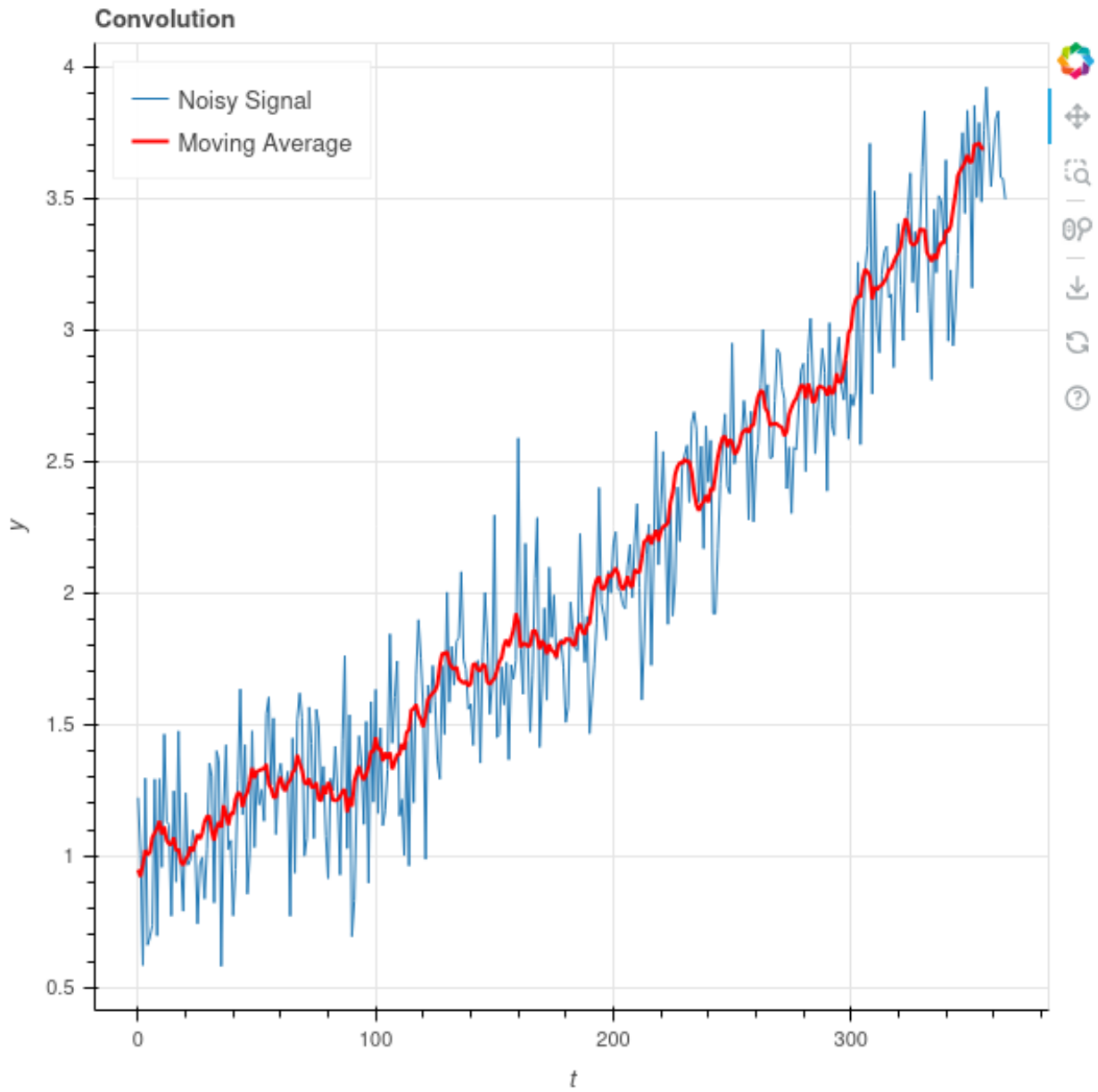


Figure 9.9: Moving Average of Noisy Signal

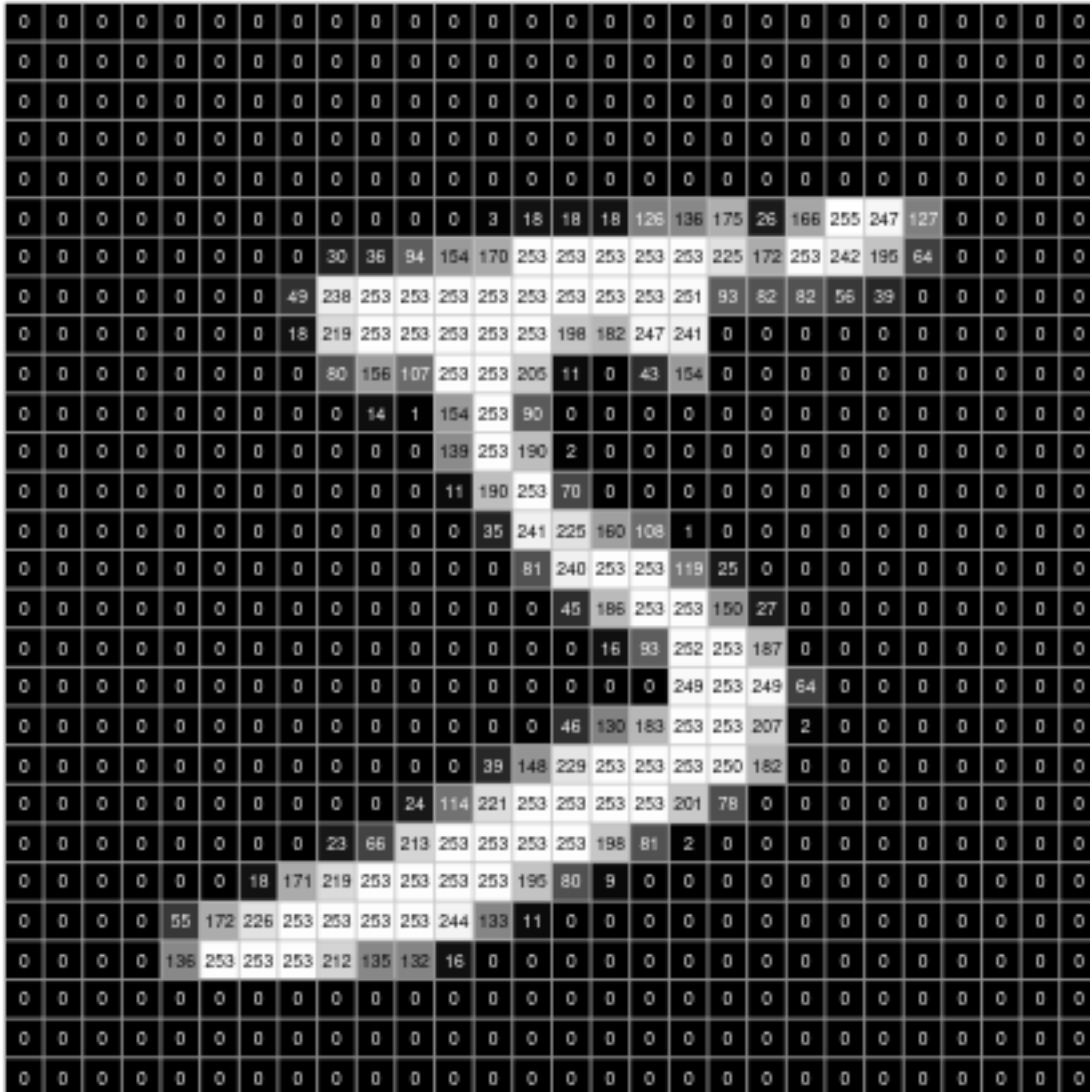
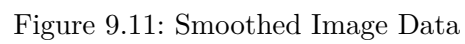


Figure 9.10: Sample Image Data



for $N \geq k \geq 0$ and $h(k) = 0$ for $k < 0$ and $k > N$.

The constant $(\lambda - 1)/(\lambda^{N+1} - 1)$ is chosen so that the sum of the values of h add up to one. From convolution we obtain:

$$(h \star f)(t) = \frac{\lambda - 1}{\lambda^{N+1} - 1} \sum_{k=0}^N \lambda^k f(t - k\delta).$$

A common version of exponential smoothing of a sequence y_i for $i = 1, \dots$, is to recursively define

$$z_1 = y_1$$

and, for $i = 2, \dots$,

$$z_i = \alpha y_i + (1 - \alpha)z_{i-1}.$$

This is equivalent to

$$z_i = \alpha \sum_{j=1}^i (1 - \alpha)^{j-1} y_{i-j+1}$$

This is equivalent to assuming that y_i is zero for $i < 1$ and defining h by

$$h(i) = \alpha(1 - \alpha)^i$$

for $i \geq 0$ and

$$h(i) = 0$$

for $i < 0$.

In the two-dimensional setting, our filter function h depends on two variables $h(x, y)$ and the convolution is

$$(h \star f)(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} h(a, b) f(x - a, y - b) da db$$

which reduces to a double sum in the discrete case. For example, we can let $h(x, y)$ be a Gaussian function

$$h(x, y) = \frac{1}{2\pi} e^{-(x^2 - y^2)/2}.$$

If we apply convolution with this Gaussian to the image studied earlier, we obtain the result in Figure 9.12.

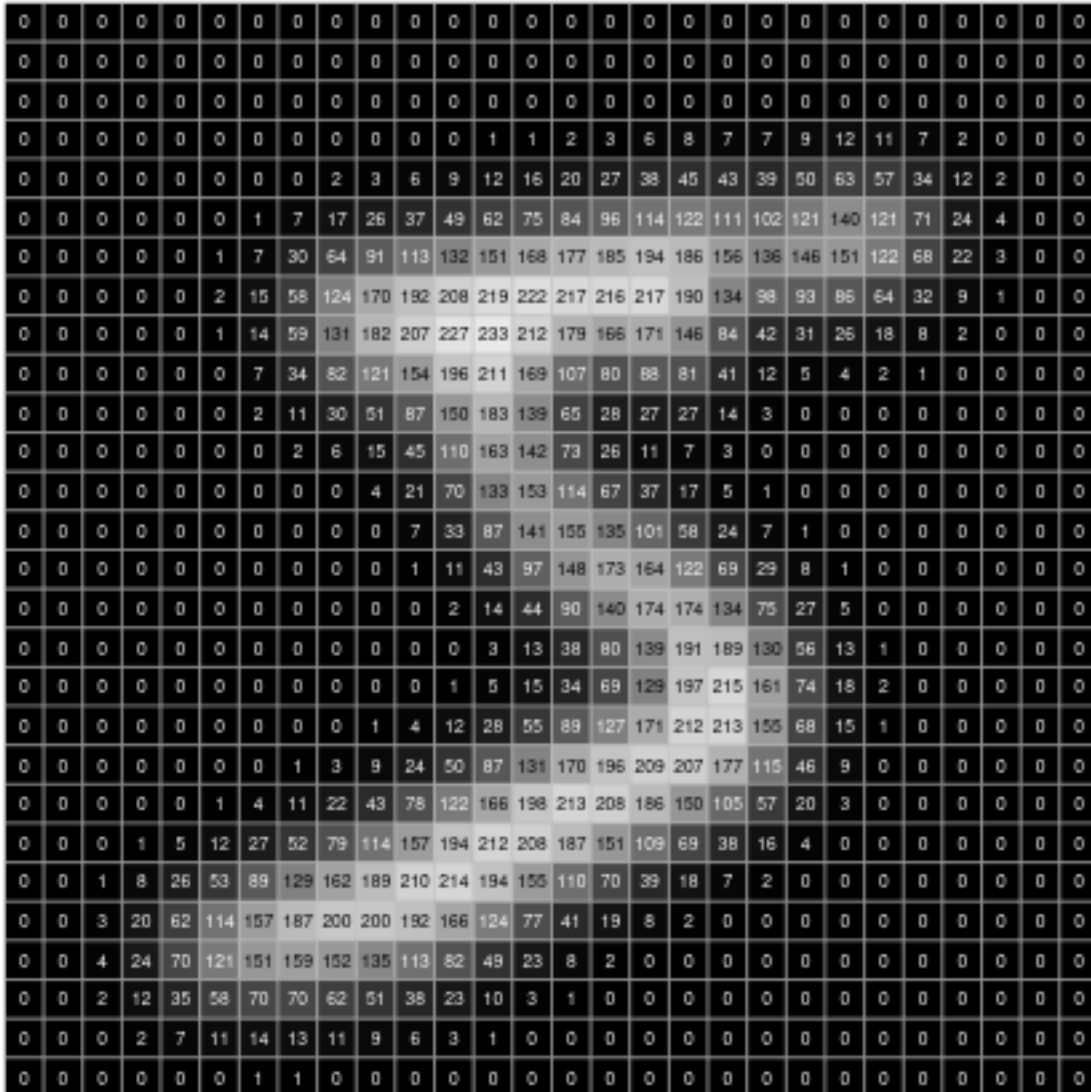


Figure 9.12: Gaussian Smoothing

9.6.1 Discrete Convolution for Image Data

Suppose that we have an image represented by a numerical array X of size $n \times m$. We can represent a discrete $2d$ convolutional filter as another array K of size $k \times k$. (K doesn't need to be square, but for simplicity we assume it is). We obtain the convolution Z of K and X by “sliding” K over the array X and computing the values of Z as the dot product of the entries in K with the different pieces of X .

For example, suppose that X is 10×10 and K is 3×3 . The first row of Z comes from sliding K across the top of X and computing the dot products:

- The first entry Z_{11} is the dot product of the upper left 3×3 submatrix of X with K .
- The second entry Z_{12} is the dot product of the submatrix coming from columns 2, 3, 4 and rows 1, 2, 3 of X with K .
- The third entry Z_{13} is the dot product of the submatrix with columns 3, 4, 5 and rows 1, 2, 3...

and so on. The last entry comes from the dot product of the submatrix with columns 8, 9, 10 and rows 1, 2, 3 with K .

Since only 8 copies of K “fit” across the 10 columns of X , the convolution Z only has 8 columns. You continue this down the rows as well, so that Z ends up being a 8×8 matrix.

In general, if X is $n \times m$ and K is $k_1 \times k_2$, then Z_{ij} is the dot product of the submatrix of X with columns $j, j+1, j+2, \dots, j+k_2-1$ and rows $i, i+1, i+2, \dots, i+k_1-1$ with K with $1 \leq i \leq n-k_1+1$ and $1 \leq j \leq m-k_2+1$. Thus Z will be $(n-k_1+1) \times (m-k_2+1)$.

9.6.1.1 Padding and Stride

There are two additional parameters that can be introduced to the convolution described above, called “padding” and “stride.”

Padding refers to what happens at the edges of the matrix X . A padding value of p adds p rows and columns of zeros all around X . One can also have a padding value of (p_1, p_2) which adds p_1 rows of zeros and p_2 columns of zeros.

Adding padding has the effect of increasing the effective size of X from $n \times m$ to $(n+2p_1) \times (m+2p_2)$, so the size of Z will be $(n+2p_1-k_1+1) \times (m+2p_2-k_2+1)$.

Stride means that instead of sliding the kernel over by 1 at each step, you slide it over by s_1 steps in the horizontal direction and s_2 steps in the vertical direction.

So if X is 10×10 , K is 3×3 , and the stride is 2 in each direction, then Z_{11} is the dot product of K with the submatrix of X with rows 1, 2, 3 and columns 1, 2, 3. But Z_{12} is the dot product of K with the submatrix of X with rows 1, 2, 3 and columns 3, 4, 5.

Finally, if X is $n \times m$, K is $k_1 \times k_2$, the padding is (p_1, p_2) , and the stride is (s_1, s_2) then the size of the convolution Z is $z_1 \times z_2$ where

$$z_1 = \frac{n + 2p_1 - k_1}{s_1} + 1 \quad (9.2)$$

and

$$z_2 = \frac{m + 2p_2 - k_2}{s_2} + 1. \quad (9.3)$$

9.6.2 Pooling

Pooling is a variant of 2d-convolution where, instead of computing the dot product of a filter with a submatrix of the data, one simply takes the maximum of the entries in the submatrix.

So for example if X is $n \times m$, and one is given a size $k_1 \times k_2$, and a stride (s_1, s_2) , the entry Z_{ij} of the pooled matrix is the maximum of the entries in the submatrix of X with rows $1 + (i - 1)s_1, \dots, (i - 1)s_1 + k_1 + 1$ and columns $(j - 1)s_2 + 1, \dots, (j - 1)s_2 + k_2 + 1$.

If pooling (p_1, p_2) also added, then you add p_1 rows of $-\infty$ at the top and bottom, and p_2 columns of $-\infty$ at the left and right of X .

The size of the pooled matrix is given by the same formulae as the convolution.

9.7 Convolutional Neural Networks

A convolutional neural network is a multi-layer neural net that incorporates layers that carry out a discrete convolution on their inputs. They have shown themselves to be extremely effective tools for a wide range of problems, particularly image recognition. We will develop the key ideas behind two-dimensional convolutional networks.

9.7.1 Convolution Layers

A standard 2d convolutional layer is described by numbers c_{in} and c_{out} , the number of input and output channels respectively, together with a kernel size (k_1, k_2) , stride parameters (s_1, s_2) , and padding parameters (p_1, p_2) . The kernel size, stride, and padding parameters tell us how an input array of size $n \times m$ is transformed via the formulae Equation 9.2 and Equation 9.3.

The channel numbers c_{in} and c_{out} add a “third dimension” to this picture. The layer takes an input of size $c_{in} \times n \times m$ – a “three dimensional” matrix that stacks c_{in} matrices of size $n \times m$ – and produces an output of size $c_{out} \times z_1 \times z_2$, a “three dimensional” matrix stacking c_{out} matrices of size $z_1 \times z_2$ where the z_i are computed from the filter size, stride, and padding parameters.

To carry out the transformation, the layer has c_{out} filters, each of size $c_{in} \times k_1 \times k_2$. For each $j = 1, \dots, c_{out}$, the layer computes the convolution of the $c_{in} \times k_1 \times k_2$ filter against the input of size $c_{in} \times n \times m$. This is done by computing the convolution of the filter in level i (for $i = 1, \dots, c_{in}$) against the i^{th} level of the input 3d-matrix and then *summing up* the results of this c_{in} convolutions. This yields the output of size $c_{out} \times z_1 \times z_2$.

To further complicate the picture, in practice the convolution layer also includes a bias term for each output layer, which is added to the sum of the convolutions at each level of the output.

Specifically, if our layer has c_{out} output channels, stride 1, padding 0, and kernel size $k \times k$, and our input has c_{in} input channels and is size $n \times m$, then the result of the convolution $Z_{c,i,j}$ is:

$$Z_{c,i,j} = b_c + \sum_{c'=1}^{c_{in}} \sum_{p=1}^k \sum_{q=1}^k W_{c,c',p,q} \cdot X_{c',i+p-1,j+q-1}$$

where $1 \leq c \leq c_{out}$, $1 \leq i \leq n - k + 1$ and $1 \leq j \leq m - k + 1$.

A convolution layer with c_{in} input layers, c_{out} output layers, and a kernel size of k_1, k_2 will have $c_{in}c_{out}k_1k_2$ total entries and can be thought of as a 4d-matrix of size $c_{out} \times c_{in} \times k_1 \times k_2$.

The term *tensor* is used for matrices that may have more than 2 dimensions; so a convolution layer is an example of a tensor.

9.7.2 Pooling Layers

A pooling layer applies a pooling operation to its input. As with convolution, the pooling operation is specified by its extent k_1, k_2 , its stride s_1, s_2 , and the padding p_1, p_2 . A pooling layer does not modify the number of channels, so it takes an input of size $c_{in} \times n \times m$ to an output of size $c_{in} \times z_1 \times z_2$ and applies the pooling filter separately to each of the c_{in} layers.

9.7.3 A neural net classifier

In figure Figure 9.13, we show the architecture of a neural network that does a very good job of classifying the images in the CIFAR-10 database. The network is taken from the pytorch documentation (see Training a Classifier); the graphic was drawn by Claude.

Let's walk through this neural network. The input layer at the right takes an image from the CIFAR-10 dataset, which consists of 32×32 color images. These are represented as 3d-matrices (tensors) of size $3 \times 32 \times 32$ where each of the three layers gives the intensity of the Red, Green, and Blue channels (RGB).

The image is fed into the first convolutional layer, which has 3 input channels, 6 output channels, and kernels of size 5. This transforms the $3 \times 32 \times 32$ image into a $6 \times 28 \times 28$ tensor. Then the Relu non-linear function is applied. Relu is the function which does nothing to positive numbers, and sets all negative numbers to zero.

After Relu, the $6 \times 28 \times 28$ tensor is fed into a 2×2 max pooling layer, which reduces the size to $6 \times 14 \times 14$. This is fed into a second convolutional layer, having 6 input channels and 16 output channels and a kernel size of 5. This yields a $16 \times 10 \times 10$ tensor.

We apply Relu again, then the max pool operation with filter size 2×2 , and obtain a tensor of size $16 \times 5 \times 5$.

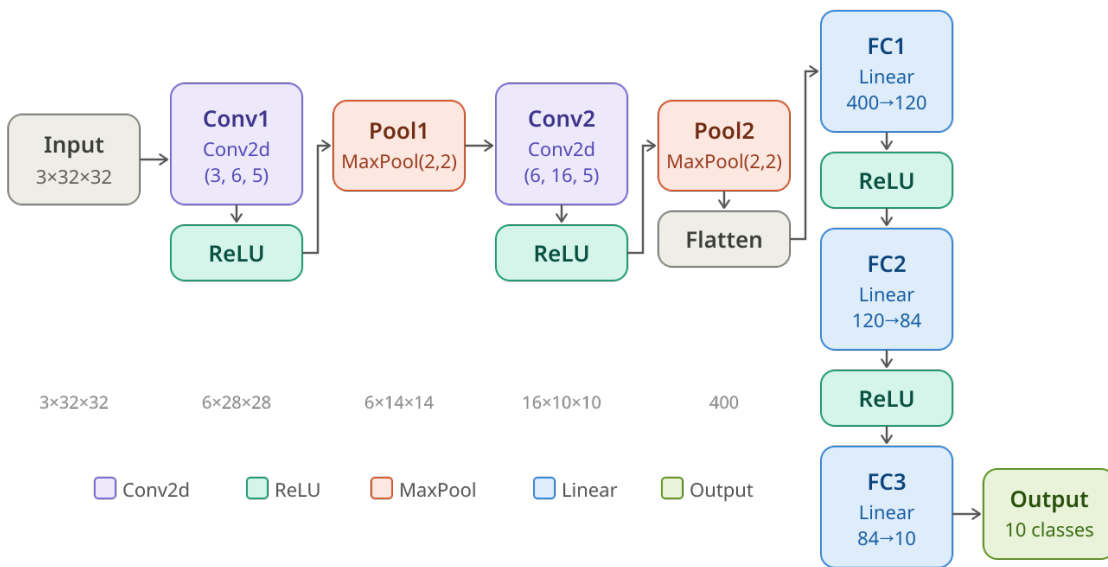


Figure 9.13: Neural Net Classifier

To carry out the final stages of the neural network, we “flatten” our $16 \times 5 \times 5$ tensor into a single vector of length $(16)(5)(5) = 400$. Then we use a “linear layer”, which is just a matrix of size 120×400 together with a bias term – a vector length 120 – to convert this vector into a vector of length 120.

We apply Relu to that vector, then multiply by another matrix of size 84×120 plus a bias, then another Relu, then another matrix of size 10×84 plus a bias.

At this point we have 10 numbers. We apply softmax to these ten numbers to get the 10 probabilities corresponding to the 10 possible classes in the CIFAR-10 dataset.

9.7.4 Training the network

To train the classifier, we use the multiclass entropy loss (the same loss function we use for multiclass linear regression). We initialize all of the weights in all of the layers randomly, and then use the backpropagation algorithm to adjust the weights and reduce the loss until we reach a point where the network performs well (or we give up).

9.7.5 Counting the weights

This neural network is an example of “deep learning” although by modern standards it is very small. Nevertheless it is instructive to count the number of weights, or unknown parameters, in the network.

A convolutional layer of size 3, 6, 5 has 3 filters of size 5×5 for each output channel, plus a bias term for each output channel, yielding $18 * 25 + 6 = 456$ weights.

The convolutional layer of size 6, 16, 5 has $6 \times 16 \times 25 + 16 = 2416$ weights.

The “linear” layers have $400 \times 120 + 120 = 48120$, $120 \times 84 = 10164$, and $84 \times 10 + 10 = 850$ weights.

All together we have:

$$456 + 2416 + 48120 + 10164 + 850 = 62006$$

parameters.

This is a lot, but notice that the convolutional layers have relatively few weights, so they help to keep the network small compared to using more linear layers.

The 2015 paper “Very Deep Convolutional Networks for Large-Scale Image Recognition”, by Simonyan and Zisserman, outlines the author’s work on CNN’s that led them to win the “ImageNet Large Scale Visual Recognition Challenge” in 2014. This involved working with a dataset of 150000 images and 1000 image categories, and included both classification (saying what the image *was*) and localization (putting a box around the main thing in the image). See Simonyan and Zisserman and ILSVRC2014.

References

- [1] U.C. IRVINE ML REPOSITORY. Auto MPG Dataset. Available at <http://archive.ics.uci.edu/ml/datasets/Auto+MPG>.
- [2] BERTSEKAS, D. P. and TSITSIKLIS, J. N. (2008). *Introduction to probability*. Athena Scientific.
- [3] U.C. IRVINE ML REPOSITORY. Sentiment Labelled Sentences Data Set. Available at <https://archive.ics.uci.edu/ml/datasets/Sentiment+Labelled+Sentences>.
- [4] JACK DAOUD. Marketing Analytics. Available at <https://www.kaggle.com/datasets/jackdaoud/marketing-data>.
- [5] LECUN, Y., CORTES, C. and BURGESS, C. The MNIST Database. Available at <http://yann.lecun.com/exdb/mnist/>.
- [6] BOSER, B., GUYON, I. and VAPNIK, V. A training algorithm for optimal margin classifiers. In *Colt '92: Proceedings of the fifth annual workshop on computational learning theory* (D. Haussler, ed) pp 144–52. ACM.
- [7] BENNETT, K. P. and BREDENSTEINER, E. J. (2000). Duality and geometry in SVM classifiers. In *Proceedings of the seventeenth international conference on machine learning* (P. Langley, ed). Morgan Kaufmann Publishers.
- [8] GORMAN, K. B., WILLIAMS, T. D. and FRASER, W. R. (2014). Ecological sexual dimorphism and environmental variability within a community of antarctic penguins (genus *pygoscelis*). *PLoS ONE* **9(3)** –13.
- [9] HORST, A. Palmer penguins. Available at <https://github.com/allisonhorst/palmerpenguins>.
- [10] PLATT, J. C. (1998). *Sequential minimal optimization: A fast algorithm for training support vector machines*. Microsoft Research.
- [11] SCHÖLKOPF, B., BURGESS, C. and SMOLA, A. (1998). *Advances in kernel methods: Support vector learning*. MIT Press.

